

UT02 · Páginas dinámicas con Spring Boot y Thymeleaf

Material de la unidad de trabajo

Ciclo	Desarrollo de Aplicaciones Web (DAW)
Curso	2.º
Módulo	0613 · Desarrollo web en entorno servidor
Curso académico	2026/2027
Autor	Francisco Javier Luque Terriza
Fecha de edición	01/10/2026 09:33

ÍNDICE

1. Del servidor al navegador.....	2
2. Controladores, rutas y vistas.....	21
3. Contenido dinámico	41
4. Listas y parámetros	63
5. Formularios.....	78
6. Validación de formularios	99
7. Aplicación completa.....	119
Ejercicios complementarios	134
Soluciones de los ejercicios complementarios	140
Respuestas de la autoevaluación.....	146
Bibliografía.....	153
Cumplimiento de RA y CE.....	154

1. Del servidor al navegador

Una página dinámica se construye en el servidor en el momento en que alguien la pide. Antes de programar ninguna, conviene saber quién hace qué entre el navegador y el servidor, y tener listo el primer proyecto de Spring Boot.

Cliente y servidor

Cada vez que escribes una dirección en el navegador empieza una conversación entre dos programas: el navegador pide y un servidor responde. Lo que el servidor devuelve casi siempre es HTML, y ese HTML puede estar guardado tal cual en un fichero o fabricarse en ese mismo instante.

DEFINICIONES

Cliente: el programa que pide las páginas y las muestra; en la web, el navegador. Ejecuta HTML, CSS y JavaScript en el ordenador del usuario.

Servidor: el programa que recibe las peticiones y devuelve las respuestas. Se ejecuta en otra máquina (o en la tuya, mientras desarrollas).

Petición y respuesta HTTP: el cliente envía una petición con un método (GET para pedir, POST para enviar datos) y una ruta (/hora); el servidor contesta con un código de estado (200 si todo va bien, 404 si no existe) y el contenido.

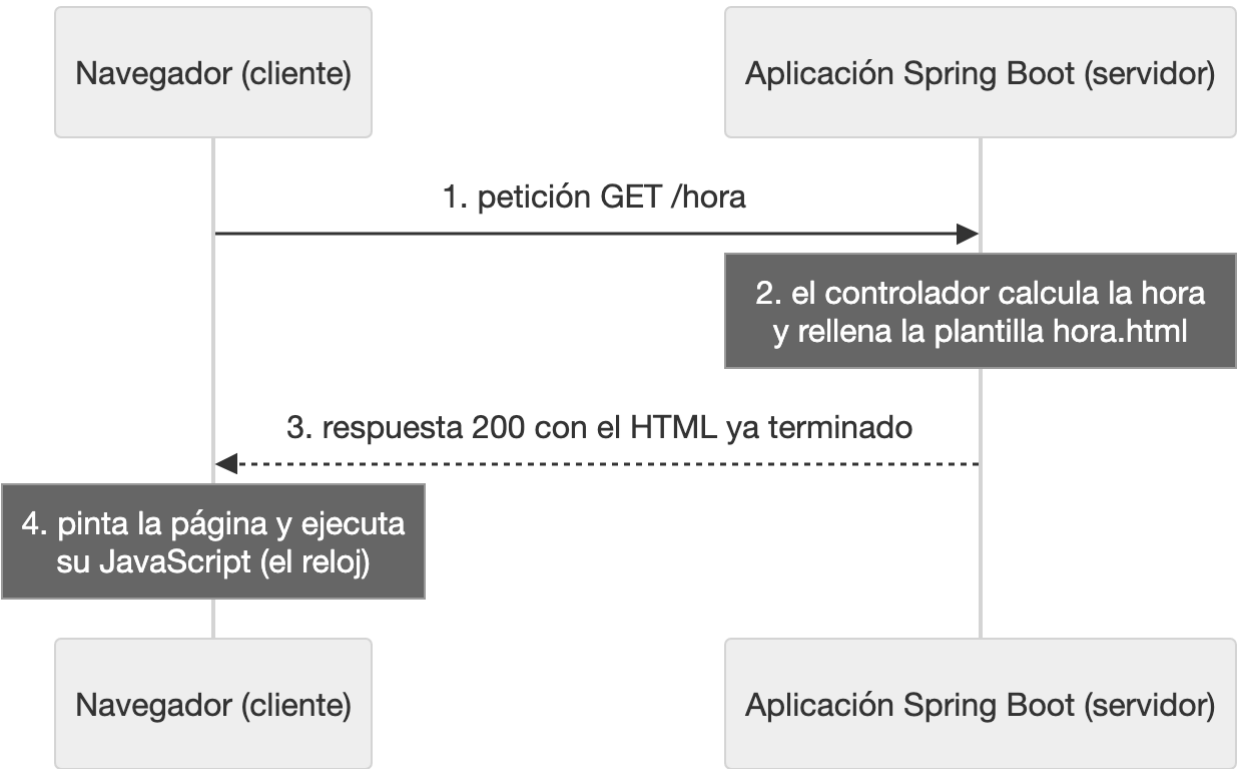
Página estática: un fichero HTML que el servidor entrega sin cambiarlo; todos los que la piden reciben lo mismo.

Página dinámica: una página que el servidor genera en cada petición, con datos que pueden cambiar (la hora, el usuario, lo que hay en una base de datos).

EJEMPLO

El viaje de una página dinámica

Así se resuelve la petición de la página /hora que verás en el apartado siguiente:



El navegador envía GET /hora; en el servidor, el controlador calcula la hora y rellena la plantilla; el servidor responde 200 con el HTML terminado; el navegador pinta la página y ejecuta su JavaScript

El navegador nunca ve cómo se fabricó la página: recibe HTML terminado, igual que si fuera un fichero estático.

REFERENCIA

Qué se ejecuta en cada lado

	En el servidor	En el cliente (navegador)
Lenguajes	Java, PHP, Python, C#, JavaScript con Node.js...	HTML, CSS y JavaScript
Cuándo se ejecuta	una vez por petición, antes de enviar la página	cuando la página ya ha llegado, y mientras sigue abierta
A qué tiene acceso	bases de datos, ficheros y datos privados del servidor	la página, el navegador y lo que haga el usuario
Quién ve el código	nadie de fuera: solo llega el resultado	cualquiera, con «Ver código fuente»
Para qué sirve	decidir qué datos se muestran, comprobarlos y guardarlos	responder al instante a clics y teclas sin pedir otra página

Combinar servidor y cliente

Una aplicación web real usa los dos lados a la vez. El servidor pone los datos que solo él conoce o en los que no se puede confiar al cliente; el navegador añade la interactividad que no necesita volver a preguntar al servidor.

EJEMPLO

Una página, dos relojes

Todavía no sabes crear un proyecto de Spring Boot (eso llega en el apartado «Crear el proyecto en Spring Tool Suite», más abajo) — este ejemplo es solo para que veas la pinta que tienen los dos ficheros que lo resuelven, antes de montarlo tú mismo más adelante. La página /hora muestra la hora a la que el servidor la generó y, debajo, un reloj de JavaScript que corre en el navegador. El atributo `th:text` lo resuelve el servidor; el `<script>`, el cliente.

Fichero `src/main/resources/templates/hora.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>¿Qué hora es?</title>
<style>
  body { font-family: sans-serif; margin: 24px; }
  strong { color: #1565c0; }
</style>
</head>
<body>
<h1>¿Qué hora es?</h1>
<p>El servidor generó esta página a las <strong th:text="${horaServidor}">00:00:00</strong>.</p>
<p>En tu navegador son las <strong id="reloj">--:--:--</strong>.</p>
<script>
  const reloj = document.getElementById('reloj');
  function actualiza() {
    reloj.textContent = new Date().toLocaleTimeString('es-ES');
  }
  actualiza();
  setInterval(actualiza, 1000);
</script>
</body>
</html>
```

Fichero `src/main/java/com/ejemplo/primerproyecto/HoraController.java`:

```
package com.ejemplo.primerproyecto;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

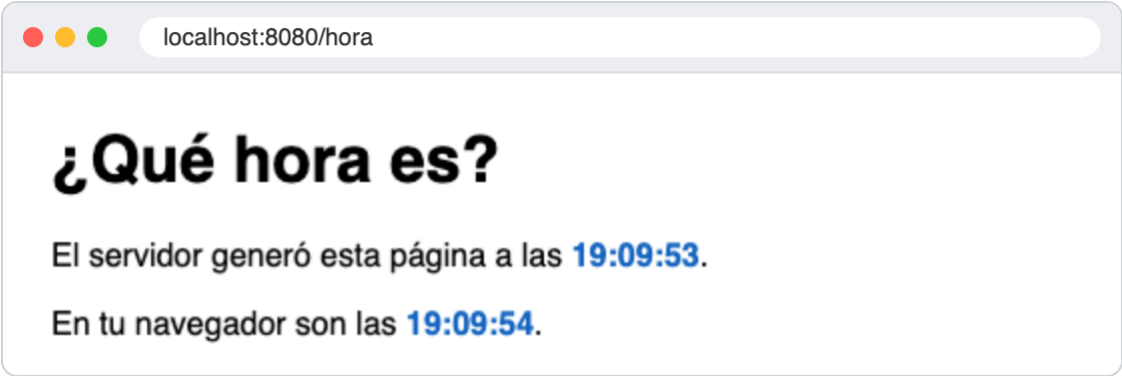
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
```

```
public class HoraController {  
  
    @GetMapping("/hora")  
    public String hora(Model model) {  
        String hora =  
        LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"));  
        model.addAttribute("horaServidor", hora);  
        return "hora";  
    }  
}
```

El controlador y el objeto `Model` son el tema de la sección siguiente; aquí basta con ver que el servidor calcula la hora y se la pasa a la plantilla con el nombre `horaServidor`.

Así se vería en el navegador, con la primera hora fija y la segunda avanzando cada segundo:



Página ¿Qué hora es?: el servidor generó esta página a las 19:09:53; en tu navegador son las 19:09:54

Si en ese momento se abriera «Ver código fuente» (Ctrl+U), se vería que al navegador le llega la hora ya escrita y ningún `th:text`:

```
<p>El servidor generó esta página a las <strong>19:09:52</strong>.</p>  
<p>En tu navegador son las <strong id="reloj">--:--:--</strong>.</p>
```

REFERENCIA

Ventajas de combinarlos

Ventaja	En el ejemplo y en una aplicación real
Datos fiables	la hora del servidor no depende del reloj del usuario; igual que un precio o una nota, que el usuario no puede cambiar
Respuesta inmediata	el reloj avanza sin pedir nada al servidor; igual que un menú desplegable o un aviso mientras escribes
Menos carga y menos tráfico	lo que resuelve el navegador no ocupa al servidor ni viaja por la red
Seguridad	lo importante se decide y se comprueba en el servidor: el código del cliente se puede ver y saltar

Ventaja	En el ejemplo y en una aplicación real
Funciona siempre	el contenido llega ya escrito en el HTML, aunque el navegador tenga JavaScript desactivado o sea un buscador

AVISO

Lo que se comprueba en el navegador, en un formulario (un campo obligatorio, un formato de correo), ayuda al usuario, pero no protege nada: cualquiera puede enviar una petición sin pasar por tu página. Por eso los datos se vuelven a comprobar siempre en el servidor.

Tecnologías de servidor con código embebido

Escribir con Java un `String` con todo el HTML de la página, etiqueta a etiqueta, es incómodo y fácil de romper: basta olvidar una comilla o cerrar mal una etiqueta para que el HTML que llega al navegador quede mal formado, sin que el compilador avise de nada. Por eso casi todas las tecnologías de servidor trabajan al revés: escribes el HTML y metes dentro, en unos pocos sitios marcados, el código o las expresiones que el servidor sustituye al generar la página.

DEFINICIONES

Plantilla: un fichero HTML con huecos marcados que el servidor rellena con datos en cada petición.

Motor de plantillas: la librería del servidor que lee la plantilla, sustituye los huecos y produce el HTML final.

Plantilla natural: un fichero que, aunque tenga marcas especiales para el servidor, sigue siendo HTML válido de verdad — se puede abrir con el navegador sin ningún servidor detrás, y se ve bien, con los valores de ejemplo que haya escrito quien la diseñó en vez de los datos reales. Es la idea central de Thymeleaf; lo verás con un ejemplo real un poco más abajo.

REFERENCIA**Las más usadas**

Tecnología	Lenguaje	Cómo se escribe la hora	Dónde se usa
JSP	Java	<code>\${hora}</code> o <code><%= hora %></code>	aplicaciones Java EE / Jakarta EE clásicas
PHP	PHP	<code><?= \$hora ?></code>	WordPress, Laravel y buena parte de la web
Razor	C#	<code>@hora</code>	ASP.NET Core
Jinja	Python	<code>{{ hora }}</code>	Flask (Django usa una sintaxis casi igual)

Tecnología	Lenguaje	Cómo se escribe la hora	Dónde se usa
EJS	JavaScript	<code><%= hora %></code>	Node.js con Express
Thymeleaf	Java	<code><strong th:text="\$ {hora}">00:00:00</code>	Spring Boot: la de esta unidad

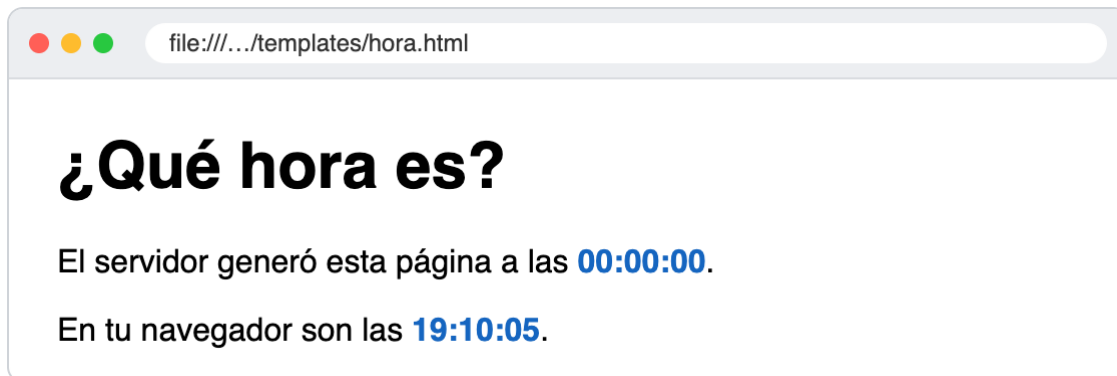
En todas, lo que va entre esas marcas lo ejecuta el servidor y al navegador solo le llega el resultado.

EJEMPLO

Una plantilla natural, sin servidor

Las marcas de JSP, PHP o EJS no son HTML: si abres esos ficheros sin su servidor, no se ven como una página. Thymeleaf, en cambio, usa atributos (`th:text`) que el navegador ignora, así que el valor de ejemplo que escribes dentro de la etiqueta hace de relleno.

Si en este momento abrieras `hora.html` con doble clic, sin arrancar nada, verías esto: el valor de ejemplo `00:00:00` ocupa el hueco del servidor, mientras que el reloj de JavaScript sí funciona, porque ese lo ejecuta el navegador, no el servidor:



La plantilla `hora.html` abierta como fichero: el servidor generó esta página a las 00:00:00; en tu navegador son las 19:10:05

Así, quien diseña la página puede trabajar con la plantilla directamente en el navegador, viéndola con esos valores de ejemplo, y quien programa la rellena después con datos reales, sobre el mismo fichero.

Crear el proyecto en Spring Tool Suite

Spring Boot es una forma de usar el framework Spring con casi toda la configuración ya hecha: incluye el servidor web (Tomcat) dentro de la aplicación, de modo que se arranca como un programa Java normal. Los proyectos se crean con un asistente que descarga una plantilla de proyecto de `start.spring.io`.

DEFINICIONES

Spring Tool Suite (STS): Eclipse preparado para Spring; su nombre actual es «Spring Tools for Eclipse». Es el entorno de esta unidad.

Maven: la herramienta que descarga las librerías del proyecto, lo compila y lo empaqueta; se configura en el fichero `pom.xml`.

Starter: una dependencia de Spring Boot que trae de una vez todo lo necesario para una tarea; por ejemplo, `spring-boot-starter-webmvc` para hacer aplicaciones web.

Tomcat: el servidor web que Spring Boot lleva dentro; escucha en el puerto 8080.

PASO A PASO**Instalar Spring Tool Suite, antes de empezar**

1. **Descargarlo ya preparado.** En spring.io/tools, la opción «Spring Tool Suite» trae Eclipse con todo lo de Spring ya instalado — se descarga, se descomprime y se abre, sin pasos adicionales.
2. **Añadirlo a un Eclipse que ya tengas.** Menú **Help** › **Eclipse Marketplace...**, buscar «Spring Tools 4» e instalar el plugin (**Install**, aceptar las licencias, reiniciar Eclipse cuando lo pida). Útil si ya usas Eclipse para otra cosa y no quieres un segundo IDE instalado aparte.

COMPRUEBA QUE FUNCIONA

Hay dos formas de tener STS listo, según de dónde partas:

Las dos opciones dejan el mismo resultado: el asistente **Spring Starter Project** disponible en el menú **File** › **New**, que es con el que se trabaja el resto de esta sección.

PASO A PASO**Crear un proyecto de Spring Boot**

1. En STS, menú **File** › **New** › **Spring Starter Project**. Si no aparece en la lista, está en **File** › **New** › **Other...** › **Spring Boot** › **Spring Starter Project**.
2. En la primera pantalla, rellena los datos del proyecto con los valores de la tabla «Pantalla 1 del asistente» y pulsa **Next**.
3. En la segunda pantalla, deja **Spring Boot Version** en la última versión estable, la que no lleva SNAPSHOT ni M (en esta unidad, 4.1.1).
4. En el buscador de dependencias (**Available**), marca estas tres: **Spring Web** (grupo Web), **Thymeleaf** (grupo Template Engines) y **Spring Boot DevTools** (grupo Developer Tools). Deben aparecer las tres en **Selected**.
5. Pulsa **Finish**. STS descarga el proyecto y sus librerías; la primera vez tarda un poco (la barra de progreso está abajo a la derecha).
6. Cuando termine, el proyecto aparece en el **Package Explorer** sin marcas rojas de error.

REFERENCIA

Pantalla 1 del asistente

Campo	Valor	Para qué
Service URL	https://start.spring.io	de dónde se descarga la plantilla (se deja)
Name	primer-proyecto	nombre del proyecto en STS
Type	Maven	herramienta de construcción
Packaging	Jar	la aplicación, con su Tomcat dentro, en un solo .jar
Java Version	21	versión de Java del proyecto; Spring Boot 4 necesita la 17 o superior
Language	Java	
Group	com.ejemplo	la organización, como un dominio al revés
Artifact	primerproyecto	identificador del proyecto para Maven
Version	0.0.1-SNAPSHOT	versión de tu proyecto (se deja)
Description	Primer proyecto con Spring Boot	texto libre
Package	com.ejemplo.primerproyecto	el paquete base, donde irá todo tu código Java

STS rellena algunos campos a partir de otros: revisa al final que **Package** quede exactamente así. Un paquete de Java no admite guiones, por eso primerproyecto va junto.

EJEMPLO

Lo que el asistente escribe en el pom.xml

El `pom.xml` es el fichero donde Maven guarda la configuración del proyecto: qué librerías usa, qué versión de Java, cómo se llama (ya se mencionó de pasada al definir Maven, más arriba). El asistente lo rellena por ti; estas son sus partes importantes:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
  <relativePath/>
</parent>
...
<properties>
  <java.version>21</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
...
</dependencies>

```

- **parent**: el proyecto hereda de `spring-boot-starter-parent`, que fija versiones compatibles entre sí para todas las librerías. Por eso las dependencias no llevan `<version>`.
- **Spring Web** se convierte en `spring-boot-starter-webmvc` (Spring MVC con Tomcat), **Thymeleaf** en `spring-boot-starter-thymeleaf` y **DevTools** en `spring-boot-devtools`, que solo actúa mientras desarrollas.
- Los dos starters que terminan en `-test` (omitidos arriba) los añade el asistente para las pruebas.

Fichero `pom.xml`:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/
xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>
  <groupId>com.ejemplo</groupId>
  <artifactId>primerproyecto</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>primer-proyecto</name>
  <description>Primer proyecto con Spring Boot</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>

```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>

```

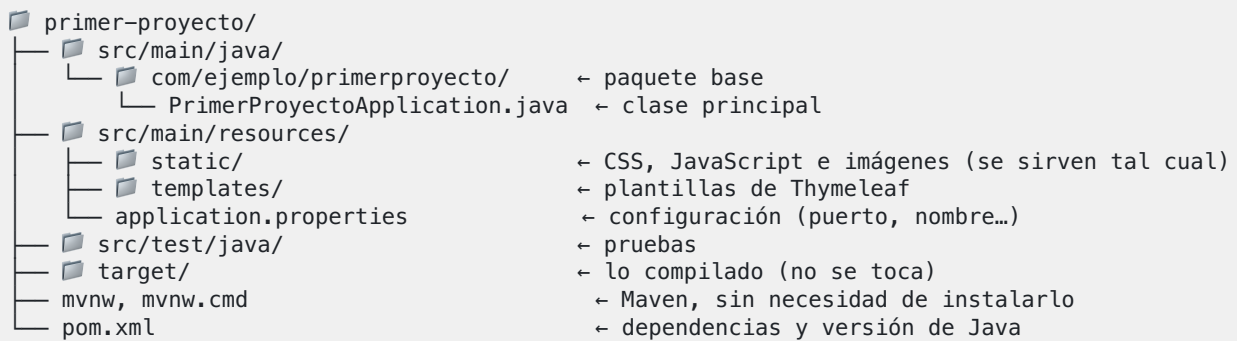
RECURSOS

Herramientas

- **Spring Tools:** descarga de Spring Tool Suite para Windows, macOS y Linux (en inglés).
<https://spring.io/tools>
- **Spring Initializr:** la web que usa el asistente; genera el mismo proyecto en un .zip si alguna vez lo necesitas fuera de STS (en inglés).
<https://start.spring.io/>

Qué hay dentro del proyecto

Todos los proyectos de Spring Boot tienen la misma estructura, así que en cuanto la conoces sabes dónde va cada cosa. En el Package Explorer, STS agrupa las carpetas de código como `src/main/java`, `src/main/resources` y `src/test/java`, y añade las librerías en «JRE System Library» y «Maven Dependencies».

**EJEMPLO****La clase principal**

El asistente crea una única clase de Java. Su método `main` arranca Spring, que a su vez arranca Tomcat y deja la aplicación esperando peticiones.

Fichero `src/main/java/com/ejemplo/primerproyecto/PrimerProyectoApplication.java`:

```

package com.ejemplo.primerproyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PrimerProyectoApplication {

    public static void main(String[] args) {
        SpringApplication.run(PrimerProyectoApplication.class, args);
    }

}
  
```

La anotación `@SpringBootApplication` activa la configuración automática (al ver Thymeleaf en el proyecto, Spring Boot lo configura solo) y la búsqueda de tus clases: Spring revisa el paquete de esta clase y todos sus subpaquetes.

AVISO

Tus clases tienen que estar en el paquete base o en uno de sus subpaquetes. Un controlador en `com.ejemplo.primerproyecto.controller` funciona; el mismo controlador en `com.ejemplo.controladores` compila sin errores, pero Spring no lo encuentra y su página da un 404.

AMPLIACIÓN**Qué agrupa @SpringBootApplication**

Es la suma de tres anotaciones de Spring.

- `@SpringBootConfiguration`: la clase puede declarar configuración propia.
- `@EnableAutoConfiguration`: Spring Boot configura lo que encuentra entre las dependencias (Tomcat, Spring MVC, Thymeleaf) con valores razonables, que puedes cambiar en `application.properties`.
- `@ComponentScan`: busca clases anotadas (`@Controller` y otras) en el paquete de la clase principal y sus subpaquetes.

Más detalle en la documentación de Spring Boot: [Structuring your code](#) (en inglés).

Arrancar y parar la aplicación

La aplicación se arranca desde STS como cualquier programa, pero no termina sola: se queda escuchando en el puerto 8080 hasta que la paras.

PASO A PASO**Arrancar la aplicación**

1. Clic derecho sobre el proyecto en el Package Explorer › **Run As › Spring Boot App**.
2. Mira la vista **Console**: tras el logotipo de Spring aparecen unas líneas de registro. La aplicación está lista cuando sale la línea `Started PrimerProyectoApplication`:

```

  ____ _
 / ___ \| | | |
 \___ \| |_| |
  ___) | __| |
 |____|_|_|_|

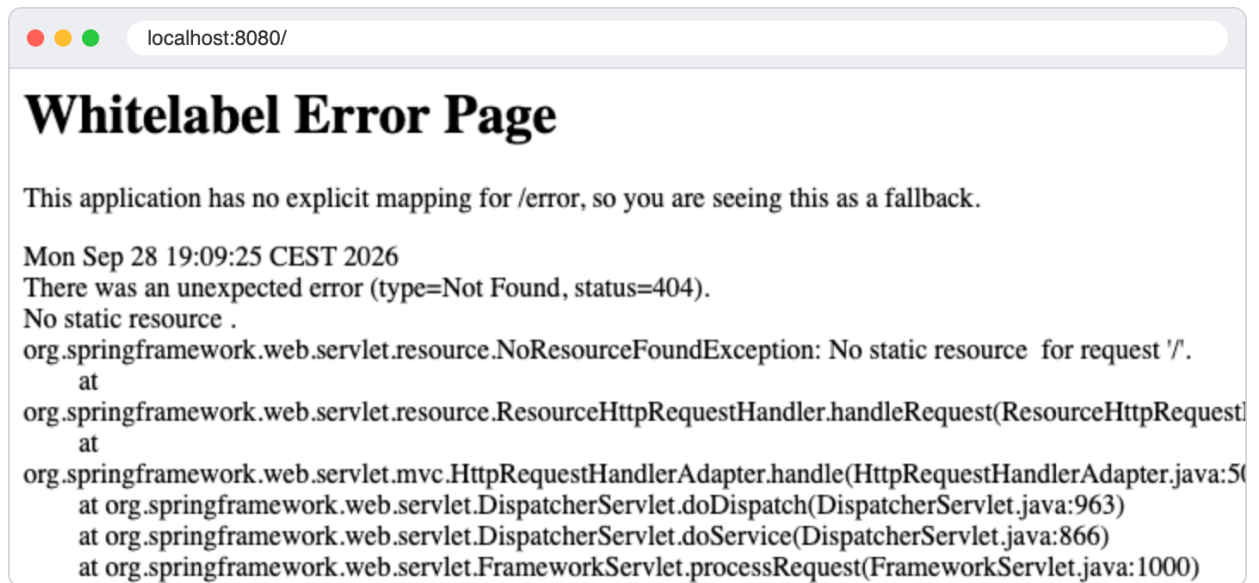
:: Spring Boot ::                (v4.1.1)

2026-09-28T19:09:24.538+02:00 INFO 3345 --- [primer-proyecto] [ restartedMain]
    o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080
    (http)
2026-09-28T19:09:24.624+02:00 WARN 3345 --- [primer-proyecto] [ restartedMain]
    ion$DefaultTemplateResolverConfiguration : Cannot find template location:
    classpath:/templates/ (please add some templates, check your Thymeleaf
    configuration, or set spring.thymeleaf.check-template-location=false)
2026-09-28T19:09:24.652+02:00 INFO 3345 --- [primer-proyecto] [ restartedMain]
    o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with
    context path '/'
2026-09-28T19:09:24.654+02:00 INFO 3345 --- [primer-proyecto] [ restartedMain]
    c.e.p.PrimerProyectoApplication          : Started PrimerProyectoApplication in
    0.488 seconds (process running for 0.614)

```

La línea `WARN ... Cannot find template location` solo avisa de que `templates/` está vacía; desaparece en cuanto creas la primera plantilla.

3. Abre `http://localhost:8080/` en el navegador. Todavía no hay ninguna página, así que Spring responde con su página de error 404:



Whitelabel Error Page con el error `type=Not Found, status=404` y el aviso `No static resource`

Es la respuesta esperada: el servidor funciona, pero en `/` no hay nada. Con DevTools activo, la página añade el detalle técnico del error.

4. Para detener la aplicación, pulsa el cuadrado rojo (**Terminate**) de la vista Console, o usa el botón de parar del **Boot Dashboard**.

EJEMPLO

DevTools: cambiar sin volver a arrancar

Con la aplicación en marcha, no hace falta pararla y arrancarla después de cada cambio.

- **Al guardar un `.java`**, STS lo compila y DevTools reinicia la aplicación en un momento. En la consola aparece `Restarting due to 1 class path change`.
- **Al guardar una plantilla** de `templates/` o un fichero de `static/`, basta con recargar el navegador: DevTools desactiva la caché de Thymeleaf mientras desarrollas.

AVISO

No arranques una aplicación que ya está en marcha: la segunda no puede usar el puerto 8080 y se detiene con este mensaje en la consola: Para la que estaba en marcha (en la vista Console, o en el Boot Dashboard) y vuelve a arrancar. Si necesitas dos aplicaciones a la vez, cambia el puerto de una en su `application.properties`, por ejemplo con `server.port=8081`.

RECURSOS

Para saber más

- **Visión general de HTTP:** peticiones, respuestas, métodos y códigos de estado, en MDN.
<https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
- **Developer Tools:** qué reinicia y qué no, y cómo configurarlo (en inglés).
<https://docs.spring.io/spring-boot/reference/using/devtools.html>
- **Tutorial de Thymeleaf:** la referencia del motor de plantillas, con el concepto de plantilla natural (en inglés).
<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>

Práctica

Tres ejercicios para crear proyectos con soltura, configurarlos y servir tus primeras páginas.

ENTRENAMIENTO 1.1

Otro puerto

Nivel · Fácil

Crea el proyecto `primer-proyecto` siguiendo los pasos de esta sección y haz que escuche en el puerto 8081 en lugar del 8080. Arráncalo y comprueba en la consola y en el navegador que responde en `http://localhost:8081/`.

PISTAS

- La configuración va en `src/main/resources/application.properties`, una propiedad por línea con la forma `nombre=valor`.
- La propiedad del puerto es `server.port`.

RESULTADO ESPERADO

En el navegador, `http://localhost:8081/` muestra la página de error 404 de antes, y en la consola la línea de Tomcat indica el nuevo puerto:

```
2026-09-28T19:10:54.770+02:00 INFO 5428 --- [primer-proyecto] [ restartedMain]
o.s.boot.tomcat.TomcatWebServer      : Tomcat started on port 8081 (http) with
context path '/'
```

UNA SOLUCIÓN

Fichero `src/main/resources/application.properties`:

```
spring.application.name=primer-proyecto
server.port=8081
```

ENTRENAMIENTO 1.2

Una portada estática

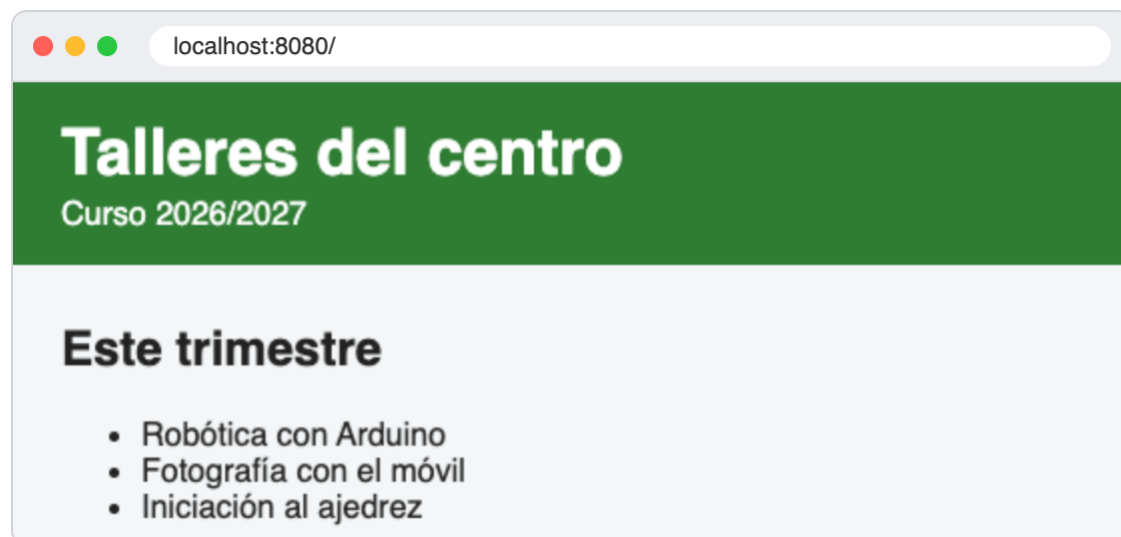
Nivel · Intermedio

Haz que `http://localhost:8080/` deje de dar el error 404 y muestre la portada de los talleres del centro: una cabecera verde con el título y el curso, y la lista de talleres del trimestre. Los estilos van en su propio fichero CSS, no dentro del HTML.

PISTAS

- Lo que pongas en `src/main/resources/static/` se sirve tal cual, y un `index.html` en esa carpeta es la página de `/`.
- Una hoja de estilos en `static/css/estilos.css` se enlaza como `/css/estilos.css`.
- Si la aplicación estaba en marcha, recarga el navegador después de guardar.

RESULTADO ESPERADO



Portada Talleres del centro, curso 2026/2027, con la cabecera verde y la lista Este trimestre: Robótica con Arduino, Fotografía con el móvil e Iniciación al ajedrez

UNA SOLUCIÓN

Fichero `src/main/resources/static/index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" href="/css/estilos.css" />
</head>
<body>
  <header>
    <h1>Talleres del centro</h1>
    <p>Curso 2026/2027</p>
  </header>
  <main>
```

```
<h2>Este trimestre</h2>
<ul>
  <li>Robótica con Arduino</li>
  <li>Fotografía con el móvil</li>
  <li>Iniciación al ajedrez</li>
</ul>
</main>
</body>
</html>
```

Fichero `src/main/resources/static/css/estilos.css`:

```
body {
  margin: 0;
  font-family: sans-serif;
  background: #f4f6f8;
  color: #222;
}

header {
  background: #2e7d32;
  color: white;
  padding: 16px 24px;
}

header h1 {
  margin: 0;
}

header p {
  margin: 4px 0 0;
}

main {
  padding: 8px 24px;
}
```

ENTRENAMIENTO 1.3

Dos aplicaciones a la vez

Nivel · Difícil

Crea dos proyectos de Spring Boot, `tienda` y `admin`, y tenlos arrancados a la vez. Cada uno tiene una portada que dice qué puerto la sirve y enlaza con la portada del otro, de modo que se pueda ir de uno a otro con los enlaces.

PISTAS

- En el asistente cambian Name, Artifact y Package (`com.ejemplo.tienda`, `com.ejemplo.admin`).
- Dos aplicaciones no pueden escuchar en el mismo puerto.
- Un enlace a otra aplicación lleva la dirección completa, con su puerto:
`http://localhost:8081/`.

RESULTADO ESPERADO

Portada Tienda en localhost:8080, con el enlace Ir a la administración



Portada Administración en localhost:8081, con el enlace Volver a la tienda

UNA SOLUCIÓN

En el Boot Dashboard (o en la vista Console) deben verse las dos aplicaciones en marcha. Solo cambian la configuración y las portadas.

PÍLDORA**Del asistente al navegador en 5 pasos**

1. **File › New › Spring Starter Project:** Maven, Jar, Java 21 y el paquete base.
2. Dependencias: Spring Web, Thymeleaf y Spring Boot DevTools.
3. **Run As › Spring Boot App** y espera a Started ... en la consola.
4. Abre `http://localhost:8080/`: un 404 significa que el servidor responde.
5. Para la aplicación antes de volver a arrancarla.

AUTOEVALUACIÓN**Del servidor al navegador****1.1** ¿Qué es una página dinámica?

- a) Una página con animaciones de CSS.
- b) Una página que el servidor genera en cada petición, con datos que pueden cambiar.
- c) Una página con JavaScript.
- d) Un fichero HTML que el servidor entrega sin cambiarlo.

1.2 ¿Dónde se ejecuta el código de un `<script>` de una página?

- a) En el servidor, antes de enviar la página.
- b) En el navegador, cuando la página ya ha llegado.
- c) En los dos, una vez en cada uno.
- d) En la base de datos.

1.3 En la página `/hora`, ¿por qué la hora del servidor no avanza y la del navegador sí?**1.4** Un usuario abre «Ver código fuente» en la página `/hora`. ¿Qué ve donde la plantilla tenía `th:text="${horaServidor}"`?

- a) El atributo `th:text` tal cual.
- b) El código Java del controlador.
- c) La hora ya escrita dentro de la etiqueta, sin ningún `th:text`.
- d) `00:00:00`.

1.5 ¿Por qué un precio o una nota deben calcularse y comprobarse en el servidor y no solo en el navegador?**1.6** Di dos ventajas de hacer una parte del trabajo en el cliente.**1.7** Relaciona cada tecnología con su lenguaje: JSP, PHP, Razor, Jinja, Thymeleaf.**1.8** ¿Qué es una plantilla natural?

- a) Una plantilla escrita sin CSS.
- b) Una plantilla que sigue siendo HTML válido y se puede abrir en el navegador sin servidor, con valores de ejemplo.
- c) Una plantilla que solo usa JavaScript.
- d) Una página estática de `static/`.

1.9 Abres `hora.html` con doble clic, sin servidor. ¿Qué ves?

- a) Un error, porque falta el servidor.
- b) El código de Thymeleaf como texto.
- c) `00:00:00` en la hora del servidor, y el reloj del navegador funcionando.
- d) La página vacía.

1.10 ¿Qué tres dependencias se marcan en el asistente para los proyectos de esta unidad?**1.11** ¿Por qué las dependencias del `pom.xml` no llevan `<version>`?

- a) Porque Maven siempre descarga la última.

- b)** Porque las fija el parent, spring-boot-starter-parent, con versiones compatibles entre sí.
- c)** Porque son opcionales.
- d)** Porque las indica application.properties.

1.12 ¿En qué carpeta van las plantillas de Thymeleaf? ¿Y la hoja de estilos de la web?

1.13 La clase principal está en com.ejemplo.primerproyecto. ¿Qué controlador encontrará Spring?

- a)** Uno en com.ejemplo.controladores.
- b)** Uno en com.ejemplo.primerproyecto.controller.
- c)** Los dos.
- d)** Ninguno: hay que registrarlos en application.properties.

1.14 Arrancas la aplicación y en http://localhost:8080/ sale la página «Whitelabel Error Page» con status=404. ¿Qué significa?

1.15 Al arrancar sale «Web server failed to start. Port 8080 was already in use.». ¿Cuál es la causa más probable y qué haces?

1.16 Con la aplicación en marcha y DevTools, cambias el texto de una plantilla y guardas. ¿Qué tienes que hacer para ver el cambio?

- a)** Parar y arrancar la aplicación.
- b)** Recargar el navegador.
- c)** Ejecutar Maven.
- d)** Nada: no se puede cambiar una plantilla en marcha.

Respuestas al final de la unidad, en «Autoevaluación · Del servidor al navegador».

2. Controladores, rutas y vistas

Un controlador recibe cada petición, prepara los datos y elige la plantilla que los muestra. Con él, tus páginas dejan de ser ficheros fijos.

El primer controlador

Spring organiza una aplicación web en tres piezas con tareas separadas: el controlador atiende la petición, el modelo lleva los datos y la vista los presenta. Así, cambiar el aspecto de una página no obliga a tocar el código Java, y al revés.

DEFINICIONES

MVC (Modelo-Vista-Controlador): forma de organizar una aplicación en esas tres piezas; Spring MVC es la parte de Spring que la aplica a la web.

Controlador: una clase anotada con `@Controller` cuyos métodos atienden peticiones.

Método manejador: cada método del controlador asociado a una ruta; devuelve el nombre de la vista que se mostrará.

Vista: la plantilla de Thymeleaf que genera el HTML; se guarda en `src/main/resources/templates/`.

PASO A PASO

Crear un controlador y su vista en STS

1. Clic derecho sobre el paquete base `com.ejemplo.primerproyecto` › **New** › **Package** y crea `com.ejemplo.primerproyecto.controller`, donde irán todos los controladores.
2. Clic derecho sobre el paquete nuevo › **New** › **Class**, con el nombre `InicioController`, y escribe su código.
3. Clic derecho sobre la carpeta `src/main/resources/templates` › **New** › **File**, con el nombre `inicio.html`, y escribe la plantilla.
4. Guarda los dos ficheros. Si la aplicación estaba en marcha, DevTools la reinicia sola; si no, arráncala con **Run As** › **Spring Boot App**.

COMPRUEBA QUE FUNCIONA

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/InicioController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
```

```
@Controller
public class InicioController {

    @GetMapping({ "/", "/inicio" })
    public String inicio() {
        return "inicio";
    }

}
```

Fichero src/main/resources/templates/inicio.html:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
</head>
<body>
<h1>Talleres del centro</h1>
<p>Mi primera página servida por un controlador.</p>
</body>
</html>
```

http://localhost:8080/ ya no da el error 404, y http://localhost:8080/inicio muestra lo mismo:



Página Talleres del centro con el texto Mi primera página servida por un controlador, en localhost:8080

REFERENCIA

Las piezas de un controlador

Código	Qué hace
@Controller	marca la clase como controlador; Spring la encuentra al arrancar si está en el paquete base o en un subpaquete
@GetMapping("/inicio")	el método atiende las peticiones GET a esa ruta
@GetMapping({ "/", "/inicio" })	el mismo método atiende varias rutas
return "inicio";	muestra la vista templates/inicio.html: el nombre va sin la extensión

Código	Qué hace
<code>return "saludo/saludo";</code>	muestra <code>templates/saludo/saludo.html</code> : las subcarpetas se escriben con <code>/</code>

Más detalle en la documentación de Spring: [Mapping Requests](#) (en inglés).

AVISO

Si el nombre que devuelve el método no coincide con ninguna plantilla (por ejemplo, `return "incio";`), la aplicación arranca bien, pero al pedir la página responde con un error 500: Revisa el nombre y la carpeta de la plantilla. Si la página da un 404, en cambio, el problema está en el controlador: le falta `@Controller`, está fuera del paquete base o la ruta de `@GetMapping` no es la que pides.

Pasar datos a la vista con Model

Una página dinámica necesita datos. El método manejador los calcula o los busca, y se los pasa a la plantilla dentro de un objeto `Model`.

DEFINICIONES

Model: el conjunto de datos que el controlador entrega a la vista. Basta con declararlo como parámetro del método para que Spring lo proporcione.

Atributo del modelo: cada dato del `Model`, con un nombre y un valor:

```
model.addAttribute("mensaje", "Hola, mundo").
```

Expresión `${...}`: en la plantilla, lee un atributo del modelo por su nombre: `${mensaje}`.

Sustitución literal `|...|`: combina texto fijo y expresiones sin concatenar:

`th:text="${nombre}, ${apellido}|"` muestra el nombre y el apellido reales separados por una coma, igual que `th:text="${nombre} + ', ' + ${apellido}"` pero más corto y más fácil de leer.

EJEMPLO

Una plantilla para dos páginas

Los dos métodos usan la misma vista, `saludo/saludo`, con datos distintos. `th:text` sustituye el contenido de la etiqueta por el valor del atributo, también en el `<title>` de la pestaña.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/SaludoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
```

```
@Controller
public class SaludoController {

    @GetMapping("/hola")
    public String hola(Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Hola, mundo");
        return "saludo/saludo";
    }

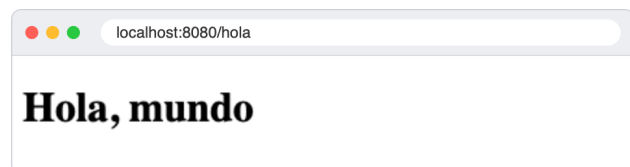
    @GetMapping("/adios")
    public String adios(Model model) {
        model.addAttribute("titulo", "Despedidas");
        model.addAttribute("mensaje", "Adiós, mundo");
        return "saludo/saludo";
    }
}
```

Fichero `src/main/resources/templates/saludo/saludo.html`:

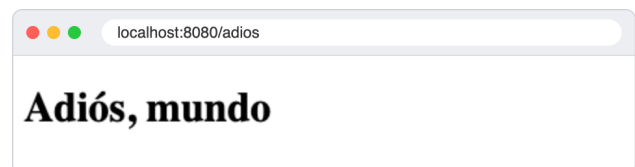
```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${titulo}">Título</title>
</head>
<body>
<h1 th:text="${mensaje}">Mensaje</h1>
</body>
</html>
```

COMPRUEBA QUE FUNCIONA

Cada ruta muestra su mensaje, y la pestaña, su título («Saludos» o «Despedidas»):

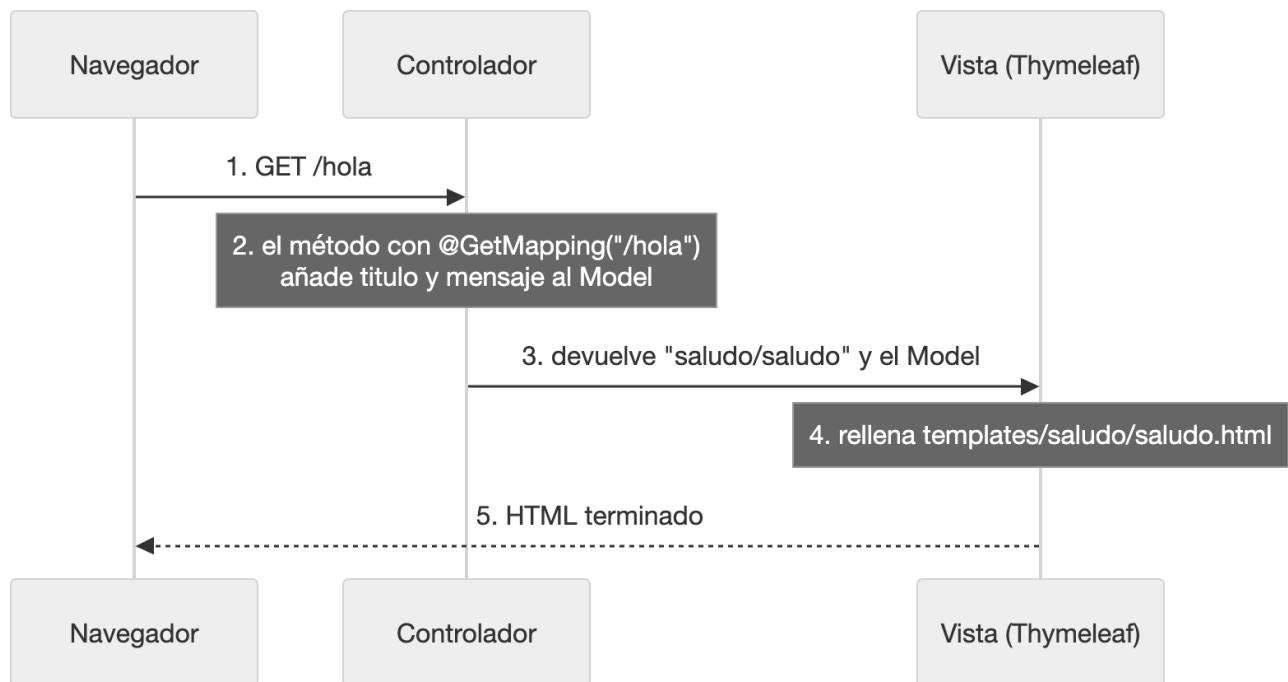


Página localhost:8080/hola con el título Hola, mundo



Página localhost:8080/adios con el título Adiós, mundo

Este es el recorrido completo de la petición `/hola`:



El navegador pide `GET /hola`; el método del controlador añade título y mensaje al Model y devuelve `saludo/saludo`; Thymeleaf rellena la plantilla y el navegador recibe el HTML terminado

AVISO

Si en la plantilla escribes un nombre que el modelo no tiene, por ejemplo `${mensaje}`, no hay error: la etiqueta sale vacía. Cuando una página aparece en blanco, compara letra a letra el nombre de `addAttribute` con el de `${...}`.

AMPLIACIÓN

Para qué sirve `xmlns:th`

La etiqueta `<html>` de las plantillas lleva `xmlns:th="http://www.thymeleaf.org"`. Thymeleaf procesa los atributos `th`: igual sin ella (una plantilla sin esa declaración genera el mismo HTML), pero así el editor sabe que son atributos de Thymeleaf y no los marca como desconocidos. Al navegador no le llega: Thymeleaf la quita al generar la página.

Recursos estáticos y enlaces

Las hojas de estilo, las imágenes y los ficheros de JavaScript no necesitan controlador: se guardan en `src/main/resources/static/` y Spring Boot los sirve tal cual. Desde las plantillas se enlazan con la sintaxis de URL de Thymeleaf, `@{...}`.

REFERENCIA

Del fichero a la dirección

Fichero en el proyecto	Dirección	En la plantilla
static/css/estilos.css	/css/estilos.css	th:href="@{/css/estilos.css}"
static/img/logo.svg	/img/logo.svg	th:src="@{/img/logo.svg}"
una ruta de un controlador	/hola	th:href="@{/hola}"
una web externa	https://spring.io/	href="https://spring.io/"

La carpeta `static` no forma parte de la dirección. Los enlaces externos llevan `href` normal, porque no son de tu aplicación.

EJEMPLO

La portada con estilos, logotipo y enlaces

La portada del ejemplo anterior enlaza su hoja de estilos, muestra el logotipo del centro y lleva a las dos páginas de `SaludoController`.

Fichero `src/main/resources/templates/inicio.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" th:href="@{/css/estilos.css}" />
</head>
<body>
<header>

<h1>Talleres del centro</h1>
</header>
<main>
<p>Mi primera página servida por un controlador.</p>
<ul>
<li><a th:href="@{/hola}">Un saludo</a></li>
<li><a th:href="@{/adios}">Una despedida</a></li>
<li><a href="https://spring.io/">Web de Spring</a></li>
</ul>
</main>
</body>
</html>
```

Fichero `src/main/resources/static/css/estilos.css`:

```
body {
margin: 0;
font-family: sans-serif;
background: #f4f6f8;
}
```

```

header {
  display: flex;
  align-items: center;
  gap: 12px;
  background: #2e7d32;
  color: white;
  padding: 12px 24px;
}

header h1 {
  margin: 0;
}

main {
  padding: 8px 24px;
}

```

Fichero `src/main/resources/static/img/logo.svg`:

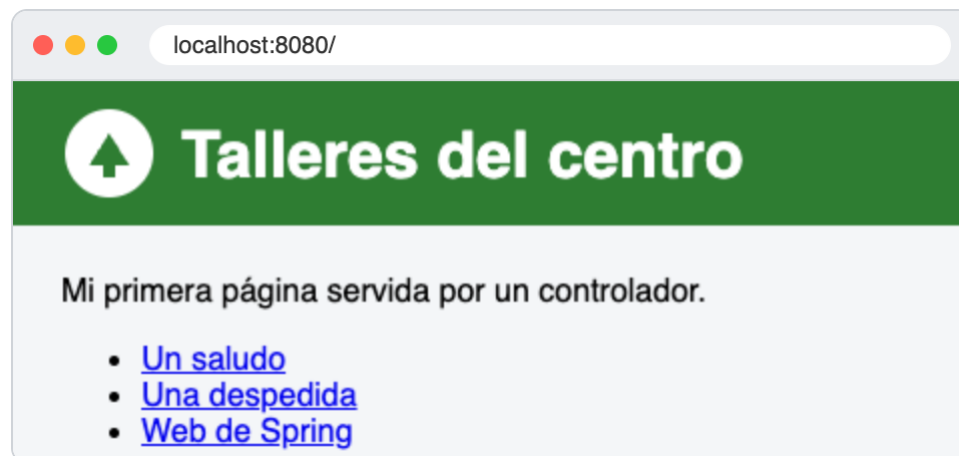
```

<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 48 48">
  <circle cx="24" cy="24" r="22" fill="white" />
  <path d="M14 30 L24 14 L34 30 Z" fill="#2e7d32" />
  <rect x="21" y="30" width="6" height="6" fill="#2e7d32" />
</svg>

```

COMPRUEBA QUE FUNCIONA

Los enlaces «Un saludo» y «Una despedida» llevan a `/hola` y `/adios`:



Portada Talleres del centro con la cabecera verde y el logotipo, el texto Mi primera página servida por un controlador y los enlaces Un saludo, Una despedida y Web de Spring

EJEMPLO**Por qué @{...} y no una dirección escrita a mano**

Una aplicación no siempre se publica en la raíz del servidor. Si se configura bajo `/app` (con `server.servlet.context-path=/app` en `application.properties`), Thymeleaf añade ese prefijo a todas las direcciones escritas con `@{...}`. Este es el HTML que recibe el navegador:

```
<link rel="stylesheet" href="/app/css/estilos.css" />

<li><a href="/app/hola">Un saludo</a></li>
<li><a href="/app/adios">Una despedida</a></li>
<li><a href="https://spring.io/">Web de Spring</a></li>
```

Un `href="/hola"` escrito a mano seguiría apuntando a `/hola`, fuera de la aplicación, y daría un 404. Con `@{...}` la plantilla funciona igual se publique donde se publique.

RECURSOS**Para saber más**

- **Standard URL Syntax:** todas las formas de `@{...}` en Thymeleaf, incluidas las direcciones con parámetros (en inglés).
<https://www.thymeleaf.org/doc/articles/standardurlsyntax.html>
- **Static Content:** qué carpetas sirve Spring Boot como recursos estáticos y cómo cambiarlas (en inglés).
<https://docs.spring.io/spring-boot/reference/web/servlet.html>

Datos en la ruta con @PathVariable

Con lo visto hasta ahora, saludar a Lucía, a Pedro y a Ana exigiría un método por persona, idénticos salvo por el nombre. Lo que cambia de una petición a otra puede ir en la propia dirección y llegar al método como un parámetro.

DEFINICIONES

Variable de ruta: un trozo de la ruta escrito entre llaves en `@GetMapping`, como `{nombre}` en `/hola/{nombre}`; vale lo que haya en esa posición de la dirección.

`@PathVariable`: anotación de un parámetro del método que recibe el valor de la variable de ruta con su mismo nombre.

EJEMPLO**Un saludo para cualquier nombre**

El método `holaA` atiende `/hola/Lucía`, `/hola/Pedro` o cualquier otro nombre; `hola`, sin variable, sigue atendiendo `/hola`.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/SaludoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@Controller
public class SaludoController {

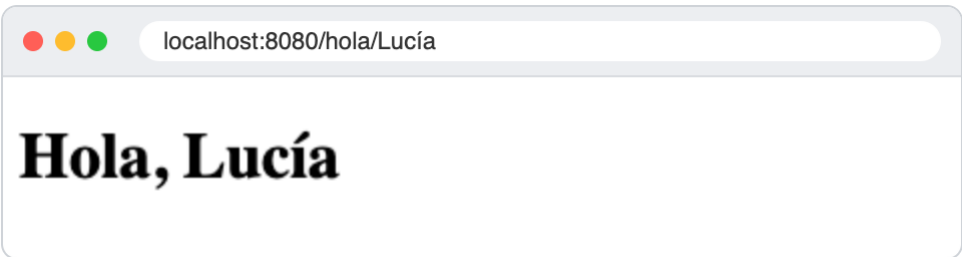
    @GetMapping("/hola")
    public String hola(Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Hola, mundo");
        return "saludo/saludo";
    }

    @GetMapping("/hola/{nombre}")
    public String holaA(@PathVariable String nombre, Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Hola, " + nombre);
        return "saludo/saludo";
    }

}
```

COMPRUEBA QUE FUNCIONA

Escribe tu nombre al final de la dirección, aunque lleve tildes:



Página `localhost:8080/hola/Lucía` con el título `Hola, Lucía`

REFERENCIA

Qué rutas encajan

Petición	Resultado con el <code>SaludoController</code> anterior
<code>/hola/Lucía</code>	<code>holaA</code> con <code>nombre = "Lucía"</code> : «Hola, Lucía»
<code>/hola</code>	<code>hola</code> , el método sin variable: «Hola, mundo»
<code>/hola/Ana/Ruiz</code>	404: <code>{nombre}</code> ocupa un solo trozo de la ruta, sin <code>/</code>
<code>/sumar/tres/4</code> con <code>@PathVariable int a</code>	400: Spring convierte el texto al tipo del parámetro y <code>"tres"</code> no es un <code>int</code> («Failed to convert value of type 'java.lang.String' to required type 'int'»)

Un método puede tener varias variables de ruta, cada una con su `@PathVariable`:

`@GetMapping("/sumar/{a}/{b}")`.

AVISO

Si al pedir la página sale el error 500 «Name for argument of type [java.lang.String] not specified, and parameter name information not available via reflection. Ensure that the compiler uses the '-parameters' flag.», el proyecto se ha compilado sin guardar los nombres de los parámetros. Los proyectos de Spring Boot vienen configurados para guardarlos, pero si te ocurre, escribe el nombre en la anotación, que funciona siempre:

Datos comunes con @ModelAttribute

Hay datos que necesitan todas las páginas de un controlador, como el título de la pestaña o el nombre del centro. En lugar de repetir el mismo `addAttribute` en cada método, se escribe una vez en un método anotado con `@ModelAttribute`.

DEFINICIÓN

Método `@ModelAttribute("nombre")`: Spring lo ejecuta antes de cada método manejador de su controlador y guarda lo que devuelve en el modelo con ese nombre.

EJEMPLO

El título, una sola vez

Ningún método manejador añade ya `título`, y todas las páginas del controlador lo tienen.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/SaludoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;

@Controller
public class SaludoController {

    @ModelAttribute("titulo")
    public String titulo() {
        return "Saludos";
    }

    @GetMapping("/hola")
    public String hola(Model model) {
        model.addAttribute("mensaje", "Hola, mundo");
        return "saludo/saludo";
    }

    @GetMapping("/hola/{nombre}")
    public String holaA(@PathVariable String nombre, Model model) {
        model.addAttribute("mensaje", "Hola, " + nombre);
        return "saludo/saludo";
    }
}
```

```
}

@GetMapping("/adios/{nombre}")
public String adiosA(@PathVariable String nombre, Model model) {
    model.addAttribute("mensaje", "Adiós, " + nombre);
    return "saludo/saludo";
}

}
```

COMPRUEBA QUE FUNCIONA

En `/hola`, `/hola/Pedro` y `/adios/Pedro` la pestaña se titula «Saludos». Este es el HTML que llega con `/adios/Pedro`:

```
<title>Saludos</title>
...
<h1>Adiós, Pedro</h1>
```

AVISO

Un método `@ModelAttribute` solo afecta a su controlador. Si otro controlador necesita el mismo dato, necesita su propio método.

RECURSOS

Para saber más

- **Modelo-Vista-Controlador:** el patrón explicado en el glosario de MDN.
<https://developer.mozilla.org/es/docs/Glossary/MVC>
- **Model:** los métodos `@ModelAttribute` en la documentación de Spring (en inglés).
<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-modelattrib-methods.html>

Práctica

Tres ejercicios para escribir controladores, pasar datos a las vistas y enlazar páginas.

ENTRENAMIENTO 2.1

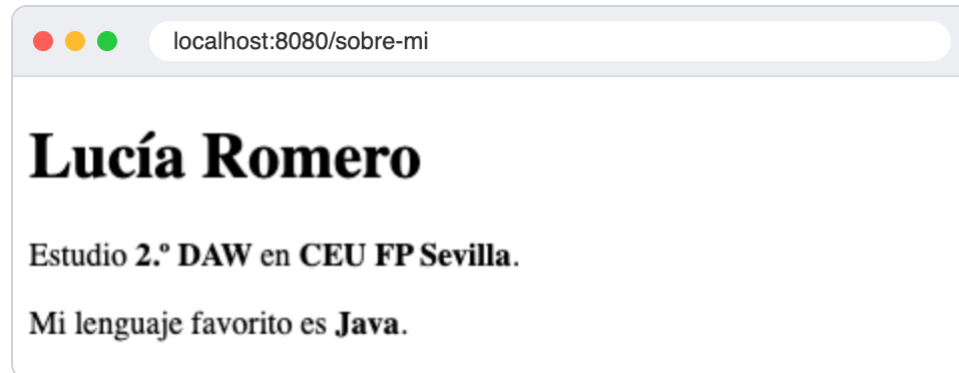
Sobre mí

Nivel · Fácil

Crea un controlador con la ruta `/sobre-mi` que pase a su vista tu nombre, tu ciclo, tu centro y tu lenguaje de programación favorito. La vista los muestra, y la pestaña del navegador lleva tu nombre como título.

PISTAS

- Cuatro `model.addAttribute`, uno por dato.
- Dentro de un párrafo, un `<strong th:text="${ciclo}">` sustituye solo esa palabra.

RESULTADO ESPERADO

Página localhost:8080/sobre-mi con el título Lucía Romero y los textos Estudio 2.º DAW en CEU FP Sevilla y Mi lenguaje favorito es Java

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/SobreMiController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class SobreMiController {

    @GetMapping("/sobre-mi")
    public String sobreMi(Model model) {
        model.addAttribute("nombre", "Lucía Romero");
        model.addAttribute("ciclo", "2.º DAW");
        model.addAttribute("centro", "CEU FP Sevilla");
        model.addAttribute("lenguaje", "Java");
        return "sobre-mi";
    }
}
```

Fichero `src/main/resources/templates/sobre-mi.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${nombre}">Nombre</title>
</head>
<body>
<h1 th:text="${nombre}">Nombre</h1>
```

```
<p>Estudio <strong th:text="${ciclo}">ciclo</strong> en <strong th:text="${centro}">centro</strong>.</p>
<p>Mi lenguaje favorito es <strong th:text="${lenguaje}">lenguaje</strong>.</p>
</body>
</html>
```

ENTRENAMIENTO 2.2

Calculadora por la dirección

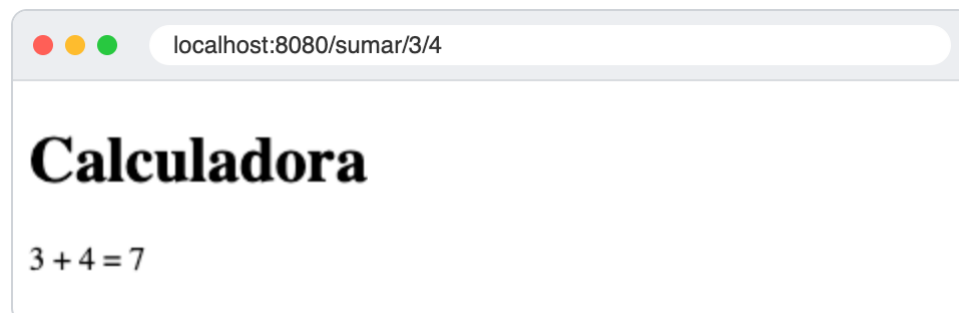
Nivel · Intermedio

Haz que `/sumar/3/4` muestre « $3 + 4 = 7$ » y `/multiplicar/6/7` muestre « $6 \times 7 = 42$ », con cualquier par de números enteros. Las dos rutas usan la misma vista. Comprueba también qué responde `/sumar/tres/4`.

PISTAS

- Dos variables de ruta, `{a}` y `{b}`, recibidas como `int`.
- La operación completa puede llegar a la vista ya escrita, en un solo atributo.
- Cuidado con `a + " + " + b + " = " + a + b`: sin paréntesis, Java concatena en vez de sumar.

RESULTADO ESPERADO



Página `localhost:8080/sumar/3/4` con el título `Calculadora` y el resultado `3 + 4 = 7`

`/sumar/tres/4` responde con un error 400: `"tres"` no se puede convertir en `int`.

UNA SOLUCIÓN

Fichero

`src/main/java/com/ejemplo/primerproyecto/controller/CalculadoraController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
```

```
@Controller
public class CalculadoraController {

    @GetMapping("/sumar/{a}/{b}")
    public String sumar(@PathVariable int a, @PathVariable int b, Model model) {
        model.addAttribute("operacion", a + " + " + b + " = " + (a + b));
        return "resultado";
    }

    @GetMapping("/multiplicar/{a}/{b}")
    public String multiplicar(@PathVariable int a, @PathVariable int b, Model
model) {
        model.addAttribute("operacion", a + " x " + b + " = " + (a * b));
        return "resultado";
    }

}
```

Fichero `src/main/resources/templates/resultado.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Calculadora</title>
</head>
<body>
<h1>Calculadora</h1>
<p th:text="${operacion}">2 + 2 = 4</p>
</body>
</html>
```

ENTRENAMIENTO 2.3

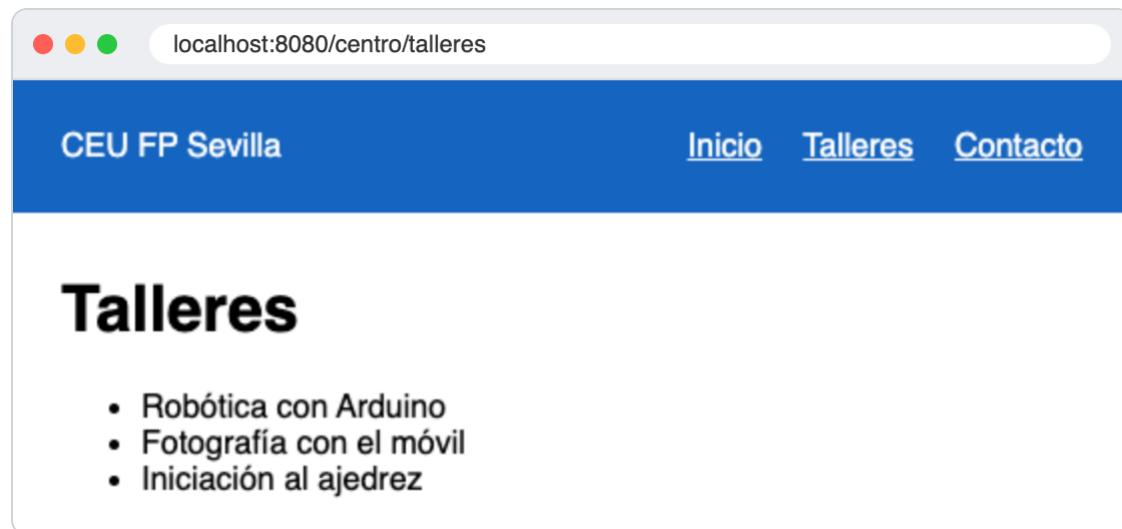
La web del centro

Nivel · Difícil

Construye una web de tres páginas (Inicio, Talleres y Contacto) con una cabecera común: el nombre del centro a la izquierda y un menú con las tres páginas a la derecha. Las tres comparten una hoja de estilos. La aplicación se publica bajo `/centro` (`http://localhost:8080/centro/`) y todos los enlaces y estilos deben funcionar ahí.

PISTAS

- Un solo controlador con tres métodos, y sus plantillas en `templates/centro/`.
- El nombre del centro lo necesitan las tres páginas: un método `@ModelAttribute`.
- `server.servlet.context-path=/centro` en `application.properties`, y todas las direcciones internas con `@{...}`.
- De momento la cabecera se repite en las tres plantillas; en la sección siguiente verás cómo escribirla una sola vez.

RESULTADO ESPERADO

Página localhost:8080/centro/talleres con la cabecera azul CEU FP Sevilla, el menú Inicio, Talleres y Contacto, y la lista de talleres

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/CentroController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;

@Controller
public class CentroController {

    @ModelAttribute("centro")
    public String centro() {
        return "CEU FP Sevilla";
    }

    @GetMapping("/")
    public String inicio() {
        return "centro/inicio";
    }

    @GetMapping("/talleres")
    public String talleres() {
        return "centro/talleres";
    }

    @GetMapping("/contacto")
    public String contacto() {
        return "centro/contacto";
    }
}
```

Fichero `src/main/resources/templates/centro/talleres.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres</title>
<link rel="stylesheet" th:href="@{/css/centro.css}" />
</head>
<body>
<header>
<p th:text="${centro}">Centro</p>
<nav>
<a th:href="@{/}">Inicio</a>
<a th:href="@{/talleres}">Talleres</a>
<a th:href="@{/contacto}">Contacto</a>
</nav>
</header>
<main>
<h1>Talleres</h1>
<ul>
<li>Robótica con Arduino</li>
<li>Fotografía con el móvil</li>
<li>Iniciación al ajedrez</li>
</ul>
</main>
</body>
</html>
```

Fichero `src/main/resources/templates/centro/inicio.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Inicio</title>
<link rel="stylesheet" th:href="@{/css/centro.css}" />
</head>
<body>
<header>
<p th:text="${centro}">Centro</p>
<nav>
<a th:href="@{/}">Inicio</a>
<a th:href="@{/talleres}">Talleres</a>
<a th:href="@{/contacto}">Contacto</a>
</nav>
</header>
<main>
<h1>Inicio</h1>
<p>Bienvenida a la web de talleres de <span th:text="${centro}">centro</span>.</p>
</main>
</body>
</html>
```

Fichero `src/main/resources/templates/centro/contacto.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
```

```

<head>
<meta charset="UTF-8" />
<title>Contacto</title>
<link rel="stylesheet" th:href="@{/css/centro.css}" />
</head>
<body>
  <header>
    <p th:text="${centro}">Centro</p>
    <nav>
      <a th:href="@{/}">Inicio</a>
      <a th:href="@{/talleres}">Talleres</a>
      <a th:href="@{/contacto}">Contacto</a>
    </nav>
  </header>
  <main>
    <h1>Contacto</h1>
    <p>Escríbenos a <a href="mailto:talleres@ejemplo.com">talleres@ejemplo.com</a>.</p>
  </main>
</body>
</html>

```

Fichero `src/main/resources/static/css/centro.css`:

```

body {
  margin: 0;
  font-family: sans-serif;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #1565c0;
  color: white;
  padding: 8px 24px;
}

nav a {
  color: white;
  margin-left: 16px;
}

main {
  padding: 8px 24px;
}

```

Fichero `src/main/resources/application.properties`:

```

spring.application.name=primer-proyecto
server.servlet.context-path=/centro

```

PÍLDORA**Una página dinámica en 4 piezas**

1. Un controlador con `@Controller`, en el paquete base o un subpaquete.
2. Un método con `@GetMapping("/ruta")`, que recibe `Model` (y `@PathVariable` si la ruta lleva datos).
3. `model.addAttribute("nombre", valor)` y `return "carpeta/vista";`.
4. La plantilla en `templates/carpeta/vista.html`, con `th:text="${nombre}"` y enlaces `@{...}`.

AUTOEVALUACIÓN**Controladores, rutas y vistas**

2.1 En MVC, ¿qué pieza atiende la petición y decide qué vista se muestra?

- a) El modelo.
- b) La vista.
- c) El controlador.
- d) El navegador.

2.2 ¿Qué dos cosas necesita una clase para que Spring la use como controlador?

2.3 Un método devuelve "alumnos/lista". ¿Qué fichero muestra?

- a) templates/alumnos/lista
- b) templates/alumnos/lista.html
- c) static/alumnos/lista.html
- d) alumnos/lista.jsp

2.4 ¿Cómo haces que un mismo método atienda / y /inicio?

2.5 Al pedir una página sale un error 500 con «Error resolving template [inicio]». ¿Qué pasa?

- a) El controlador no tiene @Controller.
- b) El método devuelve el nombre de una plantilla que no existe.
- c) El puerto está ocupado.
- d) Falta el Model.

2.6 Pides /hola y sale un 404, aunque la plantilla existe. Di dos causas posibles.

2.7 ¿Cómo recibe un método el objeto Model?

- a) Creándolo con new Model().
- b) Declarándolo como parámetro del método: Spring lo proporciona.
- c) Con @Autowired.
- d) Leyéndolo de application.properties.

2.8 El controlador hace model.addAttribute("mensaje", "Hola") y la plantilla tiene <h1 th:text="\${mensaje}>. ¿Qué se ve?

- a) Un error 500.
- b) «Hola».
- c) El <h1> vacío.
- d) El texto \${mensaje}.

2.9 ¿Qué hace th:text="\${titulo}" en una etiqueta <title>?

2.10 El fichero src/main/resources/static/img/logo.png, ¿en qué dirección se sirve?

- a) /static/img/logo.png
- b) /img/logo.png
- c) /resources/static/img/logo.png
- d) No se sirve si no hay un controlador.

2.11 ¿Por qué los enlaces internos se escriben con `th:href="@{/hola}"` y no con `href="/hola"`?

2.12 ¿Cómo se escribe en una plantilla el enlace a `https://spring.io/`?

- a) `th:href="@{https://spring.io/}"` es obligatorio.
- b) Con un `href` normal: es una dirección externa.
- c) No se puede enlazar fuera de la aplicación.
- d) `th:src="https://spring.io/"`

2.13 Con `@GetMapping("/hola/{nombre}")`, ¿qué valor recibe `nombre` al pedir `/hola/Pedro`? ¿Y qué pasa con `/hola/Pedro/Ruiz`?

2.14 El método recibe `@PathVariable int edad` y alguien pide `/edad/veinte`. ¿Qué responde la aplicación?

- a) La página con `edad = 0`.
- b) Un error 400: `"veinte"` no se puede convertir a `int`.
- c) Un error 404.
- d) La página con el texto «veinte».

2.15 Sale el error «parameter name information not available via reflection». ¿Cómo lo resuelves sin tocar la configuración del proyecto?

2.16 ¿Cuándo se ejecuta un método anotado con `@ModelAttribute("titulo")`?

- a) Solo al arrancar la aplicación.
- b) Antes de cada método manejador de su controlador.
- c) Antes de los métodos de todos los controladores.
- d) Solo si un método lo llama.

2.17 Tres métodos de un controlador hacen `model.addAttribute("centro", "CEU FP Sevilla")`. ¿Cómo lo mejorarías?

Respuestas al final de la unidad, en «Autoevaluación · Controladores, rutas y vistas».

3. Contenido dinámico

Una página dinámica no solo pone textos: muestra objetos completos, calcula, decide qué enseñar y reutiliza trozos de plantilla. Todo eso se escribe en la plantilla, con las expresiones de Thymeleaf.

Texto y HTML: `th:text` y `th:utext`

Hasta ahora los datos eran textos sencillos. Antes de pasar a datos más complejos hay que saber qué hace Thymeleaf con un texto que contiene etiquetas HTML, porque de eso depende la seguridad de la página.

DEFINICIONES

`th:text`: sustituye el contenido de la etiqueta por el valor, convirtiendo los caracteres especiales del HTML (`<`, `>`, `&`) en texto visible. Es lo que se usa por defecto.

`th:utext`: sustituye el contenido por el valor sin convertir nada, de modo que el HTML del valor se interpreta como HTML.

XSS (Cross-Site Scripting): ataque en el que alguien consigue que su código JavaScript se ejecute en las páginas que ven otros usuarios.

EJEMPLO

El mismo valor, dos formas de mostrarlo

El controlador envía un aviso con una etiqueta `<em>` y un comentario que ha escrito un usuario cualquiera; este último contiene un `<script>`. La plantilla muestra cada valor con `th:text` y con `th:utext`.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TextoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class TextoController {

    @GetMapping("/texto")
    public String texto(Model model) {
        model.addAttribute("aviso", "<em>Plazas limitadas</em>");
        // un comentario que ha escrito un usuario cualquiera
        model.addAttribute("comentario",
            "<script>document.body.style.background = 'tomato'</script>iQué buen taller!");
        return "texto";
    }
}
```

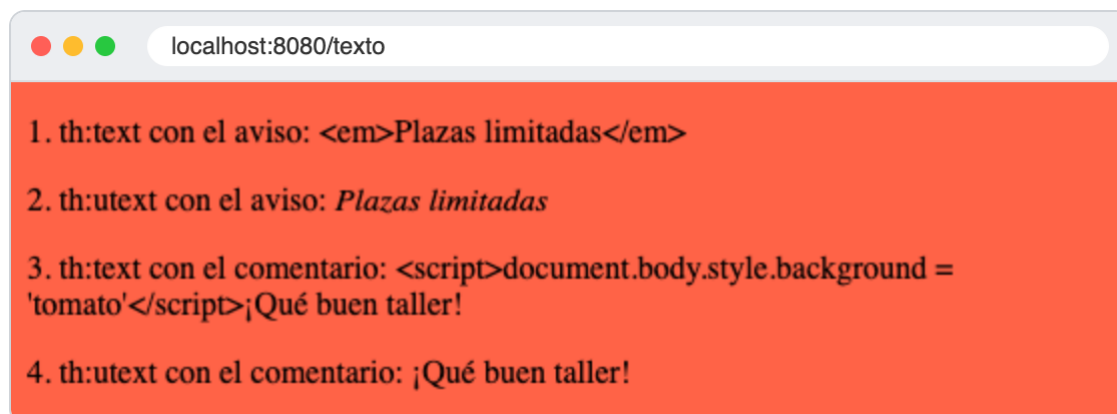
}

Fichero `src/main/resources/templates/texto.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>th:text y th:utext</title>
</head>
<body>
<p>1. th:text con el aviso: <span th:text="{aviso}">aviso</span></p>
<p>2. th:utext con el aviso: <span th:utext="{aviso}">aviso</span></p>
<p>3. th:text con el comentario: <span th:text="{comentario}">comentario</span></p>
<p>4. th:utext con el comentario: <span th:utext="{comentario}">comentario</span></p>
</body>
</html>
```

COMPRUEBA QUE FUNCIONA

Con `th:text` (líneas 1 y 3) las etiquetas se ven como texto. Con `th:utext` (líneas 2 y 4) el `<em>` pone el aviso en cursiva y el `<script>` se ejecuta: pinta el fondo de toda la página y desaparece del texto:



Página con el fondo en rojo: con `th:text` se ven las etiquetas escritas; con `th:utext` el aviso sale en cursiva y el comentario sin el script

AVISO

Cambiar el color de fondo es inofensivo, pero el mismo mecanismo permite a un atacante leer datos de la página o actuar en nombre de quien la abre. Usa `th:text` siempre, y `th:utext` solo con HTML que hayas escrito tú. Nunca con lo que llegue de un usuario, aunque lo haya escrito en un formulario tuyo.

Un modelo propio

Casi nunca hay un dato suelto: hay talleres, alumnos o productos, con varios campos cada uno. En Java se representan con una clase, y el controlador entrega a la vista el objeto entero.

DEFINICIONES

Clase de modelo: una clase Java que representa un dato de la aplicación, como `Taller`, con sus campos y sus métodos `get`. Suele guardarse en el paquete `model`.

Propiedad: en Thymeleaf, cada campo al que se accede con un `get`. `${taller.nombre}` llama a `taller.getNombre()`.

EJEMPLO**La clase `Taller`**

Un taller tiene nombre, nivel, fecha, plazas, inscritos y precio. Además, dos métodos calculados: las plazas libres y si está completo.

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Taller.java`:

```
package com.ejemplo.primerproyecto.model;

import java.time.LocalDate;

public class Taller {

    private int id;
    private String nombre;
    private String nivel; // "básico", "medio" o "avanzado"
    private LocalDate fecha;
    private int plazas;
    private int inscritos;
    private double precio;

    public Taller(int id, String nombre, String nivel, LocalDate fecha, int plazas,
int inscritos, double precio) {
        this.id = id;
        this.nombre = nombre;
        this.nivel = nivel;
        this.fecha = fecha;
        this.plazas = plazas;
        this.inscritos = inscritos;
        this.precio = precio;
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getNivel() {
        return nivel;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public int getPlazas() {
```

```

        return plazas;
    }

    public int getInscritos() {
        return inscritos;
    }

    public double getPrecio() {
        return precio;
    }

    public int getPlazasLibres() {
        return plazas - inscritos;
    }

    public boolean isCompleto() {
        return inscritos >= plazas;
    }
}

```

Los `get` son lo que usa Thymeleaf. Un método que devuelve `boolean` se llama `isXxx`, y su propiedad es `xxx`: `isCompleto()` se lee `#{taller.completo}`.

EJEMPLO

Entregar un taller a la vista

El controlador busca el taller con el número que llega en la ruta y lo añade al modelo. La plantilla lee sus propiedades con un punto.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```

package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class TallerController {

    private final List<Taller> talleres = List.of(
        new Taller(1, "Robótica con Arduino", "medio",
            LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
            LocalDate.of(2026, 10, 21), 20, 20, 0),
        new Taller(3, "Ajedrez de competición", "avanzado",
            LocalDate.of(2026, 11, 4), 12, 3, 10.5));

    @GetMapping("/talleres/{id}")
    public String taller(@PathVariable int id, Model model) {
        Taller taller = null;
    }
}

```

```
        for (Taller t : talleres) {
            if (t.getId() == id) {
                taller = t;
            }
        }
        model.addAttribute("taller", taller);
        return "talleres/taller";
    }
}
```

Fichero `src/main/resources/templates/talleres/taller.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${taller.nombre}">Taller</title>
</head>
<body>
<h1 th:text="${taller.nombre}">Nombre del taller</h1>
<ul>
<li>Nivel: <span th:text="${taller.nivel}">básico</span></li>
<li>Fecha: <span th:text="${taller.fecha}">2026-01-01</span></li>
<li>Precio: <span th:text="${taller.precio}">0.0</span> €</li>
<li>Plazas libres: <span th:text="${taller.plazasLibres}">0</span></li>
</ul>
</body>
</html>
```

Los talleres están escritos en el propio controlador. Guardar datos de verdad es tarea de otras partes de una aplicación; aquí basta con tenerlos a mano.

COMPRUEBA QUE FUNCIONA

En `http://localhost:8080/talleres/1`:



Página Robótica con Arduino con su nivel, fecha, precio y plazas libres

AVISO

Si pides `/talleres/9`, que no existe, el controlador deja `taller` a `null` y la página responde con un error 500. En la consola sale «EL1007E: Property or field 'nombre' cannot be found on null». Es el aviso de Thymeleaf cuando intentas leer una propiedad de algo que no existe; el apartado de condiciones muestra cómo evitarlo.

Expresiones

Dentro de `th:text` (y del resto de atributos `th:`) se escriben expresiones que Thymeleaf evalúa antes de generar el HTML. Van casi siempre entre `${...}`, pero se pueden combinar con textos, operaciones y funciones.

REFERENCIA**Expresiones más usadas**

Para	Expresión	Resultado (taller 1)
Leer una propiedad	<code>\${taller.nombre}</code>	Robótica con Arduino
Unir textos	<code>'Taller: ' + \${taller.nombre}</code>	Taller: Robótica con Arduino
Sustitución de literales	<code> Taller: \${taller.nombre} </code>	Taller: Robótica con Arduino
Operar	<code>\${taller.plazas - taller.inscritos}</code>	4
Comparar	<code>\${taller.inscritos >= taller.plazas}</code>	false
Comparar con un texto	<code>\${taller.nivel == 'medio'}</code>	true
Elegir entre dos valores	<code>\${taller.completo} ? 'Completo' : 'Con plazas'</code>	Con plazas
Dar formato a una fecha	<code>\${#temporals.format(taller.fecha, 'dd/MM/yyyy')}</code>	14/10/2026
Dar formato a un número	<code>\${#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')}</code>	25,00
Cambiar un texto	<code>\${#strings.toUpperCase(taller.nombre)}</code>	ROBÓTICA CON ARDUINO
Valor por defecto si es null	<code>\${apodo} ?: 'sin valor'</code>	sin valor
Leer una propiedad sin fallar si el objeto es null	<code>\${tutor?.nombre}</code>	(vacío)

Sin formato, la fecha se ve como `2026-10-14` y el precio como `25.0`. Las funciones que empiezan por `#` (`#temporals`, `#numbers`, `#strings`) son utilidades de Thymeleaf. Más detalle en el tutorial de Thymeleaf: [Standard Expression Syntax](#) (en inglés).

EJEMPLO

Pruébalas todas

Esta plantilla escribe cada expresión y, al lado, lo que devuelve; sirve para experimentar con tus propias expresiones.

Fichero `src/main/resources/templates/talleres/taller.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Expresiones</title>
</head>
<body>
<ul>
<li><code>${taller.nombre}</code> ⇒ [<span th:text="${taller.nombre}"></span>]</li>
<li><code>${taller.getNombre()}</code> ⇒ [<span th:text="${taller.getNombre()}"></span>]</li>
<li><code>'Taller: ' + ${taller.nombre}</code> ⇒ [<span th:text="'Taller: ' + ${taller.nombre}"></span>]</li>
<li><code>|Taller: ${taller.nombre}|</code> ⇒ [<span th:text="|Taller: ${taller.nombre}|"></span>]</li>
<li><code>${taller.plazas - taller.inscritos}</code> ⇒ [<span th:text="${taller.plazas - taller.inscritos}"></span>]</li>
<li><code>${taller.plazasLibres}</code> ⇒ [<span th:text="${taller.plazasLibres}"></span>]</li>
<li><code>${taller.inscritos >= taller.plazas}</code> ⇒ [<span th:text="${taller.inscritos >= taller.plazas}"></span>]</li>
<li><code>${taller.completo} ? 'Completo' : 'Con plazas'</code> ⇒ [<span th:text="${taller.completo} ? 'Completo' : 'Con plazas'"></span>]</li>
<li><code>${taller.nivel == 'medio'}</code> ⇒ [<span th:text="${taller.nivel == 'medio'}"></span>]</li>
<li><code>${taller.fecha}</code> ⇒ [<span th:text="${taller.fecha}"></span>]</li>
<li><code>${#temporals.format(taller.fecha, 'dd/MM/yyyy')}</code> ⇒ [<span th:text="${#temporals.format(taller.fecha, 'dd/MM/yyyy')}"></span>]</li>
<li><code>${taller.precio}</code> ⇒ [<span th:text="${taller.precio}"></span>]</li>
<li><code>${#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')}</code> ⇒ [<span th:text="${#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')}"></span>]</li>
<li><code>${#strings.toUpperCase(taller.nombre)}</code> ⇒ [<span th:text="${#strings.toUpperCase(taller.nombre)}"></span>]</li>
<li><code>${nadie?.nombre}</code> ⇒ [<span th:text="${nadie?.nombre}"></span>]</li>
<li><code>${nadie} ?: 'sin valor'</code> ⇒ [<span th:text="${nadie} ?: 'sin valor'"></span>]</li>
</ul>
</body>
</html>
```

Usa el mismo `Taller` y `TallerController` de antes. Pide `/talleres/1` y `/talleres/2` y compara los resultados.

AVISO

Una comparación se escribe con `==`. Con un solo `=` Thymeleaf lo entiende como una asignación y falla con «EL1010E: Property or field 'nivel' cannot be set on object of type 'Taller'».

Mostrar u ocultar contenido

La misma plantilla debe servir para un taller con plazas, uno completo o uno que no existe. Para eso se decide qué partes de la página se generan.

DEFINICIONES

`th:if`: la etiqueta se genera solo si la condición es verdadera.

`th:unless`: la etiqueta se genera solo si la condición es falsa. Es lo contrario de `th:if`.

`th:switch` y `th:case`: eligen una entre varias etiquetas según el valor de una expresión.

`th:case="*"` recoge todo lo demás.

EJEMPLO

La ficha de un taller, con todo lo anterior

La plantilla comprueba que el taller existe antes de leer sus propiedades, marca el nivel con un color según su valor, da formato a la fecha y al precio (con «Gratis» si vale cero) y avisa cuando el taller está completo.

Fichero `src/main/resources/templates/talleres/taller.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${taller != null} ? ${taller.nombre} : 'Taller no encontrado'">Taller</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<header><a th:href="@{/talleres/1}">Talleres del centro</a></header>
<main th:if="${taller != null}">
<h1 th:text="${taller.nombre}">Nombre del taller</h1>
<p th:switch="${taller.nivel}">
<span th:case="'básico'" class="nivel basico">Nivel básico</span>
<span th:case="'medio'" class="nivel medio">Nivel medio</span>
<span th:case="*" class="nivel avanzado">Nivel avanzado</span>
</p>
<ul>
<li>Fecha: <span th:text="${#temporals.format(taller.fecha, 'dd/MM/yyyy')}">01/01/2026</span></li>
<li>Precio: <span th:text="${taller.precio == 0} ? 'Gratis' : |${#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')} €|">0,00 €</span></li>
<li>Inscritos: ${taller.inscritos} de ${taller.plazas}|>Inscritos: 0 de 0</li>
</ul>
<p th:if="${taller.completo}" class="aviso">Taller completo: apúntate a la lista de espera en secretaría.</p>
<p th:unless="${taller.completo}" th:text="|Quedan ${taller.plazasLibres} plazas libres.|">Quedan plazas libres.</p>
</main>
<main th:unless="${taller != null}">
<h1>Taller no encontrado</h1>
<p>No hay ningún taller con ese número.</p>
</main>
```

```
</main>
<footer>
  <p>Curso 2026/2027 · Inscripciones en secretaría</p>
</footer>
</body>
</html>
```

Fichero `src/main/resources/static/css/talleres.css`:

```
body {
  margin: 0;
  font-family: sans-serif;
}

header {
  background: #2e7d32;
  padding: 12px 24px;
}

header a {
  color: white;
  font-weight: bold;
  text-decoration: none;
}

main {
  padding: 8px 24px;
}

footer {
  border-top: 1px solid #ccc;
  margin: 0 24px;
  color: #666;
  font-size: 14px;
}

.nivel {
  padding: 2px 8px;
  border-radius: 8px;
  font-size: 14px;
}

.basico {
  background: #c8e6c9;
}

.medio {
  background: #fff59d;
}

.avanzado {
  background: #ffcdd2;
}

.aviso {
  color: #c62828;
  font-weight: bold;
}
```

COMPRUEBA QUE FUNCIONA

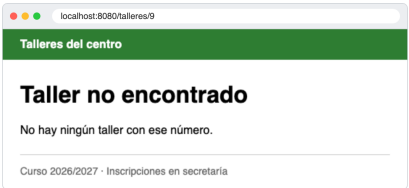
Un taller con plazas, uno completo y uno que no existe (/talleres/1, /talleres/2 y /talleres/9):



Robótica con Arduino con nivel medio en amarillo y Quedan 4 plazas libres



Fotografía con el móvil con nivel básico en verde, precio Gratis y el aviso Taller completo en rojo



Taller no encontrado con el mensaje No hay ningún taller con ese número

REFERENCIA

Qué sale en cada caso

Petición	Efecto de las condiciones
/talleres/1	th:case="'medio'" genera «Nivel medio»; th:unless="{taller.completo}" genera «Quedan 4 plazas libres»
/talleres/2	th:case="'básico'" genera «Nivel básico»; th:if="{taller.completo}" genera el aviso en rojo
/talleres/9	th:unless="{taller != null}" genera «Taller no encontrado»; el <main> del taller no se genera y no falla

La etiqueta que no cumple la condición no llega al navegador: no se oculta con CSS, no está en el HTML.

AMPLIACIÓN**Valores que pueden faltar**

Un dato puede ser `null` sin que sea un error: un tutor sin asignar, un apodo que no se ha escrito. Para esos casos hay dos atajos, que evitan un `th:if` para cada campo.

- `${apodo} ? : 'sin apodo'`: el operador Elvis usa el valor de la izquierda y, si es `null`, el de la derecha.
- `${tutor?.nombre}`: con `?.` se lee la propiedad solo si `tutor` existe; si es `null`, el resultado es `null` en lugar de un error.

Fichero `src/main/resources/templates/perfil.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Perfil</title>
</head>
<body>
<p>Nombre: <span th:text="${nombre}">nombre</span></p>
<p>Apodo: <span th:text="${apodo} ? : 'sin apodo'">apodo</span></p>
<p>Tutor: <span th:text="${tutor?.nombre} ? : 'sin tutor asignado'">tutor</span></p>
</body>
</html>
```

Con `apodo` y `tutor` a `null` en el modelo, la página muestra «sin apodo» y «sin tutor asignado».

Fragmentos de plantilla

La cabecera y el pie de una web son iguales en todas las páginas. Copiarlos en cada plantilla obliga a cambiarlos en todas cuando algo varía. Un fragmento se escribe una vez en su propio fichero y las páginas lo reutilizan.

DEFINICIONES

Fragmento: un trozo de plantilla con nombre, marcado con `th:fragment="nombre"`, que otras plantillas pueden usar.

`th:replace`: sustituye la etiqueta por el fragmento indicado, con la forma `~{carpeta/fichero :: nombre}`.

`th:insert`: en lugar de sustituir la etiqueta, mete el fragmento dentro de ella.

EJEMPLO

La cabecera y el pie, una sola vez

Los fragmentos se guardan en `templates/fragmentos/`. Así queda la ficha del taller usándolos con `th:replace`, en vez del `<header>/<footer>` sueltos del apartado anterior:

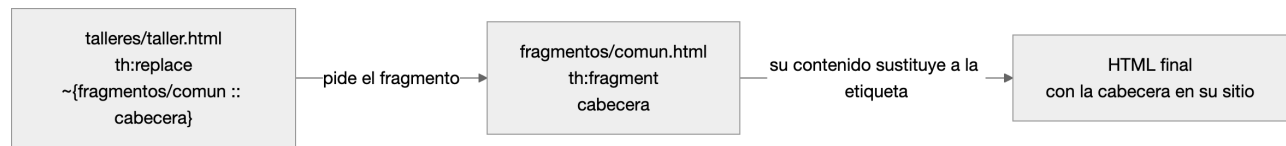
Fichero `src/main/resources/templates/fragmentos/comun.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <header th:fragment="cabecera">
    <a th:href="@{/talleres/1}">Talleres del centro</a>
  </header>
  <footer th:fragment="pie">
    <p>Curso 2026/2027 · Inscripciones en secretaría</p>
  </footer>
</body>
</html>
```

Fichero `src/main/resources/templates/talleres/taller.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${taller != null} ? ${taller.nombre} : 'Taller no encontrado'">Taller</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
  <header th:replace="~{fragmentos/comun :: cabecera}">Cabecera</header>
  <main th:if="${taller != null}">
    <h1 th:text="${taller.nombre}">Nombre del taller</h1>
    <p th:switch="${taller.nivel}">
      <span th:case="'básico'" class="nivel basico">Nivel básico</span>
      <span th:case="'medio'" class="nivel medio">Nivel medio</span>
      <span th:case="*" class="nivel avanzado">Nivel avanzado</span>
    </p>
    <ul>
      <li>Fecha: <span th:text="${#temporals.format(taller.fecha, 'dd/MM/yyyy')}">01/01/2026</span></li>
      <li>Precio: <span th:text="${taller.precio == 0} ? 'Gratis' : |${#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')} €|">0,00 €</span></li>
      <li>Inscritos: ${taller.inscritos} de ${taller.plazas}|>Inscritos: 0 de 0</li>
    </ul>
    <p th:if="${taller.completo}" class="aviso">Taller completo: apúntate a la lista de espera en secretaría.</p>
    <p th:unless="${taller.completo}" th:text="|Quedan ${taller.plazasLibres} plazas libres.">Quedan plazas libres.</p>
  </main>
  <main th:unless="${taller != null}">
    <h1>Taller no encontrado</h1>
    <p>No hay ningún taller con ese número.</p>
  </main>
  <footer th:replace="~{fragmentos/comun :: pie}">Pie</footer>
</body>
</html>
```

Así se enlazan el fichero de la página y el del fragmento:



talleres/taller.html pide el fragmento cabecera con `th:replace`; fragmentos/comun.html lo define con `th:fragment`; su contenido sustituye a la etiqueta en el HTML final

En el HTML que recibe el navegador no queda rastro de los fragmentos: el resultado final es exactamente el mismo que con el `<header>/<footer>` sueltos de antes, solo que ahora vive en un único fichero compartido.

REFERENCIA

`th:replace` frente a `th:insert`

En la plantilla	HTML resultante
<code><header th:replace="~{fragmentos/comun :: cabecera}"></code>	<code><header>...</header></code> : solo el fragmento, sin la etiqueta original
<code><div th:insert="~{fragmentos/comun :: cabecera}"></code>	<code><div><header>...</header></div></code> : el fragmento dentro de la etiqueta que lo pide

Casi siempre se quiere `th:replace`.

EJEMPLO

Un fragmento con parámetros

Un fragmento puede recibir valores, y así la misma cabecera marca en el menú la página en la que estás. El nombre del fragmento lleva sus parámetros, como un método:

```

<header th:fragment="cabecera(activa)">
  <nav>
    <a th:href="@{/}" th:classappend="${activa == 'inicio'} ? 'activa'">Inicio</a>
    <a th:href="@{/talleres}" th:classappend="${activa == 'talleres'} ? 'activa'">Talleres</a>
  </nav>
</header>
  
```

Cada página indica su valor al usarlo:

```

<header th:replace="~{fragmentos/web :: cabecera('talleres')}">Cabecera</header>
  
```

`th:classappend` añade una clase a las que ya tiene la etiqueta, y solo si la condición se cumple.

AVISO

Si el nombre del fragmento o el de su fichero están mal escritos, la página responde con un error 500 y la consola dice «Error resolving fragment: "~{'fragmentos/comun' :: cabecera}": template or fragment could not be resolved». Compara el nombre de `th:replace` con el de `th:fragment`.

RECURSOS**Para saber más**

- **Tutorial de Thymeleaf:** condicionales, `th:switch` y fragmentos, en detalle (en inglés).

<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>

- **Thymeleaf y Spring:** cómo se integra con Spring MVC (en inglés).

<https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>

Práctica

Tres ejercicios para mostrar objetos, decidir qué se enseña y reutilizar plantillas.

ENTRENAMIENTO 3.1**La ficha de un libro**

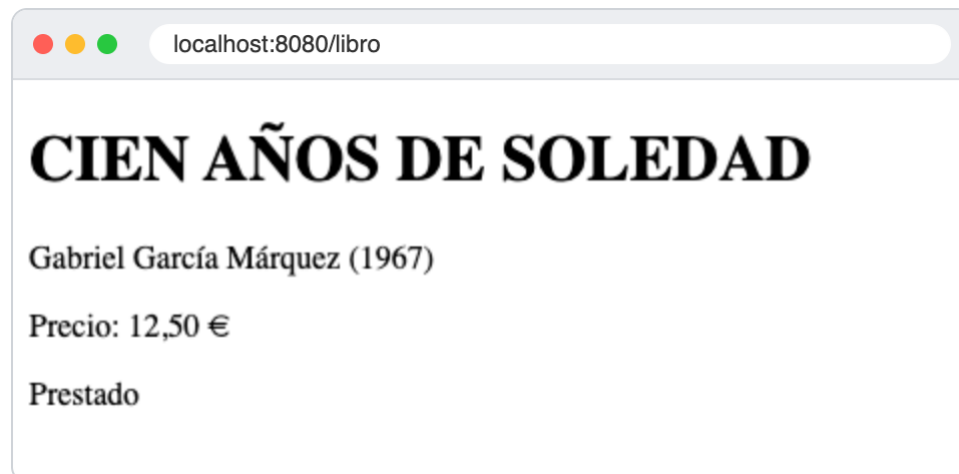
Nivel · Fácil

Crea una clase `Libro` con título, autor, año, precio y si está prestado. Un controlador con la ruta `/libro` entrega uno de ellos a la vista. La vista muestra estos datos.

- el título en mayúsculas;
- el autor y el año, en una sola línea, con el año entre paréntesis;
- el precio con dos decimales y coma, seguido del símbolo €;
- «Prestado» o «Disponible», según corresponda.

PISTAS

- Un campo `boolean` se lee en la plantilla como `libro.prestado`, y su método se llama `isPrestado()`.
- `|${libro.autor} (${libro.anio})|` escribe una línea con dos propiedades y texto fijo.
- `#strings.toUpperCase(...)` y `#numbers.formatDecimal(...)` están en la tabla de expresiones.

RESULTADO ESPERADO

Página localhost:8080/libro con CIEN AÑOS DE SOLEDAD, Gabriel García Márquez (1967), Precio: 12,50 € y Prestado

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Libro.java`:

```
package com.ejemplo.primerproyecto.model;

public class Libro {

    private String titulo;
    private String autor;
    private int anio;
    private double precio;
    private boolean prestado;

    public Libro(String titulo, String autor, int anio, double precio, boolean
prestado) {
        this.titulo = titulo;
        this.autor = autor;
        this.anio = anio;
        this.precio = precio;
        this.prestado = prestado;
    }

    public String getTitulo() {
        return titulo;
    }

    public String getAutor() {
        return autor;
    }

    public int getAnio() {
        return anio;
    }

    public double getPrecio() {
        return precio;
    }
}
```

```

    public boolean isPrestado() {
        return prestado;
    }
}

```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/LibroController.java`:

```

package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.ejemplo.primerproyecto.model.Libro;

@Controller
public class LibroController {

    @GetMapping("/libro")
    public String libro(Model model) {
        model.addAttribute("libro", new Libro("Cien años de soledad",
            "Gabriel García Márquez", 1967, 12.5, true));
        return "libro";
    }
}

```

Fichero `src/main/resources/templates/libro.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${libro.titulo}">Libro</title>
</head>
<body>
<h1 th:text="${#strings.toUpperCase(libro.titulo)}">TÍTULO</h1>
<p th:text="|${libro.autor} (${libro.anio})|">Autor (año)</p>
<p th:text="|Precio: ${#numbers.formatDecimal(libro.precio, 1, 2, 'COMMA')}<br/>€|">Precio: 0,00 €</p>
<p th:text="${libro.prestado} ? 'Prestado' : 'Disponible'">Disponible</p>
</body>
</html>

```

ENTRENAMIENTO 3.2

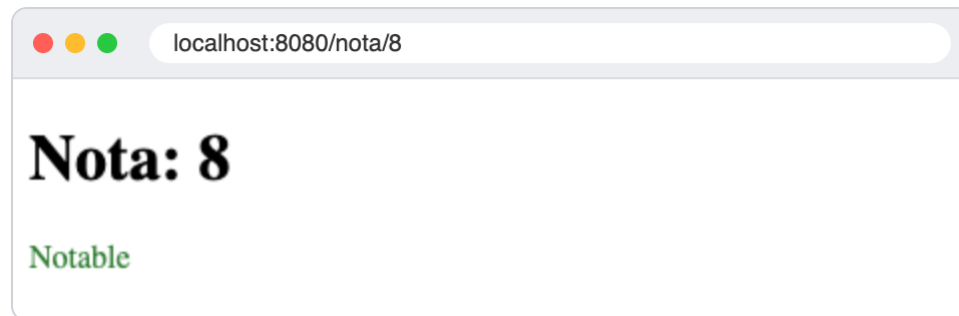
La calificación

Nivel · Intermedio

Haz que `/nota/{nota}` muestre «Nota: N» y, debajo, la calificación: Insuficiente (menos de 5), Suficiente (5), Bien (6), Notable (7 u 8) o Sobresaliente (9 o 10). Las calificaciones de aprobado van en verde y la de suspenso en rojo. Un número fuera de 0 a 10 debe mostrar solo el aviso «Esa nota no es válida: debe estar entre 0 y 10.»

PISTAS

- Cada calificación es una etiqueta con su propio `th:if`.
- Las condiciones se combinan con `and` y `or`, y se niegan con `th:unless`.
- Para el aviso de nota no válida, el resto de calificaciones van dentro de una etiqueta con `th:unless` y la condición contraria.

RESULTADO ESPERADO

Página localhost:8080/nota/8 con Nota: 8 y la calificación Notable en verde

Con `/nota/4` sale Insuficiente en rojo; con `/nota/11`, solo el aviso.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/NotaController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@Controller
public class NotaController {

    @GetMapping("/nota/{nota}")
    public String nota(@PathVariable int nota, Model model) {
        model.addAttribute("nota", nota);
        return "nota";
    }

}
```

Fichero `src/main/resources/templates/nota.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Calificación</title>
<style>
    .suspense { color: #c62828; }
    .aprobado { color: #2e7d32; }
```

```

</style>
</head>
<body>
  <h1 th:text="|Nota: ${nota}|">Nota: 0</h1>
  <p th:if="${nota < 0 or nota > 10}">Esa nota no es válida: debe estar entre 0 y
    10.</p>
  <div th:unless="${nota < 0 or nota > 10}">
    <p th:if="${nota < 5}" class="suspense">Insuficiente</p>
    <p th:if="${nota == 5}" class="aprobado">Suficiente</p>
    <p th:if="${nota == 6}" class="aprobado">Bien</p>
    <p th:if="${nota == 7 or nota == 8}" class="aprobado">Notable</p>
    <p th:if="${nota >= 9}" class="aprobado">Sobresaliente</p>
  </div>
</body>
</html>

```

ENTRENAMIENTO 3.3

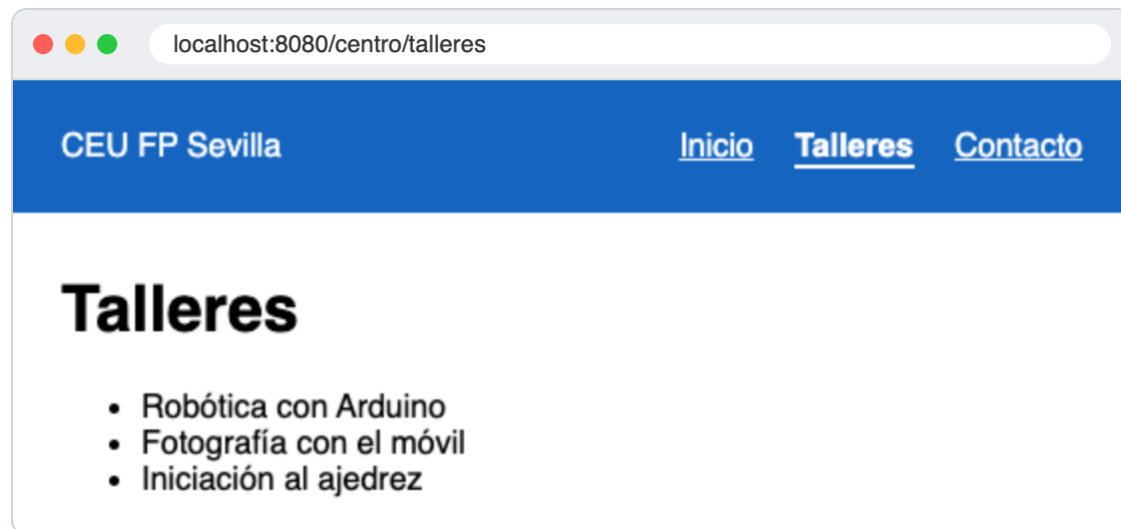
La cabecera de la web, una sola vez

Nivel · Difícil

Toma la web del centro de tres páginas (Inicio, Talleres y Contacto) publicada bajo /centro, cuya cabecera está repetida en las tres plantillas. Escribe la cabeza del documento y la cabecera una sola vez, en fragmentos/web.html, y úsalas en las tres páginas. El título de cada pestaña debe ser «Página · Nombre del centro», y en el menú debe quedar marcada la página en la que estás.

PISTAS

- Dos fragmentos: uno para el <head>, con el título como parámetro, y otro para la cabecera, con la página activa como parámetro.
- Un fragmento puede estar en una etiqueta <head>, y se usa con th:replace en la etiqueta <head> de cada página.
- th:classappend="\${activa == 'talleres'} ? 'activa'" añade la clase activa solo en la página que toca. La clase se define en el CSS.
- El controlador y application.properties no cambian.

RESULTADO ESPERADO

Página localhost:8080/centro/talleres con la cabecera azul y el menú, con la palabra Talleres en negrita y subrayada

UNA SOLUCIÓN

Los tres ficheros de `templates/centro/` son iguales salvo el nombre de la página y su contenido; se muestra el de Talleres.

Fichero `src/main/resources/templates/fragmentos/web.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:fragment="cabeza(titulo)">
<meta charset="UTF-8" />
<title th:text="|${titulo} · ${centro}|">Título</title>
<link rel="stylesheet" th:href="@{/css/centro.css}" />
</head>
<body>
<header th:fragment="cabecera(activa)">
<p th:text="${centro}">Centro</p>
<nav>
<a th:href="@{/}" th:classappend="${activa == 'inicio'} ? 'activa'">Inicio</a>
<a th:href="@{/talleres}" th:classappend="${activa == 'talleres'} ?
'activa'">Talleres</a>
<a th:href="@{/contacto}" th:classappend="${activa == 'contacto'} ?
'activa'">Contacto</a>
</nav>
</header>
</body>
</html>
```

Fichero `src/main/resources/templates/centro/talleres.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragmentos/web :: cabeza('Talleres')}">
<meta charset="UTF-8" />
<title>Talleres</title>
```

```

</head>
<body>
  <header th:replace="~{fragmentos/web :: cabecera('talleres')}">Cabecera</header>
  <main>
    <h1>Talleres</h1>
    <ul>
      <li>Robótica con Arduino</li>
      <li>Fotografía con el móvil</li>
      <li>Iniciación al ajedrez</li>
    </ul>
  </main>
</body>
</html>

```

Fichero `src/main/resources/static/css/centro.css`:

```

nav a.activa {
  font-weight: bold;
  text-decoration: none;
  border-bottom: 2px solid white;
}

```

PÍLDORA

Contenido dinámico en 5 ideas

1. `th:text` escapa el HTML y `th:utext` no: usa `th:text`.
2. Un objeto llega a la vista con `addAttribute` y se lee con `${objeto.propiedad}`.
3. Las expresiones unen textos (`|...|`), operan, comparan (`==`) y dan formato (`#temporals`, `#numbers`).
4. `th:if`, `th:unless` y `th:switch` deciden qué etiquetas se generan; `?.` y `?:` cubren los valores `null`.
5. `th:fragment` y `th:replace` reutilizan trozos de plantilla.

AUTOEVALUACIÓN**Contenido dinámico**

3.1 El modelo tiene `aviso = "<b>0jo</b>"`. ¿Qué muestra `<span th:text='${aviso}'>`?

- a) «Ojo» en negrita.
- b) El texto `<b>0jo</b>`, con las etiquetas visibles.
- c) Nada.
- d) Un error 500.

3.2 ¿Cuándo es aceptable usar `th:utext`?

3.3 Un usuario escribe un comentario con un `<script>` y la página lo muestra con `th:utext`. ¿Qué ocurre?

- a) Se muestra como texto.
- b) El script se ejecuta en el navegador de quien abre la página.
- c) Spring lo elimina antes de enviarlo.
- d) El servidor da un error.

3.4 ¿Qué método de Taller llama la expresión `${taller.plazasLibres}`?

- a) `plazasLibres()`
- b) `getPlazasLibres()`
- c) `isPlazasLibres()`
- d) Ninguno: lee un campo `plazasLibres`.

3.5 Un método boolean `isCompleto()`, ¿cómo se lee en la plantilla?

3.6 Pides `/talleres/9`, que no existe, y la plantilla lee `${taller.nombre}`. ¿Qué pasa?

- a) La etiqueta sale vacía.
- b) Error 404.
- c) Error 500: «Property or field 'nombre' cannot be found on null».
- d) Se muestra el texto de ejemplo.

3.7 Escribe una expresión que una el texto «Plazas: » con las plazas libres del taller.

3.8 ¿Qué resultado da `${taller.inscritos >= taller.plazas}` para un taller con 12 inscritos y 16 plazas?

- a) `true`
- b) `false`
- c) 12
- d) Un error.

3.9 ¿Qué escribe `${taller.completo}` ? 'Completo' : 'Con plazas' para un taller con plazas libres?

3.10 ¿Qué diferencia hay entre `${taller.nivel = 'medio'}` y `${taller.nivel == 'medio'}`?

3.11 ¿Qué muestra `${taller.precio}` sin formato para un precio de 25 euros?

- a) 25

- b) 25,00
- c) 25.0
- d) 25 €

3.12 ¿Qué diferencia hay entre `th:if` y `th:unless`?

3.13 Una etiqueta con `th:if="{taller.completo}"` en un taller con plazas, ¿está oculta con CSS?

- a) Sí, con `display: none`.
- b) Sí, con `visibility: hidden`.
- c) No: no está en el HTML que recibe el navegador.
- d) Depende del navegador.

3.14 ¿Qué hace `th:case="*"` dentro de un `th:switch`?

3.15 ¿Qué muestra `<span th:text="{apodo} ?: 'sin apodo'>` cuando `apodo` es `null`?

3.16 ¿Para qué sirve `th:fragment`?

- a) Para dividir una tabla en filas.
- b) Para marcar un trozo de plantilla con nombre que otras plantillas pueden reutilizar.
- c) Para importar una hoja de estilos.
- d) Para repetir una etiqueta varias veces.

3.17 ¿Qué diferencia hay entre `th:replace` y `th:insert`?

3.18 Al pedir una página sale «Error resolving fragment: ... template or fragment could not be resolved». ¿Qué compruebas?

Respuestas al final de la unidad, en «Autoevaluación · Contenido dinámico».

4. Listas y parámetros

Una lista de talleres, de alumnos o de productos no se escribe a mano, etiqueta a etiqueta: se genera repitiendo una plantilla por cada elemento. Y esa lista se puede filtrar desde la propia dirección, sin tocar el controlador cada vez.

Repetir con `th:each`

Hasta ahora cada plantilla mostraba un objeto. Cuando el modelo lleva una lista, `th:each` repite una etiqueta (y todo lo que hay dentro) una vez por cada elemento.

DEFINICIONES

`th:each="elemento : ${lista}"`: repite la etiqueta una vez por cada valor de la lista; dentro de ella, `elemento` es el valor de esa vuelta.

Variable de estado: con `th:each="elemento, est : ${lista}"`, `est` da información de la vuelta actual: `est.index` (empieza en 0), `est.count` (empieza en 1), `est.size`, `est.even` / `est.odd`.

`#lists`: utilidades de Thymeleaf para listas, como `#lists.isEmpty(lista)` o `#lists.size(lista)`.

EJEMPLO

La lista de talleres, en una tabla

Cada fila usa `est.count` para numerar y `est.even` para alternar el color de fondo. Si la lista está vacía, la tabla no se genera y sale un aviso en su lugar.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class TallerController {

    private final List<Taller> talleres = List.of(
        new Taller(1, "Robótica con Arduino", "medio",
            LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
            LocalDate.of(2026, 10, 21), 20, 20, 0),
    );
}
```

```

        new Taller(3, "Ajedrez de competición", "avanzado",
        LocalDate.of(2026, 11, 4), 12, 3, 10.5));

    @GetMapping("/talleres")
    public String lista(Model model) {
        model.addAttribute("talleres", talleres);
        return "talleres/lista";
    }
}

```

Fichero `src/main/resources/templates/talleres/lista.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<h1>Talleres del centro</h1>
<p th:if="${#lists.isEmpty(talleres)}">No hay talleres que mostrar.</p>
<table th:unless="${#lists.isEmpty(talleres)}">
    <thead>
        <tr>
            <th>#</th>
            <th>Nombre</th>
            <th>Nivel</th>
            <th>Plazas libres</th>
        </tr>
    </thead>
    <tbody>
        <tr th:each="taller, est : ${talleres}" th:class="${est.even} ? 'par' : 'impar'">
            <td th:text="${est.count}">1</td>
            <td th:text="${taller.nombre}">Nombre</td>
            <td th:text="${taller.nivel}">nivel</td>
            <td th:text="${taller.plazasLibres}">0</td>
        </tr>
    </tbody>
</table>
<p th:text="|Hay ${#lists.size(talleres)} talleres en total.|">Hay N talleres en
total.</p>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

http://localhost:8080/talleres genera una fila por taller, numerada y alternando el color:



Tabla de talleres numerada de 1 a 3, con Robótica con Arduino, Fotografía con el móvil y Ajedrez de competición, y el texto Hay 3 talleres en total

AVISO

`${lista == null || lista.size() == 0}` también comprueba si una lista está vacía, pero falla si `lista` es `null`: `size()` no se puede llamar sobre `null`. `#lists.isEmpty(lista)` resuelve los dos casos, `null` y lista vacía, sin fallar.

Enlaces con datos

Cada fila de la tabla necesita enlazar a su propia ficha, con el identificador de ese taller. Thymeleaf construye esa URL con una sintaxis dentro de `@{...}`.

REFERENCIA

Construir la URL de cada fila

Escrito en la plantilla	HTML que llega al navegador
<code>@{/talleres/{id}(id=\${taller.id})}</code>	<code>/talleres/3</code>
<code>@{/talleres(nivel='medio')}</code>	<code>/talleres?nivel=medio</code>
<code>@{/paginas(pagina=\${pagina + 1}, tamano=\${tamano})}</code>	<code>/paginas?pagina=2&tamano=2</code>

Con llaves `{...}` se rellena un segmento de la ruta; con paréntesis `(...)`, parámetros de la dirección (`?clave=valor`). Los dos se pueden combinar y llevar más de un valor, separados por comas.

AVISO

`@{'/talleres/' + ${taller.id}}` también genera la misma URL, uniendo texto a mano. Funciona, pero es más frágil: hay que acordarse de las barras y de los signos de interrogación y `&` cuando hay varios parámetros. La sintaxis con `{...}` y `(...)` los añade sola.

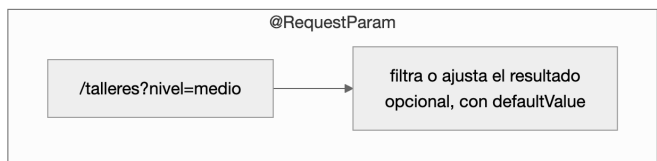
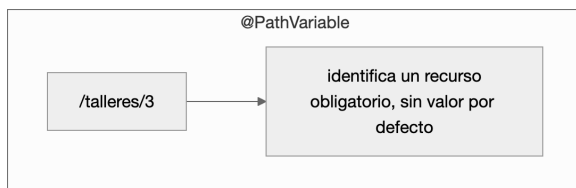
Filtrar con @RequestParam

Una variable de ruta identifica un recurso concreto, como el taller número 3. Filtrar una lista, en cambio, no encaja ahí: puede o no llevar filtro, y con distintos valores. Para eso están los parámetros de la dirección, después del `?`.

DEFINICIÓN

`@RequestParam`: anotación de un parámetro del método que recibe un valor de la dirección tras el `?` (`?nivel=medio`). Con `required = false` es opcional, y `defaultValue` fija el valor cuando no está.

Así se resumen las dos anotaciones:



`@PathVariable` identifica un recurso, obligatorio y sin valor por defecto; `@RequestParam` filtra o ajusta el resultado, opcional y con `defaultValue`

EJEMPLO**Filtrar la lista por nivel**

Cuatro enlaces llevan a la misma ruta con un `nivel` distinto cada uno; el controlador filtra la lista según ese valor. Sin el parámetro, `nivel` vale `"todos"` y se ven todos los talleres.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```

package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
  
```

```
import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class TallerController {

    private final List<Taller> talleres = List.of(
        new Taller(1, "Robótica con Arduino", "medio",
        LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
        LocalDate.of(2026, 10, 21), 20, 20, 0),
        new Taller(3, "Ajedrez de competición", "avanzado",
        LocalDate.of(2026, 11, 4), 12, 3, 10.5),
        new Taller(4, "Costura creativa", "básico", LocalDate.of(2026,
        10, 28), 15, 6, 8),
        new Taller(5, "Programación en Scratch", "medio",
        LocalDate.of(2026, 11, 11), 18, 18, 0));

    @GetMapping("/talleres")
    public String lista(Model model, @RequestParam(required = false, defaultValue =
    "todos") String nivel) {
        List<Taller> filtrados = new ArrayList<>();
        for (Taller t : talleres) {
            if (nivel.equals("todos") || t.getNivel().equals(nivel)) {
                filtrados.add(t);
            }
        }
        model.addAttribute("talleres", filtrados);
        model.addAttribute("nivel", nivel);
        return "talleres/lista";
    }

    @GetMapping("/talleres/{id}")
    public String taller(@PathVariable int id, Model model) {
        Taller taller = null;
        for (Taller t : talleres) {
            if (t.getId() == id) {
                taller = t;
            }
        }
        model.addAttribute("taller", taller);
        return "talleres/taller";
    }
}
```

Fichero `src/main/resources/templates/talleres/lista.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<h1>Talleres del centro</h1>
<nav>
    Filtrar por nivel:
    <a th:href="@{/talleres(nivel='todos')}">Todos</a>
    <a th:href="@{/talleres(nivel='básico')}">Básico</a>
    <a th:href="@{/talleres(nivel='medio')}">Medio</a>
</nav>
</body>
</html>
```

```

    <a th:href="@{/talleres(nivel='avanzado')}">Avanzado</a>
</nav>
<p th:text="|Nivel: ${nivel}|">Nivel: todos</p>
<p th:if="${#lists.isEmpty(talleres)}">No hay talleres con ese nivel.</p>
<table th:unless="${#lists.isEmpty(talleres)}">
    <thead>
        <tr>
            <th>Nombre</th>
            <th>Nivel</th>
            <th>Plazas libres</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        <tr th:each="taller, est : ${talleres}" th:class="${est.even} ? 'par' : 'impar'">
            <td th:text="${taller.nombre}">Nombre</td>
            <td th:text="${taller.nivel}">nivel</td>
            <td th:text="${taller.plazasLibres}">0</td>
            <td><a th:href="@{/talleres/{id}(id=${taller.id})}">Ver ficha</a></td>
        </tr>
    </tbody>
</table>
</body>
</html>

```

Fichero `src/main/resources/templates/talleres/taller.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${taller.nombre}">Taller</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
    <a th:href="@{/talleres}">Volver a la lista</a>
    <h1 th:text="${taller.nombre}">Nombre</h1>
    <p th:text="|Nivel: ${taller.nivel}|">Nivel</p>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

`http://localhost:8080/talleres?nivel=básico` deja solo los talleres de nivel básico, y cada fila enlaza a su ficha con `@{/talleres/{id}(id=${taller.id})}`:



Talleres del centro filtrado por nivel básico: Fotografía con el móvil y Costura creativa, cada una con su enlace Ver ficha

REFERENCIA

`@PathVariable` frente a `@RequestParam`

	@PathVariable	@RequestParam
Dónde va	segmento de la ruta: <code>/talleres/3</code>	tras el <code>?</code> : <code>/talleres?nivel=medio</code>
Obligatorio	sí, por defecto	configurable con <code>required</code>
Valor por defecto	no existe	sí, con <code>defaultValue</code>
Para qué se usa	identificar un recurso concreto	filtrar, buscar o paginar

RECURSOS

Para saber más

- **Handling parameters:** `@RequestParam` con todas sus opciones, en la documentación de Spring (en inglés).
<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/requestparam.html>
- **Tutorial de Thymeleaf:** `th:each` y las utilidades `#lists`, con más ejemplos (en inglés).
<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>

Práctica

Tres ejercicios para listar, enlazar y filtrar.

ENTRENAMIENTO 4.1

Lista de alumnos

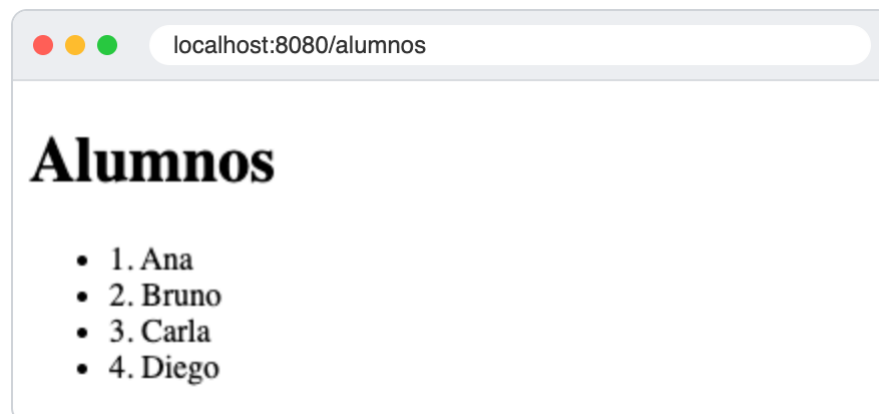
Nivel · Fácil

Con una lista de nombres en el controlador, muestra en `/alumnos` una lista numerada: «1. Ana», «2. Bruno»...

PISTAS

- `List.of("Ana", "Bruno", "Carla", "Diego")` crea la lista directamente en el controlador.
- La variable de estado de `th:each` da el número de cada vuelta con `est.count`.

RESULTADO ESPERADO



Página localhost:8080/alumnos con la lista 1. Ana, 2. Bruno, 3. Carla, 4. Diego

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/AlumnoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class AlumnoController {

    @GetMapping("/alumnos")
    public String lista(Model model) {
        model.addAttribute("alumnos", List.of("Ana", "Bruno", "Carla",
"Diego"));
        return "alumnos";
    }
}
```

```
}  
  
}
```

Fichero `src/main/resources/templates/alumnos.html`:

```
<!DOCTYPE html>  
<html lang="es" xmlns:th="http://www.thymeleaf.org">  
  <head>  
    <meta charset="UTF-8" />  
    <title>Alumnos</title>  
  </head>  
  <body>  
    <h1>Alumnos</h1>  
    <ul>  
      <li th:each="alumno, est : ${alumnos}" th:text="${est.count}. ${alumno}">1.  
        Nombre</li>  
    </ul>  
  </body>  
</html>
```

ENTRENAMIENTO 4.2

Buscador de talleres

Nivel · Intermedio

Haz que `/buscar?texto=algo` muestre los talleres cuyo nombre contiene ese texto, sin distinguir mayúsculas de minúsculas. Sin el parámetro, no debe fallar: debe mostrar todos los talleres.

PISTAS

- `@RequestParam(required = false, defaultValue = "")` hace que un texto vacío no filtre nada, porque todo nombre lo contiene.
- `texto.toLowerCase()` y `nombre.toLowerCase()` antes de `contains` ignoran mayúsculas y minúsculas.

RESULTADO ESPERADO



Página localhost:8080/buscar?texto=con con Buscando: 'con' y los resultados Robótica con Arduino y Fotografía con el móvil

/buscar?texto=zzz muestra «Sin resultados.»; /buscar (sin parámetro) muestra los tres talleres.

UNA SOLUCIÓN

Fichero src/main/java/com/ejemplo/primerproyecto/controller/BuscadorController.java:

```
package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class BuscadorController {

    private final List<Taller> talleres = List.of(
        new Taller(1, "Robótica con Arduino", "medio",
            LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
            LocalDate.of(2026, 10, 21), 20, 20, 0),
        new Taller(3, "Ajedrez de competición", "avanzado",
            LocalDate.of(2026, 11, 4), 12, 3, 10.5));

    @GetMapping("/buscar")
    public String buscar(Model model, @RequestParam(required = false,
        defaultValue = "") String texto) {
        List<Taller> resultado = new ArrayList<>();
        for (Taller t : talleres) {
            if
(t.getNombre().toLowerCase().contains(texto.toLowerCase())) {
                resultado.add(t);
            }
        }
    }
}
```

```

    }
    model.addAttribute("talleres", resultado);
    model.addAttribute("texto", texto);
    return "buscar";
}
}

```

Fichero `src/main/resources/templates/buscar.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Buscar talleres</title>
</head>
<body>
<h1>Buscar talleres</h1>
<p th:text="'Buscando: '${texto}'">Buscando: ''</p>
<p th:if="${#lists.isEmpty(talleres)}">Sin resultados.</p>
<ul th:unless="${#lists.isEmpty(talleres)}">
<li th:each="taller : ${talleres}" th:text="${taller.nombre}">Nombre</li>
</ul>
</body>
</html>

```

ENTRENAMIENTO 4.3

Talleres por páginas

Nivel · Difícil

Muestra los talleres de dos en dos, con `pagina` y `tamano` como parámetros de la dirección (por defecto, página 1 de tamaño 2). Añade los enlaces «Anterior» y «Siguiente» solo cuando tengan sentido, y evita que una página fuera de rango (por ejemplo, `pagina=99`) dé un error.

PISTAS

- El número de páginas es $(total + tamaño - 1) / tamaño$ (división entera).
- Antes de calcular el trozo de la lista, ajusta `pagina` a los límites 1 y `totalPaginas`.
- `List.subList(desde, hasta)` devuelve el trozo de la lista.
- Cada enlace pasa `pagina` y `tamano` con `@{/paginas(pagina=..., tamano=${tamano})}`.

RESULTADO ESPERADO

Página localhost:8080/paginas?pagina=2 con Ajedrez de competición y Costura creativa, Página 2 de 3, y los enlaces Anterior y Siguiente

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/PaginaController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class PaginaController {

    private final List<Taller> talleres = List.of(
        new Taller(1, "Robótica con Arduino", "medio",
        LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
        LocalDate.of(2026, 10, 21), 20, 20, 0),
        new Taller(3, "Ajedrez de competición", "avanzado",
        LocalDate.of(2026, 11, 4), 12, 3, 10.5),
        new Taller(4, "Costura creativa", "básico",
        LocalDate.of(2026, 10, 28), 15, 6, 8),
        new Taller(5, "Programación en Scratch", "medio",
        LocalDate.of(2026, 11, 11), 18, 18, 0));

    @GetMapping("/paginas")
    public String pagina(Model model,
        @RequestParam(required = false, defaultValue = "1") int
        pagina,
        @RequestParam(required = false, defaultValue = "2") int
        tamano) {
        int total = talleres.size();
        int totalPaginas = (total + tamano - 1) / tamano;
```

```

        if (pagina < 1) {
            pagina = 1;
        }
        if (pagina > totalPaginas) {
            pagina = totalPaginas;
        }
        int desde = (pagina - 1) * tamano;
        int hasta = Math.min(desde + tamano, total);
        model.addAttribute("talleres", talleres.subList(desde, hasta));
        model.addAttribute("pagina", pagina);
        model.addAttribute("tamano", tamano);
        model.addAttribute("totalPaginas", totalPaginas);
        return "paginas";
    }
}

```

Fichero src/main/resources/templates/paginas.html:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres por páginas</title>
</head>
<body>
<h1>Talleres por páginas</h1>
<ul>
<li th:each="taller : ${talleres}" th:text="${taller.nombre}">Nombre</li>
</ul>
<p th:text="|Página ${pagina} de ${totalPaginas}|">Página 1 de 1</p>
<nav>
<a th:if="${pagina > 1}" th:href="@{/paginas(pagina=${pagina - 1}, tamano=${tamano})}">Anterior</a>
<a th:if="${pagina < totalPaginas}" th:href="@{/paginas(pagina=${pagina + 1}, tamano=${tamano})}">Siguiente</a>
</nav>
</body>
</html>

```

PÍLDORA

Listas y parámetros en 4 ideas

1. `th:each="elemento, est : ${lista}"` repite la etiqueta; `est.count`, `est.even` ... dan la posición.
2. `#lists.isEmpty(lista)` comprueba `null` y lista vacía sin fallar.
3. `@{/ruta/{id}(id=${valor})}` construye URLs con segmentos y parámetros, sin concatenar a mano.
4. `@PathVariable` identifica un recurso; `@RequestParam` (con `required` y `defaultValue`) filtra o pagina.

AUTOEVALUACIÓN**Listas y parámetros**

4.1 ¿Qué hace `th:each="taller : ${talleres}"` sobre una etiqueta `<tr>`?

- a) La oculta si la lista está vacía.
- b) La repite una vez por cada elemento de `talleres`.
- c) La convierte en una tabla.
- d) Ordena la lista.

4.2 Con `th:each="taller, est : ${talleres}"`, ¿qué diferencia hay entre `est.index` y `est.count`?

4.3 ¿Qué comprueba `#lists.isEmpty(talleres)` que `${talleres.size() == 0}` no cubre?

- a) Que la lista tenga más de un elemento.
- b) El caso en que `talleres` sea `null`.
- c) Que los elementos no estén repetidos.
- d) Nada: hacen lo mismo.

4.4 ¿Qué genera `@{/talleres/{id}(id=${taller.id})}` para un taller con `id` 3?

4.5 ¿Qué genera `@{/talleres(nivel='medio')}`?

- a) `/talleres/medio`
- b) `/talleres?nivel=medio`
- c) `/talleres{nivel=medio}`
- d) Un error: falta `{id}`.

4.6 Escribe la expresión que enlaza a `/paginas` pasando `pagina` con el valor de una variable siguiente y `tamano` con el valor de `tamano`.

4.7 ¿Cuál es la diferencia principal entre `@PathVariable` y `@RequestParam`?

4.8 Un método tiene

`@RequestParam(required = false, defaultValue = "todos") String nivel`. ¿Qué valor recibe `nivel` en `/talleres` (sin parámetro)?

- a) `null`
- b) Una cadena vacía.
- c) `"todos"`
- d) Da un error 400.

4.9 ¿Por qué `/buscar` (sin texto) con `defaultValue = ""` no falla y muestra todos los talleres?

4.10 ¿Qué le pasaría a `/buscar` sin `defaultValue` en texto?

4.11 En la paginación, ¿por qué hay que ajustar `pagina` a los límites 1 y `totalPaginas` antes de calcular desde y hasta?

4.12 ¿Por qué el enlace «Anterior» de la paginación lleva `th:if="${pagina > 1}"`?

4.13 Con talleres a null (no una lista vacía, sino sin asignar), ¿qué hace `<table th:unless="${#lists.isEmpty(talleres)}">`?

- a) Lanza una excepción.
- b) No genera la tabla.
- c) Genera una tabla vacía.
- d) La tabla se genera sin filas.

4.14 Escribe con `th:each` y una variable de estado una fila que alterne su clase CSS entre "par" e "impar".

4.15 ¿Para qué sirve concatenar (`@{'/talleres/' + ${taller.id}}`) en lugar de usar `{id}(id=...)`? ¿Qué desventaja tiene?

4.16 ¿Qué elemento HTML repite `th:each="animal : ${animales}"` sobre un `<option>` dentro de un `<select>`?

Respuestas al final de la unidad, en «Autoevaluación · Listas y parámetros».

5. Formularios

Hasta ahora, cada petición leía datos. Un formulario los envía: se apunta a un taller, se da de alta uno nuevo, se edita. Eso cambia cómo se piden los datos, cómo se guardan y qué pasa si el navegador repite la petición.

GET y POST

Todas las peticiones que has hecho hasta ahora eran `GET`. Sirve para pedir una página, pero no está pensado para modificar nada: el navegador puede repetir un `GET` sin avisar (al recargar, al volver atrás, al guardarlo en marcadores), y si ese `GET` cambia algo, se cambia otra vez sin que nadie lo decida.

DEFINICIONES

`GET`: pide datos. No debería tener efectos: pedir la misma página dos veces debe dejar todo igual (se dice que es *idempotente*).

`POST`: envía datos para que el servidor haga algo con ellos: guardar, crear, modificar.

`@PostMapping`: como `@GetMapping`, pero solo para peticiones `POST`.

EJEMPLO

El problema de escribir con GET

Un enlace normal genera siempre una petición `GET`. Si ese enlace incrementa un contador en el servidor, repetir la misma dirección (recargar la página, volver a pulsar el enlace) lo incrementa otra vez, sin avisar.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/FavoritoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class FavoritoController {

    private int veces = 0;

    @GetMapping("/favorito/sumar")
    public String sumar(Model model) {
        veces++;
        model.addAttribute("veces", veces);
        return "favorito";
    }

}
```

COMPRUEBA QUE FUNCIONA

Pedir tres veces `http://localhost:8080/favorito/sumar` (como pulsar el enlace tres veces, o recargar dos):

```
GET /favorito/sumar → Marcado como favorito 1 veces.
GET /favorito/sumar → Marcado como favorito 2 veces.
GET /favorito/sumar → Marcado como favorito 3 veces.
```

El navegador nunca avisa al repetir un `GET`, así que este error es fácil de no notar hasta que alguien recarga la página y ve el contador duplicado.

REFERENCIA**Cuándo usar cada uno**

	GET	POST
Para qué	pedir una página, filtrar, buscar	guardar, crear, modificar, enviar un formulario
Datos	en la propia dirección (<code>?clave=valor</code>)	en el cuerpo de la petición, no en la dirección
Repetirla	segura: no cambia nada	el navegador avisa antes de reenviarla
En Thymeleaf	un enlace, <code><a th:href="..."></code>	un formulario, <code><form method="post"></code>

Formularios con `th:object` y `th:field`

Un formulario podría leerse campo a campo con `@RequestParam`, uno por cada `<input>`. Thymeleaf ofrece algo más cómodo: ligar todo el formulario a un objeto, y que Spring lo rellene solo.

DEFINICIONES

`th:object="${objeto}"`: fija, en un `<form>`, el objeto sobre el que trabajan sus campos.

`th:field="*{propiedad}"`: liga un `<input>` (o `<select>`) a una propiedad de ese objeto: pone su valor al mostrar el formulario y, al enviarlo, Spring reconstruye el objeto con lo escrito.

`*{...}`: dentro de un `th:object`, accede a una propiedad de ese objeto; es más corto que repetir `${objeto.propiedad}`.

EJEMPLO**Un formulario para los talleres**

El formulario usa una clase propia, `TallerFormulario`, con solo los campos que se editan (no la fecha ni los inscritos, que no cambian desde aquí). Necesita un constructor sin parámetros y sus `set`, para que Spring pueda crearlo y rellenarlo.

Fichero `src/main/java/com/ejemplo/primerproyecto/model/TallerFormulario.java`:

```
package com.ejemplo.primerproyecto.model;

public class TallerFormulario {

    private int id; // 0 significa que es un taller nuevo
    private String nombre;
    private String nivel;
    private int plazas;

    public TallerFormulario() {
    }

    public TallerFormulario(int id, String nombre, String nivel, int plazas) {
        this.id = id;
        this.nombre = nombre;
        this.nivel = nivel;
        this.plazas = plazas;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getNivel() {
        return nivel;
    }

    public void setNivel(String nivel) {
        this.nivel = nivel;
    }

    public int getPlazas() {
        return plazas;
    }

    public void setPlazas(int plazas) {
        this.plazas = plazas;
    }
}
```

Fichero `src/main/resources/templates/talleres/formulario.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
```

```

<title th:text="${tallerFormulario.id == 0} ? 'Nuevo taller' : 'Editar taller'">Formulario</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<h1 th:text="${tallerFormulario.id == 0} ? 'Nuevo taller' : 'Editar taller'">Formulario</h1>
<form th:action="@{/talleres/guardar}" th:object="${tallerFormulario}" method="post">
<input type="hidden" th:field="*{id}" />
<label>Nombre
<input type="text" th:field="*{nombre}" />
</label>
<label>Nivel
<select th:field="*{nivel}">
<option value="básico">Básico</option>
<option value="medio">Medio</option>
<option value="avanzado">Avanzado</option>
</select>
</label>
<label>Plazas
<input type="number" th:field="*{plazas}" />
</label>
<button type="submit">Guardar</button>
</form>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

El formulario vacío, en /talleres/nuevo:

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/talleres/nuevo'. The main content area has a heading 'Nuevo taller'. Below the heading are three form elements: a text input labeled 'Nombre', a dropdown menu labeled 'Nivel' with 'Básico' selected, and a number input labeled 'Plazas' with the value '0'. At the bottom of the form is a green button labeled 'Guardar'.

Formulario Nuevo taller con los campos Nombre, Nivel (Básico) y Plazas (0), y el botón Guardar

`th:field="*{nivel}"` selecciona sola la opción que coincide con el valor del objeto: al editar un taller de nivel medio, el `<select>` ya muestra «Medio» sin nada más que escribir.

AVISO

`th:field` genera el `id` y el `name` del campo a partir del nombre de la propiedad. Si `TallerFormulario` no tuviera el método `getNivel()/setNivel(...)`, `th:field="{nivel}"` fallaría al generar la página: Thymeleaf necesita los dos para leer el valor inicial y para saber en qué tipo convertir lo que llegue.

Estado en memoria y @PostMapping

Los talleres, hasta ahora, eran una lista fija creada al arrancar. Para dar de alta uno nuevo hace falta una lista que se pueda modificar, y que esos cambios se mantengan mientras la aplicación siga en marcha.

DEFINICIÓN

Estado en memoria: datos que vive el controlador guarda en un campo, no en la base de datos. Como el controlador es un singleton (una sola instancia para toda la aplicación), esos datos se comparten entre todas las peticiones mientras el servidor está arrancado; al reiniciarlo, se pierden.

EJEMPLO**Dar de alta un taller**

La lista pasa de `List.of(...)` (fija) a `new ArrayList<>(List.of(...))` (modificable). El método `guardar`, con `@PostMapping`, recibe el formulario ya relleno y añade un `Taller` nuevo con un `id` calculado.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Taller;
import com.ejemplo.primerproyecto.model.TallerFormulario;

@Controller
public class TallerController {

    // estado en memoria: una lista mutable, la misma para todas las peticiones
    // mientras la aplicación esté arrancada
    private final List<Taller> talleres = new ArrayList<>(List.of(
        new Taller(1, "Robótica con Arduino", "medio",
        LocalDate.of(2026, 10, 14), 16, 12, 25),
```

```

        new Taller(2, "Fotografía con el móvil", "básico",
        LocalDate.of(2026, 10, 21), 20, 20, 0));

    @GetMapping("/talleres")
    public String lista(Model model) {
        model.addAttribute("talleres", talleres);
        return "talleres/lista";
    }

    @GetMapping("/talleres/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("tallerFormulario", new TallerFormulario());
        return "talleres/formulario";
    }

    @GetMapping("/talleres/editar/{id}")
    public String editar(@PathVariable int id, Model model) {
        Taller taller = buscar(id);
        model.addAttribute("tallerFormulario",
            new TallerFormulario(taller.getId(), taller.getNombre(),
            taller.getNivel(), taller.getPlazas()));
        return "talleres/formulario";
    }

    @PostMapping("/talleres/guardar")
    public String guardar(@ModelAttribute("tallerFormulario") TallerFormulario
    formulario) {
        if (formulario.getId() == 0) {
            int nuevoId =
            talleres.stream().mapToInt(Taller::getId).max().orElse(0) + 1;
            talleres.add(new Taller(nuevoId, formulario.getNombre(),
            formulario.getNivel(), LocalDate.now(),
            formulario.getPlazas(), 0, 0));
        } else {
            for (int i = 0; i < talleres.size(); i++) {
                Taller anterior = talleres.get(i);
                if (anterior.getId() == formulario.getId()) {
                    talleres.set(i, new Taller(anterior.getId(),
            formulario.getNombre(), formulario.getNivel(),
            formulario.getPlazas(), anterior.getInscritos(),
            anterior.getFecha(),
            anterior.getPrecio()));
                }
            }
            return "redirect:/talleres";
        }
    }

    private Taller buscar(int id) {
        for (Taller t : talleres) {
            if (t.getId() == id) {
                return t;
            }
        }
        return null;
    }
}

```

Fichero `src/main/resources/templates/talleres/lista.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<h1>Talleres del centro</h1>
<a th:href="@{/talleres/nuevo}">Nuevo taller</a>
<table>
<thead>
<tr>
<th>Nombre</th>
<th>Nivel</th>
<th>Plazas</th>
<th></th>
</tr>
</thead>
<tbody>
<tr th:each="taller : ${talleres}">
<td th:text="${taller.nombre}">Nombre</td>
<td th:text="${taller.nivel}">nivel</td>
<td th:text="${taller.plazas}">0</td>
<td><a th:href="@{/talleres/editar/{id}(id=${taller.id})}">Editar</a></td>
</tr>
</tbody>
</table>
</body>
</html>
```

COMPRUEBA QUE FUNCIONA

Antes de dar de alta nada, la lista tiene dos talleres:

localhost:8080/talleres

Talleres del centro

[Nuevo taller](#)

Nombre	Nivel	Plazas	
Robótica con Arduino	medio	16	Editar
Fotografía con el móvil	básico	20	Editar

Tabla con Robótica con Arduino y Fotografía con el móvil, cada uno con su enlace Editar, y el enlace Nuevo taller arriba

Se envía el formulario con `POST /talleres/guardar` y `id=0`, `nombre=Costura creativa`, `nivel=básico`, `plazas=15`; la aplicación responde `302` con `Location: /talleres`. Al pedir la lista, aparece el taller nuevo, con `id 3`. Un segundo envío con `id=1` (edición) sustituye el taller «Robótica con Arduino» por los datos nuevos, sin crear una fila más:



Nombre	Nivel	Plazas	
Robótica avanzada	avanzado	10	Editar
Fotografía con el móvil	básico	20	Editar
Costura creativa	básico	15	Editar

Tabla con Robótica avanzada (editada), Fotografía con el móvil y Costura creativa (nueva)

AMPLIACIÓN

Por qué se sustituye el Taller entero

Taller no tiene métodos `set`: sus campos no se pueden cambiar después de crearlo. Al editar, el controlador no modifica el objeto guardado, crea uno nuevo con los datos actualizados y lo pone en su posición de la lista, con `talleres.set(i, nuevo)`. Es una forma de trabajar con objetos inmutables: en vez de mutarlos, se sustituyen.

redirect: y el patrón POST-Redirect-GET

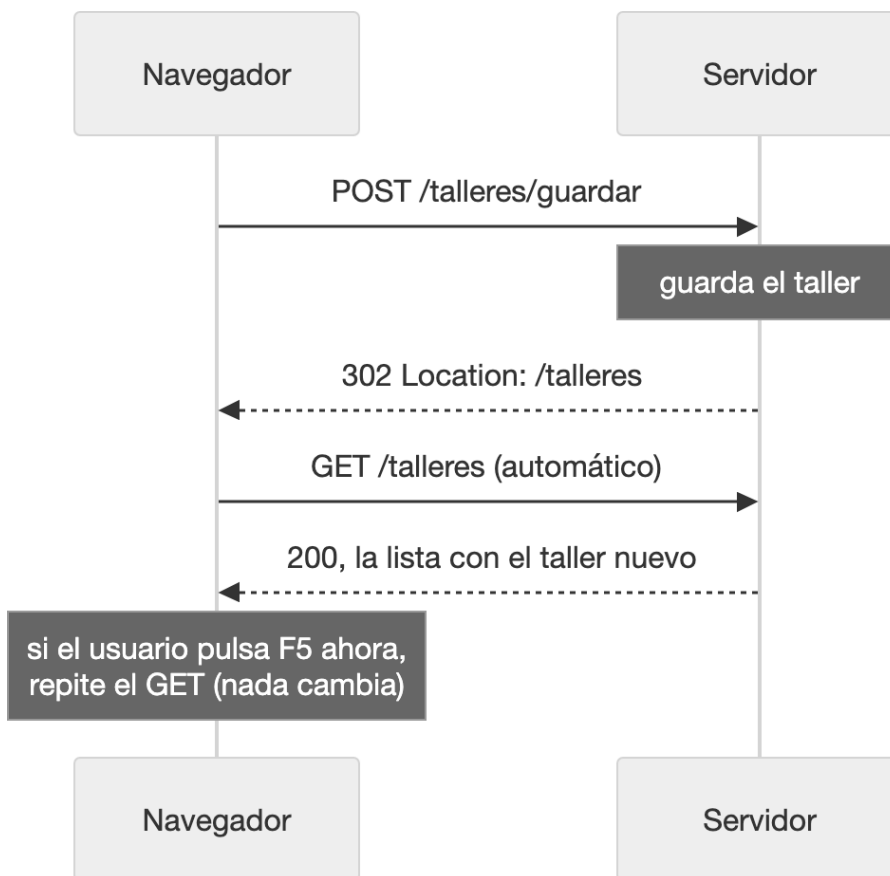
El método `guardar` no devuelve una vista: devuelve `"redirect:/talleres"`. Eso cambia lo que ve el navegador y evita un problema parecido al de escribir con `GET`, pero por el lado del `POST`.

DEFINICIONES

`redirect::` un valor de retorno que empieza así hace que Spring, en vez de renderizar una vista, responda con un código 302 y una cabecera `Location`; el navegador hace entonces una nueva petición GET a esa dirección.

PRG (Post-Redirect-Get): el patrón de responder a un POST con una redirección a una página que se pide por GET. Así, lo último que el navegador recuerda haber enviado es un GET, seguro de repetir.

Así se encadenan las tres peticiones:



POST /talleres/guardar; el servidor guarda el taller y responde 302 Location /talleres; el navegador hace GET /talleres automáticamente; si el usuario pulsa F5 ahora, repite el GET, que no cambia nada

EJEMPLO**Sin redirect, un F5 duplica el taller**

Esta versión del controlador guarda el taller y devuelve directamente la vista de la lista, sin `redirect::`. Repetir la misma petición POST (lo que haría un F5 sobre la página resultante) da de alta el taller otra vez.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
package com.ejemplo.primerproyecto.controller;
```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Taller;
import com.ejemplo.primerproyecto.model.TallerFormulario;

@Controller
public class TallerController {

    private final List<Taller> talleres = new ArrayList<>();

    @GetMapping("/talleres")
    public String lista(Model model) {
        model.addAttribute("talleres", talleres);
        return "talleres/lista";
    }

    @GetMapping("/talleres/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("tallerFormulario", new TallerFormulario());
        return "talleres/formulario";
    }

    // sin redirect: al terminar, renderiza la lista directamente en la respuesta del
    POST
    @PostMapping("/talleres/guardar")
    public String guardar(@ModelAttribute("tallerFormulario") TallerFormulario
formulario, Model model) {
        int nuevoId = talleres.stream().mapToInt(Taller::getId).max().orElse(0)
+ 1;
        talleres.add(new Taller(nuevoId, formulario.getNombre(),
formulario.getNivel(), LocalDate.now(),
formulario.getPlazas(), 0, 0));
        model.addAttribute("talleres", talleres);
        return "talleres/lista";
    }
}

```

COMPRUEBA QUE FUNCIONA

Se envía dos veces el mismo POST /talleres/guardar con id=0&nombre=Costura creativa&...:

```

POST /talleres/guardar → 200, «1 talleres»: Costura creativa
POST /talleres/guardar → 200, «2 talleres»: Costura creativa, Costura creativa

```

Con redirect:, la última petición que queda es un GET /talleres; repetirla no añade nada. Sin él, la última petición es el POST que da de alta, y repetirla lo repite.

AVISO

Los navegadores sí avisan al recargar una página que llegó por `POST` («¿Reenviar los datos del formulario?»), pero solo en ese caso concreto: no avisan si el `GET` es el que escribe, como en el primer apartado. El patrón PRG evita depender de ese aviso.

RECURSOS**Para saber más**

- **Handling Forms:** el formulario completo (mostrar, enviar, redirigir) en el tutorial de Thymeleaf (en inglés).

<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#creating-a-form>

- **HTTP request methods:** `GET` y `POST`, con su idempotencia, en MDN.

<https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods>

Práctica

Tres ejercicios para escribir formularios, guardar en memoria y aplicar PRG.

ENTRENAMIENTO 5.1**Buzón de contacto**

Nivel · Fácil

Un formulario en `/contacto` con nombre y mensaje. Al enviarlo, el mensaje se añade a una lista en memoria y se ve debajo del formulario. Usa `redirect:` tras guardar.

PISTAS

- Una clase `Mensaje` con `nombre` y `texto`, con sus `get` y `set`.
- `<textarea th:field="*{texto}"></textarea>` funciona igual que un `<input>`.

RESULTADO ESPERADO

localhost:8080/contacto

Contacta con el centro

Nombre Mensaje

- Ana: Hola

Formulario Contacta con el centro, con Nombre y Mensaje, y debajo Ana: Hola

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Mensaje.java`:

```
package com.ejemplo.primerproyecto.model;

public class Mensaje {

    private String nombre;
    private String texto;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getTexto() {
        return texto;
    }

    public void setTexto(String texto) {
        this.texto = texto;
    }

}
```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/ContactoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Mensaje;

@Controller
public class ContactoController {

    private final List<Mensaje> mensajes = new ArrayList<>();

    @GetMapping("/contacto")
    public String formulario(Model model) {
        model.addAttribute("mensaje", new Mensaje());
        model.addAttribute("mensajes", mensajes);
        return "contacto";
    }

    @PostMapping("/contacto/enviar")
    public String enviar(@ModelAttribute("mensaje") Mensaje mensaje) {
        mensajes.add(mensaje);
        return "redirect:/contacto";
    }

}
```

Fichero `src/main/resources/templates/contacto.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
    <h1>Contacta con el centro</h1>
    <form th:action="@{/contacto/enviar}" th:object="${mensaje}" method="post">
        <label>Nombre <input type="text" th:field="*{nombre}" /></label>
        <label>Mensaje <textarea th:field="*{texto}"></textarea></label>
        <button type="submit">Enviar</button>
    </form>
    <ul>
        <li th:each="m : ${mensajes}" th:text="|${m.nombre}: ${m.texto}|"></li>
    </ul>
</body>
</html>
```

ENTRENAMIENTO 5.2

Lista de la compra

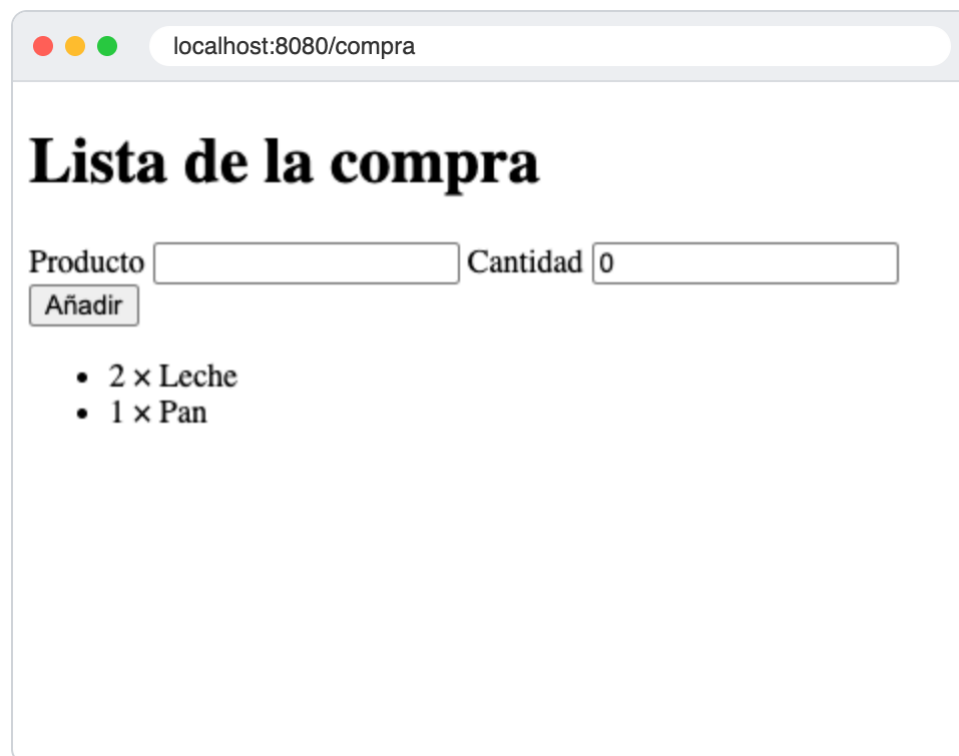
Nivel · Intermedio

Un formulario con producto y cantidad, que añade productos a una lista en memoria. Muestra un aviso si la lista está vacía. Comprueba que enviar el formulario dos veces con los mismos datos añada dos líneas, no las suma.

PISTAS

- `#lists.isEmpty(productos)` para el aviso.
- No hace falta impedir productos repetidos: cada envío es una línea nueva, aunque el nombre coincida.

RESULTADO ESPERADO



Formulario Lista de la compra con Producto y Cantidad, y debajo 2 x Leche, 1 x Pan

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Producto.java`:

```
package com.ejemplo.primerproyecto.model;

public class Producto {

    private String nombre;
    private int cantidad;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

```

    public int getCantidad() {
        return cantidad;
    }

    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }
}

```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/CompraController.java`:

```

package com.ejemplo.primerproyecto.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Producto;

@Controller
public class CompraController {

    private final List<Producto> productos = new ArrayList<>();

    @GetMapping("/compra")
    public String lista(Model model) {
        model.addAttribute("producto", new Producto());
        model.addAttribute("productos", productos);
        return "compra";
    }

    @PostMapping("/compra/anadir")
    public String anadir(@ModelAttribute("producto") Producto producto) {
        productos.add(producto);
        return "redirect:/compra";
    }
}

```

Fichero `src/main/resources/templates/compra.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
    <h1>Lista de la compra</h1>
    <form th:action="@{/compra/anadir}" th:object="${producto}" method="post">
        <label>Producto <input type="text" th:field="*{nombre}" /></label>
        <label>Cantidad <input type="number" th:field="*{cantidad}" /></label>
        <button type="submit">Añadir</button>
    </form>
    <p th:if="${#lists.isEmpty(productos)}">La lista está vacía.</p>
    <ul th:unless="${#lists.isEmpty(productos)}">

```

```
<li th:each="p : ${productos}" th:text="${p.cantidad} × ${p.nombre}|">--</li>
</ul>
</body>
</html>
```

ENTRENAMIENTO 5.3

Lista de espera

Nivel · Difícil

El taller «Fotografía con el móvil» está completo. En `/talleres/{id}/espera` muestra un formulario para apuntarse (nombre y email, con el identificador del taller en un campo oculto). Al enviarlo, redirige a una página que dice en qué puesto de la lista de espera ha quedado esa persona.

PISTAS

- El campo oculto se rellena en el controlador antes de mostrar el formulario, no lo escribe el usuario.
- El puesto es cuántas inscripciones de ese taller había ya en la lista, más una.
- Un `redirect:` puede llevar datos en la propia URL:
`"redirect:/talleres/" + id + "/espera/hecho"`.

RESULTADO ESPERADO



Formulario Lista de espera: Fotografía con el móvil, con Nombre, Email y el botón Apuntarme



Página Apuntado en la lista de espera: Eres el número 2 en la lista de espera de Fotografía con el móvil

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Inscripcion.java`:

```
package com.ejemplo.primerproyecto.model;
```

```

public class Inscripcion {

    private int tallerId;
    private String nombre;
    private String email;

    public int getTallerId() {
        return tallerId;
    }

    public void setTallerId(int tallerId) {
        this.tallerId = tallerId;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

}

```

Fichero src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java:

```

package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Inscripcion;
import com.ejemplo.primerproyecto.model.Taller;

@Controller
public class TallerController {

    private final List<Taller> talleres = new ArrayList<> (List.of(
        new Taller(1, "Fotografía con el móvil", "básico",
        LocalDate.of(2026, 10, 21), 20, 20, 0)));
    private final List<Inscripcion> listaEspera = new ArrayList<>();

    @GetMapping("/talleres/{id}/espera")
    public String formulario(@PathVariable int id, Model model) {

```

```

        Incripcion inscripcion = new Incripcion();
        inscripcion.setTallerId(id);
        model.addAttribute("inscripcion", inscripcion);
        model.addAttribute("taller", buscar(id));
        return "talleres/espera";
    }

    @PostMapping("/talleres/espera/apuntarse")
    public String apuntarse(@ModelAttribute("inscripcion") Incripcion
        inscripcion) {
        listaEspera.add(inscripcion);
        return "redirect:/talleres/" + inscripcion.getTallerId() + "/espera/
        hecho";
    }

    @GetMapping("/talleres/{id}/espera/hecho")
    public String hecho(@PathVariable int id, Model model) {
        model.addAttribute("taller", buscar(id));
        int puesto = 0;
        for (Incripcion i : listaEspera) {
            if (i.getTallerId() == id) {
                puesto++;
            }
        }
        model.addAttribute("puesto", puesto);
        return "talleres/espera-hecho";
    }

    private Taller buscar(int id) {
        for (Taller t : talleres) {
            if (t.getId() == id) {
                return t;
            }
        }
        return null;
    }
}

```

Fichero src/main/resources/templates/talleres/espera.html:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
    <h1 th:text="|Lista de espera: ${taller.nombre}|"></h1>
    <p>Este taller está completo. Apúntate y te avisaremos si queda una plaza libre.</p>
    <form th:action="@{/talleres/espera/apuntarse}" th:object="${inscripcion}"
        method="post">
        <input type="hidden" th:field="*{tallerId}" />
        <label>Nombre <input type="text" th:field="*{nombre}" /></label>
        <label>Email <input type="text" th:field="*{email}" /></label>
        <button type="submit">Apuntarme</button>
    </form>
</body>
</html>

```

Fichero src/main/resources/templates/talleres/espera-hecho.html:

```

<!DOCTYPE html>

```

```
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <h1>Apuntado en la lista de espera</h1>
  <p th:text="|Eres el número ${puesto} en la lista de espera de $
    {taller.nombre}.|"></p>
</body>
</html>
```

PÍLDORA

Un formulario en 4 piezas

1. Una clase con los campos del formulario, sus `get` y `set`, y un constructor sin parámetros.
2. `<form th:object="${objeto}" method="post">` y `th:field="*{propiedad}"` en cada campo.
3. Un método con `@PostMapping` que recibe el objeto ya relleno con `@ModelAttribute`.
4. `return "redirect:/...";` tras guardar, nunca la vista directamente.

AUTOEVALUACIÓN**Formularios**

5.1 ¿Por qué no es correcto usar un enlace (GET) para incrementar un contador en el servidor?

5.2 ¿Qué significa que un GET sea idempotente?

- a) Que tarda siempre lo mismo.
- b) Que repetirlo no cambia el estado del servidor.
- c) Que solo se puede hacer una vez.
- d) Que necesita autenticación.

5.3 ¿Qué hace `th:object="${tallerFormulario}"` en un `<form>`?

5.4 ¿Qué dos cosas hace `th:field="*{nivel}"` en un `<select>`?

5.5 ¿Qué necesita una clase para poder usarse con `th:object/th:field`?

- a) Extender una clase de Spring.
- b) Un constructor sin parámetros y `get/set` para cada campo.
- c) Implementar `Serializable`.
- d) Estar en el paquete `model`.

5.6 ¿Qué diferencia hay entre `List.of(...)` y `new ArrayList<>(List.of(...))` para la lista de talleres?

5.7 ¿Por qué al editar un taller el controlador crea un `Taller` nuevo en vez de cambiar el que ya existe?

5.8 ¿Qué hace `return "redirect:/talleres";` en un método `@PostMapping`?

- a) Renderiza la plantilla `redirect`.
- b) Responde con un 302 y la cabecera `Location`; el navegador hace un GET a esa ruta.
- c) Reenvía la misma petición POST a otra ruta.
- d) Nada distinto de `return "talleres";`.

5.9 En el patrón PRG, ¿qué letra corresponde a cada paso: enviar el formulario, redirigir, mostrar el resultado?

5.10 Sin `redirect:`, el controlador guarda el taller y devuelve directamente `"talleres/lista"`. Si el usuario recarga la página con F5, ¿qué ocurre?

- a) Nada: F5 solo recarga la vista.
- b) El navegador repite el POST y el taller se da de alta otra vez.
- c) Da un error 405.
- d) Se pierde el taller que se acababa de guardar.

5.11 Con `redirect:`, ¿cuál es la última petición que recuerda el navegador tras guardar un taller?

5.12 ¿Los navegadores avisan siempre antes de repetir cualquier petición que escribe datos?

5.13 En el formulario de alta y edición de talleres, ¿qué distingue una alta de una edición?

- a) La ruta a la que se envía el formulario.

- b)** El valor del campo oculto `id` : 0 para alta, el `id` real para edición.
- c)** El método HTTP.
- d)** El nombre del taller.

5.14 ¿Por qué el campo `id` del formulario es un `<input type="hidden">` y no uno visible?

5.15 Escribe la firma de un método que reciba, por `POST`, un formulario ligado a la clase `Producto`.

Respuestas al final de la unidad, en «Autoevaluación · Formularios».

6. Validación de formularios

Un formulario sin validar acepta cualquier cosa: nombres en blanco, correos sin arroba, plazas negativas. Bean Validation deja esas reglas escritas sobre la propia clase, y Spring las comprueba antes de guardar nada.

El formulario admite cualquier cosa

El formulario de talleres de la sección anterior no comprueba nada: cualquier texto en «Nombre», cualquier valor en «Plazas», incluso vacío o negativo, se guarda sin protestar.

EJEMPLO

El problema

Con el mismo formulario y controlador de antes, se envía `nombre vacío`, `nivel vacío` y `plazas=-5`.

Fichero `src/main/resources/templates/talleres/lista.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <table>
    <tr th:each="taller : ${talleres}">
      <td th:text="'${taller.nombre}'">--</td>
      <td th:text="${taller.nivel}">--</td>
      <td th:text="${taller.plazas}">--</td>
    </tr>
  </table>
</body>
</html>
```

Comprueba que funciona (mal). El taller se guarda tal cual, sin ningún aviso:

```
POST /talleres/guardar (id=0&nombre=&nivel=&plazas=-5) → 302
Tabla resultante: '' (nivel vacío) -5
```

AVISO

Si intentas usar `@NotBlank` o cualquier otra anotación de validación sin la dependencia del apartado siguiente, el proyecto no llega a arrancar: **no compila**. Faltan las clases `jakarta.validation.constraints.*`, así que ni siquiera puedes escribir el código hasta añadirla.

La dependencia de validación

Las anotaciones de Bean Validation (`@NotBlank`, `@Size...`) no vienen con `spring-boot-starter-webmvc`: hace falta añadir su starter propio.

DEFINICIONES

Bean Validation: el estándar de Java (Jakarta Bean Validation) para declarar restricciones sobre los campos de una clase con anotaciones, en vez de comprobarlos a mano con `if`.

`spring-boot-starter-validation`: el starter que trae Bean Validation (Hibernate Validator) y lo conecta con Spring MVC.

EJEMPLO**Añadir la dependencia**

En el `pom.xml`, junto a las demás dependencias:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Tras guardar, actualiza el proyecto Maven (clic derecho sobre el proyecto › **Maven** › **Update Project**, o `Alt+F5`) para que STS descargue el `.jar` y deje de marcar las anotaciones como error.

Anotar el formulario

Con la dependencia añadida, cada campo del formulario declara su propia regla, directamente sobre la clase.

REFERENCIA**Las anotaciones de esta unidad**

Anotación	Sobre	Comprueba
@NotBlank	String	no es null, ni vacío, ni solo espacios
@Size(min, max)	String	la longitud del texto está entre los dos límites
@Email	String	tiene forma de dirección de correo
@Min(valor)	int, double...	el número no es menor que valor

EJEMPLO**TallerFormulario con restricciones**

Se añade un campo nuevo, `contacto` (el correo de quien organiza el taller), para tener un `@Email` que comprobar.

Fichero `src/main/java/com/ejemplo/primerproyecto/model/TallerFormulario.java`:

```
package com.ejemplo.primerproyecto.model;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class TallerFormulario {

    private int id;

    @NotBlank(message = "{taller.nombre.notblank}")
    @Size(min = 3, max = 60, message = "{taller.nombre.size}")
    private String nombre;

    @NotBlank(message = "{taller.nivel.notblank}")
    private String nivel;

    @Min(value = 1, message = "{taller.plazas.min}")
    private int plazas;

    @NotBlank(message = "{taller.contacto.notblank}")
    @Email(message = "{taller.contacto.email}")
    private String contacto;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getNivel() {
        return nivel;
    }

    public void setNivel(String nivel) {
        this.nivel = nivel;
    }

    public int getPlazas() {
        return plazas;
    }

    public void setPlazas(int plazas) {
        this.plazas = plazas;
    }

    public String getContacto() {
        return contacto;
    }
}
```

```

    public void setContacto(String contacto) {
        this.contacto = contacto;
    }
}

```

@Valid y BindingResult

Las anotaciones, por sí solas, no hacen nada: hay que decirle al método que las compruebe, y recoger el resultado.

DEFINICIONES

@Valid: delante de un parámetro, hace que Spring valide ese objeto con Bean Validation antes de ejecutar el método.

BindingResult: recoge los errores de esa validación sin lanzar una excepción, para que el método decida qué hacer.

EJEMPLO

El controlador completo

Si hay errores, guardar vuelve a mostrar el formulario (sin `redirect:`, para que Thymeleaf pueda pintar los mensajes); si no los hay, guarda y redirige.

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```

package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Taller;
import com.ejemplo.primerproyecto.model.TallerFormulario;

import jakarta.validation.Valid;

@Controller
public class TallerController {

    private final List<Taller> talleres = new ArrayList<>(
        List.of(new Taller(1, "Robótica con Arduino", "medio",
            LocalDate.of(2026, 10, 14), 16, 12, 25)));
}

```

```

@GetMapping("/talleres")
public String lista(Model model) {
    model.addAttribute("talleres", talleres);
    return "talleres/lista";
}

@GetMapping("/talleres/nuevo")
public String nuevo(Model model) {
    model.addAttribute("tallerFormulario", new TallerFormulario());
    return "talleres/formulario";
}

// @Valid activa Bean Validation sobre el formulario; BindingResult recoge los
// errores sin lanzar
// excepción, y debe ir siempre inmediatamente después del parámetro que valida.
@PostMapping("/talleres/guardar")
@Valid @ModelAttribute("tallerFormulario")
TallerFormulario formulario,
    BindingResult resultado) {
    if (resultado.hasErrors()) {
        return "talleres/formulario";
    }
    int nuevoId = talleres.stream().mapToInt(Taller::getId).max().orElse(0)
+ 1;
    talleres.add(new Taller(nuevoId, formulario.getNombre(),
formulario.getNivel(), LocalDate.now(),
    formulario.getPlazas(), 0, 0));
    return "redirect:/talleres";
}
}

```

AVISO

`BindingResult` tiene que ir **inmediatamente después** del parámetro que valida. Si se cuela otro parámetro en medio (por ejemplo, `Model model` entre `@Valid TallerFormulario formulario` y `BindingResult resultado`), Spring deja de asociarlos: ya no rebota al formulario, sino que responde con un error 500 «Validation failed for argument... MethodArgumentNotValidException». No es un fallo al arrancar, sino al primer envío con datos inválidos, así que conviene comprobarlo con un envío de prueba, no solo con que la aplicación arranque.

Mostrar los errores: `th:errors` y `th:errorclass`

Cuando `guardar` rebota al formulario, el objeto sigue en el modelo, con lo que el usuario escribió y con los errores ya calculados. La plantilla los pinta con dos atributos de Thymeleaf.

DEFINICIONES

`#fields.hasErrors('campo')`: true si ese campo tiene algún error de validación.

`th:errors="*{campo}"`: escribe el mensaje de error de ese campo (combínalo con

`th:if="#{#fields.hasErrors('campo')}"` para que solo aparezca cuando hace falta).

`th:errorclass="clase"`: añade esa clase CSS al elemento si su campo tiene errores; útil para marcar el borde en rojo.

EJEMPLO**El formulario con mensajes de error**

Fichero `src/main/resources/templates/talleres/formulario.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Nuevo taller</title>
<link rel="stylesheet" th:href="@{/css/talleres.css}" />
</head>
<body>
<h1>Nuevo taller</h1>
<form th:action="@{/talleres/guardar}" th:object="${tallerFormulario}" method="post">
  <input type="hidden" th:field="*{id}" />
  <label>Nombre
    <input type="text" th:field="*{nombre}" th:errorclass="campo-error" required
      minlength="3" maxlength="60" />
    <span th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}" class="error"></span>
  </label>
  <label>Nivel
    <select th:field="*{nivel}" th:errorclass="campo-error" required>
      <option value="">Elige un nivel</option>
      <option value="básico">Básico</option>
      <option value="medio">Medio</option>
      <option value="avanzado">Avanzado</option>
    </select>
    <span th:if="${#fields.hasErrors('nivel')}" th:errors="*{nivel}" class="error"></span>
  </label>
  <label>Plazas
    <input type="number" th:field="*{plazas}" th:errorclass="campo-error" required
      min="1" />
    <span th:if="${#fields.hasErrors('plazas')}" th:errors="*{plazas}" class="error"></span>
  </label>
  <label>Correo de contacto
    <input type="email" th:field="*{contacto}" th:errorclass="campo-error" required />
    <span th:if="${#fields.hasErrors('contacto')}" th:errors="*{contacto}"
      class="error"></span>
  </label>
  <button type="submit">Guardar</button>
</form>
</body>
</html>
```

COMPRUEBA QUE FUNCIONA

Se envía nombre=a, sin nivel, plazas=-2 y contacto=hola:

localhost:8080/talleres/nuevo

Nuevo taller

Nombre

a

El nombre debe tener entre 3 y 60 caracteres

Nivel

Elige un nivel

Elige un nivel

Plazas

-2

Debe haber al menos 1 plaza

Correo de contacto

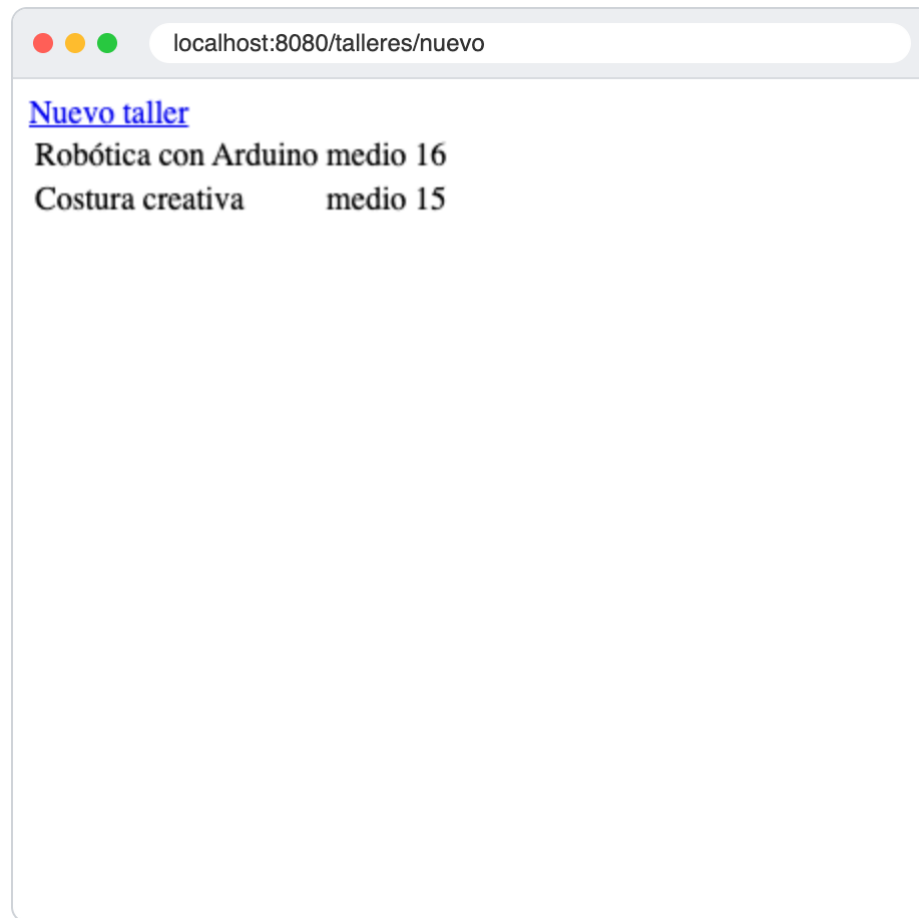
hola

El formato del correo no es válido

Guardar

Formulario Nuevo taller con los cuatro campos en rojo: El nombre debe tener entre 3 y 60 caracteres, Elige un nivel, Debe haber al menos 1 plaza, El formato del correo no es válido

Los valores escritos (a, -2, ho1a) siguen en el formulario: nada se pierde al rebotar. Corrigiendo los cuatro campos y reenviando, el taller se guarda y aparece en la lista:



Lista con Robótica con Arduino y el taller nuevo, Costura creativa, nivel medio, 15 plazas

Mensajes en español: ValidationMessages.properties

Los mensajes de los ejemplos anteriores ya estaban en español porque cada anotación llevaba su propio `message`. Escribirlos así, con el texto entre comillas dentro de la anotación, funciona pero mezcla el texto que ve el usuario con el código Java. Hay una forma de separarlos.

DEFINICIÓN

`ValidationMessages.properties`: fichero en `src/main/resources/` (nombre exacto, fijo) donde Bean Validation busca los mensajes cuyo `message` es una clave entre llaves, `{clave}`.

EJEMPLO

Mensajes externalizados

Cada anotación pasa una clave, y esa clave se traduce en el fichero de mensajes.

```
@NotBlank(message = "{taller.nombre.notblank}")
```

Fichero `src/main/resources/ValidationMessages.properties`:

```
taller.nombre.notblank=El nombre del taller no puede estar vacío  
taller.nombre.size=El nombre debe tener entre 3 y 60 caracteres  
taller.nivel.notblank=Elige un nivel  
taller.plazas.min=Debe haber al menos 1 plaza  
taller.contacto.notblank=El correo de contacto no puede estar vacío  
taller.contacto.email=El formato del correo no es válido
```

Los acentos se escriben con su código Unicode (í para í, é para é...): es la convención de los ficheros `.properties` de Java.

AVISO

Además del fichero de mensajes, `application.properties` necesita `spring.messages.encoding=UTF-8` para que los acentos se muestren bien en la página. Sin esa línea, los mensajes se ven con caracteres corruptos.

Validar en el cliente y en el servidor

El formulario de talleres lleva, además de `th:field`, atributos HTML como `required`, `minlength` o `type="email"`. Son la validación del navegador: HTML5.

DEFINICIONES

Validación en el cliente (HTML5): atributos como `required`, `minlength`, `min` o `type="email"` que el navegador comprueba antes de enviar el formulario. Si algo falla, ni siquiera llega a hacerse la petición.

Validación en el servidor (Bean Validation): las anotaciones de esta sección, que se comprueban siempre, venga la petición de donde venga.

EJEMPLO

El navegador bloquea antes de enviar

Con `nombre="a"` (menos de 3 caracteres, `minlength="3"`) y sin elegir nivel (`required`), al intentar enviar el formulario el navegador muestra su propio aviso y no llega a hacer la petición: no hay ningún POST en la consola del servidor.

Nuevo taller

Nombre

Nivel

Elige un nivel

Plazas



Selecciona un elemento de la lista

Correo de contacto

Guardar

Formulario Nuevo taller con el desplegable Nivel enmarcado en azul y un aviso del navegador: Selecciona un elemento de la lista

Comprobado con un script: `reportValidity()` devuelve `false` y no se produce ninguna navegación, así que la petición nunca sale del navegador.

AVISO

La validación del navegador no protege nada: cualquiera puede enviar una petición `POST` directamente, sin pasar por el formulario ni por sus atributos `required`. Es justo lo que hacen los envíos de esta sección hechos a mano (sin usar el navegador): llegan igual al servidor con `nombre=a`, `plazas=-2`, etc., y es Bean Validation, en el servidor, quien los rechaza de verdad. Por eso se combinan los dos: el HTML5 da una respuesta inmediata y sin viajar al servidor; Bean Validation es el que de verdad garantiza que los datos guardados son correctos.

RECURSOS

Para saber más

- **Validating Form Input:** el tutorial de Spring sobre `@Valid` y `BindingResult` con Thymeleaf (en inglés).

<https://spring.io/guides/gs/validating-form-input>

- **Jakarta Bean Validation:** la especificación completa, con todas las anotaciones (en inglés).

<https://jakarta.ee/specifications/bean-validation/3.0/>

- **Constraint validation:** cómo funciona la validación HTML5 en el navegador, en MDN.

https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Forms/Form_validation

Práctica

Tres ejercicios para anotar, validar y mostrar errores.

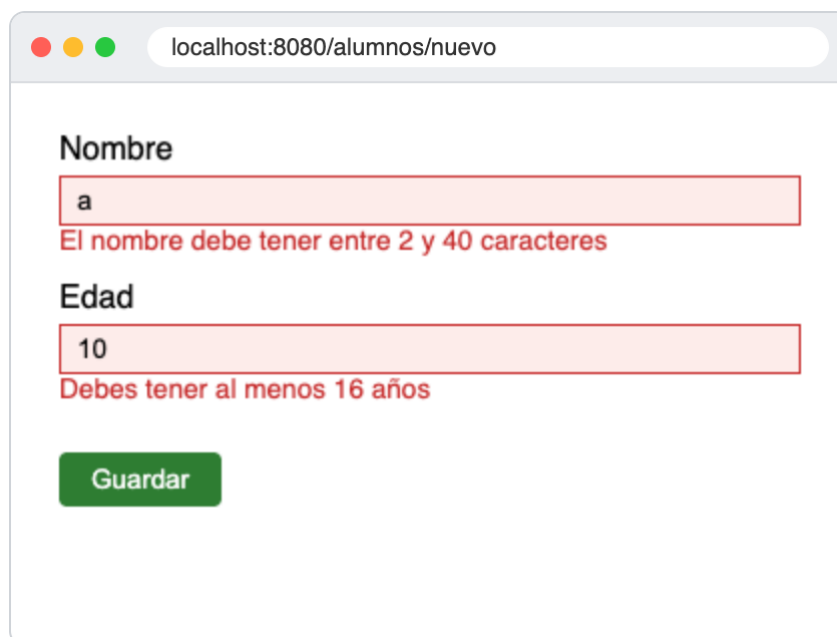
ENTRENAMIENTO 6.1

Alta de alumno

Nivel · Fácil

Una clase `Alumno` con nombre (`@NotBlank`, `@Size(min=2, max=40)`) y edad (`@Min(16)`). Un formulario que muestra los errores y, si todo es correcto, un mensaje de «Alumno guardado correctamente.».

RESULTADO ESPERADO



Formulario con Nombre "a" y Edad 10, con El nombre debe tener entre 2 y 40 caracteres y Debes tener al menos 16 años, en rojo

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Alumno.java`:

```
package com.ejemplo.primerproyecto.model;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class Alumno {

    @NotBlank(message = "El nombre no puede estar vacío")
    @Size(min = 2, max = 40, message = "El nombre debe tener entre 2 y 40 caracteres")
    private String nombre;

    @Min(value = 16, message = "Debes tener al menos 16 años")
    private int edad;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }

}
```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/AlumnoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Alumno;

import jakarta.validation.Valid;

@Controller
public class AlumnoController {

    @GetMapping("/alumnos/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("alumno", new Alumno());
    }

}
```

```

        return "alumno";
    }

    @PostMapping("/alumnos/guardar")
    public String guardar(@Valid @ModelAttribute("alumno") Alumno alumno,
        BindingResult resultado, Model model) {
        if (resultado.hasErrors()) {
            return "alumno";
        }
        model.addAttribute("guardado", true);
        return "alumno";
    }
}

```

Fichero `src/main/resources/templates/alumno.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head><meta charset="UTF-8" /><title>Alta de alumno</title><link rel="stylesheet"
th:href="@{/css/estilos.css}" /></head>
<body>
<p th:if="${guardado}">Alumno guardado correctamente.</p>
<form th:action="@{/alumnos/guardar}" th:object="${alumno}" method="post">
<label>Nombre
<input type="text" th:field="*{nombre}" th:errorclass="campo-error" />
<span th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}"
class="error"></span>
</label>
<label>Edad
<input type="number" th:field="*{edad}" th:errorclass="campo-error" />
<span th:if="${#fields.hasErrors('edad')}" th:errors="*{edad}" class="error"></
span>
</label>
<button type="submit">Guardar</button>
</form>
</body>
</html>

```

ENTRENAMIENTO 6.2

Mensajes propios

Nivel · Intermedio

Un formulario de contacto (nombre, email) con sus mensajes de error en `ValidationMessages.properties`, con un texto distinto al de los ejemplos anteriores.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Contacto.java`:

```

package com.ejemplo.primerproyecto.model;

import jakarta.validation.constraints.Email;

```

```
import jakarta.validation.constraints.NotBlank;

public class Contacto {

    @NotBlank(message = "{contacto.nombre.notblank}")
    private String nombre;

    @NotBlank(message = "{contacto.email.notblank}")
    @Email(message = "{contacto.email.email}")
    private String email;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

}
```

Fichero `src/main/resources/ValidationMessages.properties`:

```
contacto.nombre.notblank=Escribe tu nombre, por favor
contacto.email.notblank=El correo es obligatorio
contacto.email.email=Ese correo no tiene un formato válido
```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/ContactoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Contacto;

import jakarta.validation.Valid;

@Controller
public class ContactoController {

    @GetMapping("/contacto")
    public String formulario(Model model) {
        model.addAttribute("contacto", new Contacto());
        return "contacto";
    }

}
```

```

    @PostMapping("/contacto/enviar")
    public String enviar(@Valid @ModelAttribute("contacto") Contacto contacto,
        BindingResult resultado) {
        if (resultado.hasErrors()) {
            return "contacto";
        }
        return "redirect:/contacto";
    }
}

```

Fichero `src/main/resources/templates/contacto.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
    <form th:action="@{/contacto/enviar}" th:object="${contacto}" method="post">
        <label>Nombre <input type="text" th:field="*{nombre}" th:errorclass="campo-
            error" />
        <span th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}"
            class="error"></span>
        </label>
        <label>Email <input type="text" th:field="*{email}" th:errorclass="campo-error" />
        <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}"
            class="error"></span>
        </label>
        <button type="submit">Enviar</button>
    </form>
</body>
</html>

```

ENTRENAMIENTO 6.3

El buzón de contacto, validado

Nivel · Difícil

Retoma el buzón de contacto de la sección anterior (nombre y mensaje, con `redirect`: tras guardar) y añádele validación completa: nombre (`@NotBlank`, `@Size(min=2, max=40)`) y texto (`@NotBlank`, `@Size(min=5, max=300)`), con sus atributos HTML5 a juego (`required`, `minlength`, `maxlength`) y sus mensajes de error en la plantilla.

PISTAS

- Con errores, el método `enviar` tiene que volver a añadir mensajes al modelo antes de devolver la vista: al no redirigir, la plantilla necesita la lista igual que en la primera visita.
- El `redirect`: solo se hace en la rama sin errores.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Mensaje.java`:

```
package com.ejemplo.primerproyecto.model;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class Mensaje {

    @NotBlank(message = "Escribe tu nombre")
    @Size(min = 2, max = 40, message = "El nombre debe tener entre 2 y 40 caracteres")
    private String nombre;

    @NotBlank(message = "El mensaje no puede estar vacío")
    @Size(min = 5, max = 300, message = "El mensaje debe tener entre 5 y 300 caracteres")
    private String texto;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getTexto() {
        return texto;
    }

    public void setTexto(String texto) {
        this.texto = texto;
    }

}
```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/ContactoController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import com.ejemplo.primerproyecto.model.Mensaje;

import jakarta.validation.Valid;

@Controller
public class ContactoController {
```

```

    private final List<Mensaje> mensajes = new ArrayList<>();

    @GetMapping("/contacto")
    public String formulario(Model model) {
        model.addAttribute("mensaje", new Mensaje());
        model.addAttribute("mensajes", mensajes);
        return "contacto";
    }

    @PostMapping("/contacto/enviar")
    public String enviar(@Valid @ModelAttribute("mensaje") Mensaje mensaje,
        BindingResult resultado, Model model) {
        if (resultado.hasErrors()) {
            model.addAttribute("mensajes", mensajes);
            return "contacto";
        }
        mensajes.add(mensaje);
        return "redirect:/contacto";
    }
}

```

Fichero src/main/resources/templates/contacto.html:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Contacta con el centro</title>
<link rel="stylesheet" th:href="@{/css/estilos.css}" />
</head>
<body>
<h1>Contacta con el centro</h1>
<form th:action="@{/contacto/enviar}" th:object="${mensaje}" method="post">
    <label>Nombre
        <input type="text" th:field="*{nombre}" th:errorclass="campo-error" required
            minlength="2" maxlength="40" />
        <span th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}"
            class="error"></span>
    </label>
    <label>Mensaje
        <textarea th:field="*{texto}" th:errorclass="campo-error" required
            minlength="5" maxlength="300"></textarea>
        <span th:if="${#fields.hasErrors('texto')}" th:errors="*{texto}"
            class="error"></span>
    </label>
    <button type="submit">Enviar</button>
</form>
<ul>
<li th:each="m : ${mensajes}" th:text="|${m.nombre}: ${m.texto}|"></li>
</ul>
</body>
</html>

```

PÍLDORA

Validar un formulario en 4 pasos

1. Añade `spring-boot-starter-validation` al `pom.xml`.
2. Anota los campos de la clase: `@NotBlank`, `@Size`, `@Email`, `@Min...`
3. En el método `@PostMapping`: `@Valid` delante del objeto, `BindingResult` justo después.
4. En la plantilla: `th:errorclass` en cada campo y `th:errors` (con `th:if="${#fields.hasErrors(...)}"`) para el mensaje.

AUTOEVALUACIÓN**Validación de formularios**

6.1 Sin ninguna anotación de validación, ¿qué pasa si se envía un formulario con todos los campos vacíos?

6.2 ¿Qué ocurre al usar `@NotBlank` en un proyecto sin `spring-boot-starter-validation`?

- a) Se ignora en tiempo de ejecución, sin avisar.
- b) El proyecto no compila.
- c) Da un error 500 al arrancar.
- d) Funciona igual, porque `@NotBlank` no necesita la dependencia.

6.3 ¿Qué comprueba `@Size(min = 3, max = 60)` sobre un campo `String`?

6.4 ¿Qué anotación usarías para exigir que un campo `int` no sea menor que 1?

- a) `@NotBlank`
- b) `@Size(min = 1)`
- c) `@Min(1)`
- d) `@Positive`

6.5 ¿Qué dos cosas hace `@Valid` delante de un parámetro?

6.6 ¿Para qué sirve `BindingResult`?

- a) Para guardar el objeto en la base de datos.
- b) Para recoger los errores de validación sin que se lance una excepción.
- c) Para traducir los mensajes de error.
- d) Para redirigir tras guardar.

6.7 Escribe la firma correcta de un método que valide un `TallerFormulario` y reciba también un `Model`.

6.8 Si `BindingResult` no va justo después del parámetro `@Valid`, ¿qué ocurre?

- a) El proyecto no arranca.
- b) Nada, Spring lo asocia igual.
- c) Al enviar datos inválidos, la aplicación responde con un error 500 en vez de rebotar al formulario.
- d) Los errores se muestran, pero sin sus mensajes.

6.9 ¿Qué hace `th:if="${#fields.hasErrors('nombre')}"` junto a `th:errors="*{nombre}"`?

6.10 ¿Qué añade `th:errorclass="campo-error"` a un `<input>` sin errores? ¿Y a uno con errores?

6.11 El formulario rebota con errores. ¿Por qué el valor que había escrito el usuario sigue en el `<input>`?

6.12 ¿Dónde va el fichero `ValidationMessages.properties` y qué formato lleva `message` para usarlo?

6.13 nombre="a" con minlength="3" en el <input>. Al pulsar «Guardar» en un navegador real, ¿llega la petición al servidor?

- a) Sí, y el servidor la rechaza.
- b) No: el navegador bloquea el envío antes de hacer la petición.
- c) Sí, pero sin el campo nombre.
- d) Depende del servidor.

6.14 Alguien envía una petición POST directamente (sin usar el formulario del navegador), con nombre vacío. El <input> tenía required. ¿Le afecta eso?

6.15 ¿Por qué conviene tener validación en el cliente y en el servidor a la vez, y no solo una de las dos?

Respuestas al final de la unidad, en «Autoevaluación · Validación de formularios».

7. Aplicación completa

Es hora de juntarlo todo: una aplicación con varias páginas, una lista filtrable, fichas con contenido dinámico y un formulario validado, que comparten cabecera, pie y estilos.

El proyecto

Vas a construir la web de talleres del centro completa. Estos son sus requisitos.

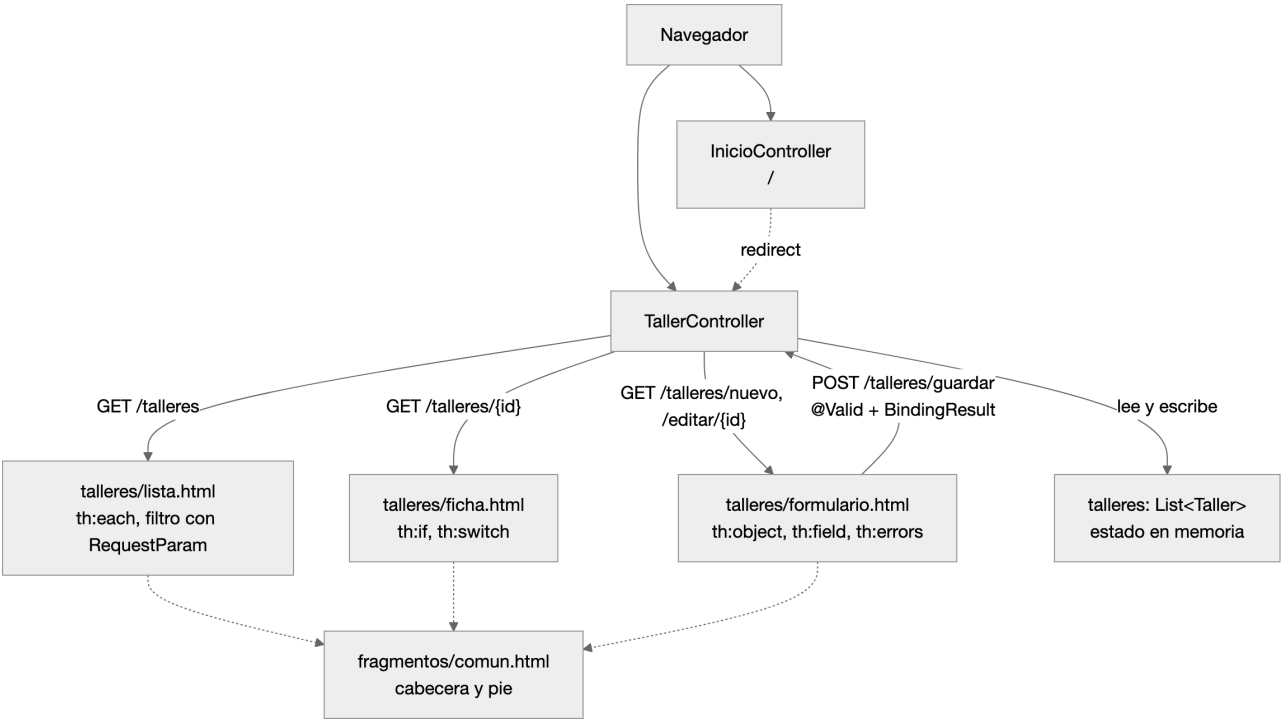
- Mostrar los talleres en una tabla, con su nivel y sus plazas libres, y poder filtrarlos por nivel.
- Una ficha por taller, con su fecha, su precio y un aviso si está completo.
- Dar de alta un taller nuevo y editar uno existente, con el mismo formulario.
- Comprobar los datos del formulario antes de guardarlos, con mensajes de error claros.
- Una cabecera y un pie iguales en todas las páginas.

El resultado, con los datos de partida:



Talleres del centro, con la cabecera verde, el filtro por nivel (Todos, Básico, Medio, Avanzado) y la tabla con Robótica con Arduino, Fotografía con el móvil (Completo) y Ajedrez de competición

El reparto en piezas usa lo que ya sabes: un controlador con todas las rutas de /talleres , una clase Taller (los datos) y otra TallerFormulario (lo que se edita), tres plantillas y los fragmentos comunes:



El navegador pide las rutas de TallerController; GET /talleres usa lista.html con th:each y el filtro por RequestParam; GET /talleres/{id} usa ficha.html con th:if y th:switch; GET /talleres/nuevo y /editar/{id} usan formulario.html con th:object, th:field y th:errors; el formulario envía POST /talleres/guardar con @Valid y BindingResult; las tres plantillas comparten fragmentos/comun.html; el controlador lee y escribe la lista de talleres en memoria

REFERENCIA

Los ficheros del proyecto

Fichero	De qué sección
model/Taller.java	3 (un modelo propio) y 4 (plazasLibres, completo)
model/TallerFormulario.java	5 (formularios) y 6 (@NotBlank, @Size, @Email, @Min)
controller/InicioController.java	2 (controladores) y 5 (redirect:)
controller/TallerController.java	2 a 6, todo junto
templates/fragmentos/comun.html	3 (th:fragment, th:replace, fragmento con parámetro)
templates/talleres/lista.html	4 (th:each, filtro con @RequestParam)
templates/talleres/ficha.html	3 (th:if, th:switch, expresiones)
templates/talleres/formulario.html	5 (th:object, th:field) y 6 (th:errors, th:errorclass)
ValidationMessages.properties	6 (mensajes en español)
static/css/estilos.css	2 (recursos estáticos)

El controlador, de principio a fin

Un único `TallerController` atiende las siete rutas de la aplicación. La lista de talleres es un campo de la clase: estado en memoria, compartido por todas las peticiones.

EJEMPLO

TallerController **completo**

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
package com.ejemplo.primerproyecto.controller;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.ejemplo.primerproyecto.model.Taller;
import com.ejemplo.primerproyecto.model.TallerFormulario;

import jakarta.validation.Valid;

@Controller
public class TallerController {

    // estado en memoria: la misma lista para todas las peticiones, mientras la
    // aplicación esté arrancada
    private final List<Taller> talleres = new ArrayList<> (List.of(
        new Taller(1, "Robótica con Arduino", "medio",
            LocalDate.of(2026, 10, 14), 16, 12, 25),
        new Taller(2, "Fotografía con el móvil", "básico",
            LocalDate.of(2026, 10, 21), 20, 20, 0),
        new Taller(3, "Ajedrez de competición", "avanzado",
            LocalDate.of(2026, 11, 4), 12, 3, 10.5)));

    @GetMapping("/talleres")
    public String lista(Model model, @RequestParam(required = false, defaultValue =
"todos") String nivel) {
        List<Taller> filtrados = new ArrayList<>();
        for (Taller t : talleres) {
            if (nivel.equals("todos") || t.getNivel().equals(nivel)) {
                filtrados.add(t);
            }
        }
        model.addAttribute("talleres", filtrados);
        model.addAttribute("nivel", nivel);
        return "talleres/lista";
    }

    @GetMapping("/talleres/{id}")
    public String ficha(@PathVariable int id, Model model) {
```

```

        model.addAttribute("taller", buscar(id));
        return "talleres/ficha";
    }

    @GetMapping("/talleres/nuevo")
    public String nuevo(Model model) {
        model.addAttribute("tallerFormulario", new TallerFormulario());
        return "talleres/formulario";
    }

    @GetMapping("/talleres/editar/{id}")
    public String editar(@PathVariable int id, Model model) {
        Taller taller = buscar(id);
        TallerFormulario formulario = new TallerFormulario();
        formulario.setId(taller.getId());
        formulario.setNombre(taller.getNombre());
        formulario.setNivel(taller.getNivel());
        formulario.setPlazas(taller.getPlazas());
        model.addAttribute("tallerFormulario", formulario);
        return "talleres/formulario";
    }

    @PostMapping("/talleres/guardar")
    public String guardar(@Valid @ModelAttribute("tallerFormulario")
        TallerFormulario formulario,
        BindingResult resultado) {
        if (resultado.hasErrors()) {
            return "talleres/formulario";
        }
        if (formulario.getId() == 0) {
            int nuevoId =
talleres.stream().mapToInt(Taller::getId).max().orElse(0) + 1;
            talleres.add(new Taller(nuevoId, formulario.getNombre(),
formulario.getNivel(), LocalDate.now(),
                                formulario.getPlazas(), 0, 0));
        } else {
            for (int i = 0; i < talleres.size(); i++) {
                Taller anterior = talleres.get(i);
                if (anterior.getId() == formulario.getId()) {
                    talleres.set(i, new Taller(anterior.getId(),
formulario.getNombre(), formulario.getNivel(),
anterior.getFecha(),
formulario.getPlazas(), anterior.getInscritos(),
anterior.getPrecio()));
                }
            }
        }
        return "redirect:/talleres";
    }

    private Taller buscar(int id) {
        for (Taller t : talleres) {
            if (t.getId() == id) {
                return t;
            }
        }
        return null;
    }
}

```

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/InicioController.java`:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class InicioController {

    @GetMapping("/")
    public String inicio() {
        return "redirect:/talleres";
    }

}
```

/ no tiene su propia plantilla: redirige a `/talleres`, así que solo hay una página de portada que mantener.

EJEMPLO

Los dos modelos

Taller son los datos guardados; TallerFormulario, solo lo que el formulario necesita, con sus anotaciones de validación.

Fichero `src/main/java/com/ejemplo/primerproyecto/model/Taller.java`:

```
package com.ejemplo.primerproyecto.model;

import java.time.LocalDate;

public class Taller {

    private int id;
    private String nombre;
    private String nivel;
    private LocalDate fecha;
    private int plazas;
    private int inscritos;
    private double precio;

    public Taller(int id, String nombre, String nivel, LocalDate fecha, int plazas,
int inscritos, double precio) {
        this.id = id;
        this.nombre = nombre;
        this.nivel = nivel;
        this.fecha = fecha;
        this.plazas = plazas;
        this.inscritos = inscritos;
        this.precio = precio;
    }

    public int getId() {
        return id;
    }

}
```

```
public String getNombre() {  
    return nombre;  
}  
  
public String getNivel() {  
    return nivel;  
}  
  
public LocalDate getFecha() {  
    return fecha;  
}  
  
public int getPlazas() {  
    return plazas;  
}  
  
public int getInscritos() {  
    return inscritos;  
}  
  
public double getPrecio() {  
    return precio;  
}  
  
public int getPlazasLibres() {  
    return plazas - inscritos;  
}  
  
public boolean isCompleto() {  
    return inscritos >= plazas;  
}  
  
}
```

Fichero `src/main/java/com/ejemplo/primerproyecto/model/TallerFormulario.java`:

```
package com.ejemplo.primerproyecto.model;  
  
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.Min;  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.Size;  
  
public class TallerFormulario {  
  
    private int id;  
  
    @NotBlank(message = "{taller.nombre.notblank}")  
    @Size(min = 3, max = 60, message = "{taller.nombre.size}")  
    private String nombre;  
  
    @NotBlank(message = "{taller.nivel.notblank}")  
    private String nivel;  
  
    @Min(value = 1, message = "{taller.plazas.min}")  
    private int plazas;  
  
    @NotBlank(message = "{taller.contacto.notblank}")  
    @Email(message = "{taller.contacto.email}")  
    private String contacto;  
  
}
```

```
public int getId() {  
    return id;  
}  
  
public void setId(int id) {  
    this.id = id;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getNivel() {  
    return nivel;  
}  
  
public void setNivel(String nivel) {  
    this.nivel = nivel;  
}  
  
public int getPlazas() {  
    return plazas;  
}  
  
public void setPlazas(int plazas) {  
    this.plazas = plazas;  
}  
  
public String getContacto() {  
    return contacto;  
}  
  
public void setContacto(String contacto) {  
    this.contacto = contacto;  
}  
}
```

Fichero src/main/resources/ValidationMessages.properties:

```
taller.nombre.notblank=El nombre del taller no puede estar vacío  
taller.nombre.size=El nombre debe tener entre 3 y 60 caracteres  
taller.nivel.notblank=Elige un nivel  
taller.plazas.min=Debe haber al menos 1 plaza  
taller.contacto.notblank=El correo de contacto no puede estar vacío  
taller.contacto.email=El formato del correo no es válido
```

Las plantillas

Las tres páginas comparten cabecera y pie, y cada una usa las técnicas de su sección.

EJEMPLO

Los fragmentos comunes

Fichero `src/main/resources/templates/fragmentos/comun.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <header th:fragment="cabecera(activa)">
    <a th:href="@{/talleres}" class="marca">Talleres del centro</a>
    <nav>
      <a th:href="@{/talleres}" th:classappend="${activa == 'talleres'} ?
'activa'">Talleres</a>
      <a th:href="@{/talleres/nuevo}" th:classappend="${activa == 'nuevo'} ?
'activa'">Nuevo taller</a>
    </nav>
  </header>
  <footer th:fragment="pie">
    <p>Curso 2026/2027 · Inscripciones en secretaría</p>
  </footer>
</body>
</html>
```

EJEMPLO

La lista, filtrable

Fichero `src/main/resources/templates/talleres/lista.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Talleres del centro</title>
<link rel="stylesheet" th:href="@{/css/estilos.css}" />
</head>
<body>
  <header th:replace="~{fragmentos/comun :: cabecera('talleres')}">Cabecera</header>
  <main>
    <h1>Talleres del centro</h1>
    <nav class="filtro">
      Nivel:
      <a th:href="@{/talleres(nivel='todos')}" th:classappend="${nivel == 'todos'} ?
'activa'">Todos</a>
      <a th:href="@{/talleres(nivel='básico')}" th:classappend="${nivel == 'básico'} ?
'activa'">Básico</a>
      <a th:href="@{/talleres(nivel='medio')}" th:classappend="${nivel == 'medio'} ?
'activa'">Medio</a>
      <a th:href="@{/talleres(nivel='avanzado')}" th:classappend="${nivel ==
'avanzado'} ? 'activa'">Avanzado</a>
    </nav>
    <p th:if="${#lists.isEmpty(talleres)}">No hay talleres con ese nivel.</p>
    <table th:unless="${#lists.isEmpty(talleres)}">
      <thead>
        <tr>
          <th>Nombre</th>
          <th>Nivel</th>
          <th>Plazas libres</th>
          <th></th>
        </tr>
      </thead>
    </table>
```

```

    </tr>
  </thead>
  <tbody>
    <tr th:each="taller, est : ${talleres}" th:class="${est.even} ? 'par' : 'impar'">
      <td><a th:href="@{/talleres/{id}(id=${taller.id})}" th:text="${
taller.nombre}">Nombre</a></td>
      <td th:text="${taller.nivel}">nivel</td>
      <td th:text="${taller.completo} ? 'Completo' : ${taller.plazasLibres}">0</td>
      <td><a th:href="@{/talleres/editar/{id}(id=${taller.id})}">Editar</a></td>
    </tr>
  </tbody>
</table>
</main>
<footer th:replace="~{fragmentos/comun :: pie}">Pie</footer>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

Filtrando por nivel básico, solo queda «Fotografía con el móvil», marcado como completo:

```

GET /talleres?nivel=básico → 200
Fotografía con el móvil    básico    Completo

```

EJEMPLO

La ficha, con contenido dinámico

Fichero `src/main/resources/templates/talleres/ficha.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${taller != null} ? ${taller.nombre} : 'Taller no encontrado'">Taller</
title>
<link rel="stylesheet" th:href="@{/css/estilos.css}" />
</head>
<body>
<header th:replace="~{fragmentos/comun :: cabecera('talleres')}">Cabecera</header>
<main th:if="${taller != null}">
<h1 th:text="${taller.nombre}">Nombre del taller</h1>
<p th:switch="${taller.nivel}">
  <span th:case="'básico'" class="nivel basico">Nivel básico</span>
  <span th:case="'medio'" class="nivel medio">Nivel medio</span>
  <span th:case="*" class="nivel avanzado">Nivel avanzado</span>
</p>
<ul>
<li>Fecha: <span th:text="${#temporals.format(taller.fecha, 'dd/MM/
yyyy')}">01/01/2026</span></li>
<li>Precio: <span th:text="${taller.precio == 0} ? 'Gratis' : |$
{#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')}">€|">0,00 €</span></li>
<li>th:text="|Inscritos: ${taller.inscritos} de ${taller.plazas}">Inscritos: 0 de
0</li>
</ul>
<p th:if="${taller.completo}" class="aviso">Este taller está completo.</p>
<p th:unless="${taller.completo}" th:text="|Quedan ${taller.plazasLibres} plazas
libres.|">Quedan plazas libres.</p>
<a th:href="@{/talleres/editar/{id}(id=${taller.id})}">Editar este taller</a>

```

```

</main>
<main th:unless="${taller != null}">
  <h1>Taller no encontrado</h1>
  <p>No hay ningún taller con ese número.</p>
</main>
<footer th:replace="~{fragmentos/comun :: pie}">Pie</footer>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

http://localhost:8080/talleres/1:



Robótica con Arduino con Nivel medio en amarillo, Fecha 14/10/2026, Precio 25,00 €, Inscritos 12 de 16 y Quedan 4 plazas libres

/talleres/99, que no existe, no falla: muestra «Taller no encontrado» gracias al `th:if/th:unless` sobre `taller != null`.

EJEMPLO

El formulario, validado

Fichero `src/main/resources/templates/talleres/formulario.html`:

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title th:text="${tallerFormulario.id == 0} ? 'Nuevo taller' : 'Editar
  taller'">Formulario</title>

```

```

<link rel="stylesheet" th:href="@{/css/estilos.css}" />
</head>
<body>
  <header th:replace="~{fragmentos/comun :: cabecera('nuevo')}}">Cabecera</header>
  <main>
    <h1 th:text="${tallerFormulario.id == 0} ? 'Nuevo taller' : 'Editar taller'">Formulario</h1>
    <form th:action="@{/talleres/guardar}" th:object="${tallerFormulario}" method="post">
      <input type="hidden" th:field="*{id}" />
      <label>Nombre
        <input type="text" th:field="*{nombre}" th:errorclass="campo-error" required minlength="3" maxlength="60" />
        <span th:if="${#fields.hasErrors('nombre')}}" th:errors="*{nombre}" class="error"></span>
      </label>
      <label>Nivel
        <select th:field="*{nivel}" th:errorclass="campo-error" required>
          <option value="">Elige un nivel</option>
          <option value="básico">Básico</option>
          <option value="medio">Medio</option>
          <option value="avanzado">Avanzado</option>
        </select>
        <span th:if="${#fields.hasErrors('nivel')}}" th:errors="*{nivel}" class="error"></span>
      </label>
      <label>Plazas
        <input type="number" th:field="*{plazas}" th:errorclass="campo-error" required min="1" />
        <span th:if="${#fields.hasErrors('plazas')}}" th:errors="*{plazas}" class="error"></span>
      </label>
      <label>Correo de contacto
        <input type="email" th:field="*{contacto}" th:errorclass="campo-error" required />
        <span th:if="${#fields.hasErrors('contacto')}}" th:errors="*{contacto}" class="error"></span>
      </label>
      <button type="submit">Guardar</button>
    </form>
  </main>
  <footer th:replace="~{fragmentos/comun :: pie}">Pie</footer>
</body>
</html>

```

COMPRUEBA QUE FUNCIONA

Un envío con datos inválidos rebota con sus cuatro mensajes; uno correcto se guarda y redirige:

```

POST /talleres/guardar (id=0&nombre=a&nivel=&plazas=-2&contacto=hola) → 200 (rebota, con errores)
POST /talleres/guardar (id=0&nombre=Costura creativa&nivel=básico&plazas=15&contacto=info@centro.es) → 302 Location: /talleres

```

Repetir ese último POST (como un F5) no duplicaría el taller: la última petición que recuerda el navegador es el GET /talleres de la redirección.

Práctica

Tres formas de ampliar la aplicación completa, cada una añadiendo una pieza nueva sobre lo que ya funciona.

ENTRENAMIENTO 7.1

Ordenar la lista

Nivel · Fácil

Añade a `/talleres` un parámetro `orden` (nombre o plazas, con nombre por defecto) que ordene la tabla por ese campo, con dos enlaces arriba para cambiarlo.

PISTAS

- `List.sort(Comparator.comparing(Taller::getNombre))` (o `Taller::getPlazasLibres`) ordena la lista filtrada antes de añadirla al modelo.
- Los dos enlaces son como los de nivel: `@{/talleres(nivel=${nivel}, orden='plazas')}`, para no perder el filtro activo.

RESULTADO ESPERADO

Con `?orden=plazas`, la tabla empieza por «Fotografía con el móvil» (0 plazas libres) y sigue por las demás de menor a mayor.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java` (método `lista`):

```
@GetMapping("/talleres")
public String lista(Model model, @RequestParam(required = false, defaultValue = "todos") String nivel,
    @RequestParam(required = false, defaultValue = "nombre") String
orden) {
    List<Taller> filtrados = new ArrayList<>();
    for (Taller t : talleres) {
        if (nivel.equals("todos") || t.getNivel().equals(nivel)) {
            filtrados.add(t);
        }
    }
    if (orden.equals("plazas")) {
        filtrados.sort(Comparator.comparing(Taller::getPlazasLibres));
    } else {
        filtrados.sort(Comparator.comparing(Taller::getNombre));
    }
    model.addAttribute("talleres", filtrados);
    model.addAttribute("nivel", nivel);
    model.addAttribute("orden", orden);
    return "talleres/lista";
}
```

Fichero `src/main/resources/templates/talleres/lista.html`:

```
Ordenar por:
<a th:href="@{/talleres(nivel=${nivel}, orden='nombre')}" th:classappend="${orden ==
'nombre'} ? 'activa'">Nombre</a>
<a th:href="@{/talleres(nivel=${nivel}, orden='plazas')}" th:classappend="${orden ==
'plazas'} ? 'activa'">Plazas</a>
```

ENTRENAMIENTO 7.2

Borrar un taller

Nivel · Intermedio

Añade un enlace «Borrar» en cada fila de la lista y en la ficha, que quite el taller de la lista en memoria y redirija a `/talleres`.

PISTAS

- Un enlace normal es un GET: borrar con un enlace repite el problema de la sección 5 (recargar la página lo repetiría). Usa un formulario con `@PostMapping`, como cualquier otra escritura.
- `List.removeIf(t -> t.getId() == id)` quita el taller por su id.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
@PostMapping("/talleres/borrar/{id}")
public String borrar(@PathVariable int id) {
    talleres.removeIf(t -> t.getId() == id);
    return "redirect:/talleres";
}
```

Fichero `src/main/resources/templates/talleres/lista.html`:

```
<a th:href="@{/talleres/editar/{id}(id=${taller.id})}">Editar</a>
<form th:action="@{/talleres/borrar/{id}(id=${taller.id})}" method="post" class="en-
linea">
    <button type="submit">Borrar</button>
</form>
```

Comprueba que funciona. `POST /talleres/borrar/2` quita «Fotografía con el móvil» de la lista, con `redirect: a /talleres`; repetir la petición (como un F5 tras la redirección) no borra nada más, porque la última petición que recuerda el navegador es el GET de la lista.

ENTRENAMIENTO 7.3

Contador de plazas totales

Nivel · Difícil

Añade a la cabecera común un contador con las plazas libres de todos los talleres sumadas, visible en las tres páginas, que se actualice cuando se da de alta o se edita un taller.

PISTAS

- Un método `@ModelAttribute` en `TallerController` se ejecuta antes de cada uno de sus métodos: puede añadir el total al modelo sin repetirlo en cada uno.
- El fragmento `cabecera` necesita un parámetro nuevo, o leer el dato con `${plazasLibresTotal}` si lo añades con ese nombre fijo desde el `@ModelAttribute`.

UNA SOLUCIÓN

Fichero `src/main/java/com/ejemplo/primerproyecto/controller/TallerController.java`:

```
@ModelAttribute("plazasLibresTotal")
public int plazasLibresTotal() {
    int total = 0;
    for (Taller t : talleres) {
        total += t.getPlazasLibres();
    }
    return total;
}
```

Fichero `src/main/resources/templates/fragmentos/comun.html`:

```
<a th:href="@{/talleres}" class="marca">Talleres del centro</a>
<span class="contador" th:text="${plazasLibresTotal} plazas libres">0 plazas
libres</span>
```

Comprueba que funciona. El contador (13 plazas libres) aparece igual en `/talleres` y en `/talleres/1`, porque `@ModelAttribute` se ejecuta antes de cualquier método del controlador. Tras dar de alta un taller de 15 plazas, sube a 28: el `@ModelAttribute` recalcula el total en cada petición, sin guardar un número aparte que se pudiera desincronizar.

PÍLDORA

Toda la unidad en un vistazo

1. Un proyecto de Spring Boot: Spring Web, Thymeleaf, DevTools (y Validation si hay formularios).
2. Controladores con `@GetMapping`/`@PostMapping`, que leen y escriben el modelo.
3. Plantillas con `th:text`, condiciones, `th:each` y fragmentos, para no repetir HTML.
4. Enlaces con `@{...}`, con segmentos y parámetros, para navegar y filtrar.
5. Formularios con `th:object`/`th:field`, `redirect:` tras guardar, y `@Valid`/`BindingResult`/`th:errors` para validarlos.

AUTOEVALUACIÓN**Aplicación completa**

- 7.1** En esta aplicación, ¿qué ruta atiende `InicioController` y qué hace?
- 7.2** ¿Por qué `talleres` es un campo de `TallerController` y no una variable local de cada método?
- 7.3** ¿Qué plantilla usa `th:each` y para qué?
- 7.4** ¿Qué diferencia hay entre las rutas `/talleres/{id}` y `/talleres?nivel=medio`?
- 7.5** En `ficha.html`, ¿qué pasa si `taller` es `null` (un id que no existe)?
- 7.6** ¿Qué dos clases usa el formulario, y por qué no es la misma para las dos cosas?
- 7.7** ¿Qué distingue un alta de una edición en `guardar`?
- 7.8** Ordena estos tres pasos de `guardar` como ocurren de verdad: (a) `comprobar resultado.hasErrors()`, (b) `@Valid` valida el objeto, (c) `redirect:/talleres`.
- 7.9** ¿Por qué el formulario devuelve la propia vista ("`talleres/formulario`") cuando hay errores, en vez de un `redirect`?
- 7.10** ¿Qué fragmento reutilizan las tres plantillas de la aplicación, y con qué parámetro?
- 7.11** Si `TallerController` no tuviera `@Controller`, ¿qué pasaría al pedir cualquier ruta de `/talleres`?
- 7.12** ¿Qué comprobación falta si se quita `@Min(value = 1, ...)` de `plazas` en `TallerFormulario`?

Respuestas al final de la unidad, en «Autoevaluación · Aplicación completa».

Ejercicios complementarios

Ejercicios para practicar más, por sección. En el PDF salen sin solución y las soluciones van al final de la unidad.

Sección 1 · Del servidor al navegador

EJERCICIO C.1

Un banner propio

Nivel · Fácil

Sustituye el logotipo de Spring que sale en la consola al arrancar por uno propio con el nombre de tu aplicación y la versión de Spring Boot que usa. La versión no se escribe a mano: la pone Spring Boot.

PISTAS

- Un fichero `banner.txt` en `src/main/resources/` sustituye al logotipo.
- Dentro, `${spring-boot.version}` se cambia por la versión de Spring Boot.

Solución del ejercicio C.1 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.2

Todo bajo /talleres

Nivel · Intermedio

Haz que toda la aplicación cuelgue de la ruta `/talleres`: su portada estática debe verse en `http://localhost:8080/talleres/` y `http://localhost:8080/` debe dejar de responder con la aplicación.

PISTAS

- La propiedad es `server.servlet.context-path`.
- Busca en la consola la línea `Tomcat started on port 8080 (http) with context path ...`.

Solución del ejercicio C.2 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.3

Tu propia página 404

Nivel · Intermedio

Cambia la página «Whitelabel Error Page» que sale al pedir una ruta que no existe por una página tuya, en español, con un enlace a la portada.

PISTAS

- Spring Boot busca una página para cada código de error en la carpeta `error/`, con el código como nombre.
- Como no necesita datos del servidor, puede ser un fichero estático.

Solución del ejercicio C.3 en «Soluciones de los ejercicios complementarios».

Sección 2 · Controladores, rutas y vistas

EJERCICIO C.4

Nombre y apellidos

Nivel · Fácil

Haz que `/hola/Ana/Ruiz Pérez` muestre «Hola, Ana Ruiz Pérez», con la plantilla `saludo/saludo.html` de la sección. ¿Qué responde `/hola/Ana` si solo existe esta ruta?

Solución del ejercicio C.4 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.5

Un prefijo para todo el controlador

Nivel · Intermedio

Todas las rutas de `SaludoController` deben empezar por `/saludos`: `/saludos/hola` y `/saludos/adios`. Hazlo sin escribir `/saludos` en cada `@GetMapping`, y comprueba qué responde ahora `/hola`.

PISTAS

- `@RequestMapping("/prefijo")` sobre la clase se antepone a las rutas de todos sus métodos.

Solución del ejercicio C.5 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.6

Busca los errores

Nivel · Difícil

La aplicación se publica bajo /app y la plantilla está en src/main/resources/templates/inicio.html. Tiene tres errores. Di qué síntoma produce cada uno y corrígelos.

Fichero InicioController.java:

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.web.bind.annotation.GetMapping;

public class InicioController {

    @GetMapping({ "/", "/inicio" })
    public String inicio() {
        return "inicio";
    }

}
```

Fichero inicio.html:

```
<link rel="stylesheet" href="/css/estilos.css" />
```

Solución del ejercicio C.6 en «Soluciones de los ejercicios complementarios».

Sección 3 · Contenido dinámico

EJERCICIO C.7

Apodo y tutor

Nivel · Fácil

Con apodo y tutor a null en el modelo, escribe una plantilla que muestre «sin apodo» y «sin tutor asignado» sin que la página falle.

Solución del ejercicio C.7 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.8

Nivel medio, mal escrito

Nivel · Intermedio

Esta línea de una plantilla no compila. ¿Qué falla y cómo se corrige?

```
<p th:if="${taller.nivel = 'medio'}">Nivel medio</p>
```

Solución del ejercicio C.8 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.9**Tres erratas, tres páginas**

Nivel · Difícil

Cada una de estas plantillas tiene una sola errata. Localízala a partir del error que da la aplicación y corrígela.

Solución del ejercicio C.9 en «Soluciones de los ejercicios complementarios».

Sección 4 · Listas y parámetros**EJERCICIO C.10****Índice y posición**

Nivel · Fácil

Con `List.of("Ana", "Bruno", "Carla")`, escribe una lista que muestre, para cada alumno, su índice y su posición: «índice 0, posición 1: Ana».

Solución del ejercicio C.10 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.11**Dos parámetros en el mismo enlace**

Nivel · Intermedio

Un enlace debe ordenar por precio en orden descendente: `campo=precio` y `direccion=desc` a la vez, sobre la ruta `/orden`. Escribe ese enlace con la sintaxis nativa de Thymeleaf.

Solución del ejercicio C.11 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.12**Un parámetro con el tipo equivocado**

Nivel · Difícil

`/paginas?pagina=dos` se pide sobre un método con `@RequestParam(required = false, defaultValue = "1") int pagina`. ¿Qué código de estado responde la aplicación, y por qué el `defaultValue` no evita el problema?

Solución del ejercicio C.12 en «Soluciones de los ejercicios complementarios».

Sección 5 · Formularios

EJERCICIO C.13

Contador sin GET

Nivel · Fácil

Rehaz el contador de favoritos del principio de la sección para que incremente con un formulario y `@PostMapping`, en vez de con un enlace GET, con `redirect:` al terminar.

Solución del ejercicio C.13 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.14

Un campo sin set

Nivel · Intermedio

Producto tiene `getPrecio()` pero no `setPrecio(...)`. El formulario tiene `<input th:field="*{precio}">` y se envía con `precio=2.5`. ¿Da error la aplicación? ¿Qué valor queda en `precio`?

Solución del ejercicio C.14 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.15

Editar sin el id

Nivel · Difícil

En el método `editar` de `TallerController`, el `TallerFormulario` se construye con `0` en vez de con `taller.getId()`. Sin tocar nada más, edita el taller «Robótica con Arduino» cambiando su nivel a avanzado. ¿Qué ocurre?

Solución del ejercicio C.15 en «Soluciones de los ejercicios complementarios».

Sección 6 · Validación de formularios

EJERCICIO C.16

@Email sin @NotBlank

Nivel · Fácil

Una clase con un solo campo, `email`, con `@Email` pero sin `@NotBlank`. ¿Qué responde el servidor si se envía `email=` (vacío)? ¿Y si se envía `email=hola`?

Solución del ejercicio C.16 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.17**Todos los errores a la vez**

Nivel · Intermedio

Con el `TallerFormulario` de la sección (cuatro campos, cuatro restricciones), envía un formulario con los cuatro campos inválidos a la vez. ¿`BindingResult` para en el primer error que encuentra, o recoge todos?

Solución del ejercicio C.17 en «Soluciones de los ejercicios complementarios».

EJERCICIO C.18**`BindingResult` mal colocado**

Nivel · Difícil

En `guardar`, se cuela un `Model model` entre el parámetro `@Valid` y `BindingResult`:

```
guardar(@Valid @ModelAttribute(...) TallerFormulario formulario, Model model, BindingResult result)
```

¿Arranca la aplicación? ¿Qué pasa al enviar un formulario con errores?

Solución del ejercicio C.18 en «Soluciones de los ejercicios complementarios».

Soluciones de los ejercicios complementarios

Ejercicio C.1 · Un banner propio

Fichero `src/main/resources/banner.txt`:

```
=====
Talleres del centro
Spring Boot ${spring-boot.version}
=====
```

Al arrancar, la consola empieza con el banner y la versión ya sustituida (Spring Boot 4.1.1).

Ejercicio C.2 · Todo bajo /talleres

Fichero `src/main/resources/application.properties`:

```
spring.application.name=primer-proyecto
server.servlet.context-path=/talleres
```

Con una portada en `static/index.html`, la consola muestra

Tomcat started on port 8080 (http) with context path '/talleres',

`http://localhost:8080/talleres/` sirve la portada y `http://localhost:8080/` responde con la página 404 de Tomcat, porque ya no hay ninguna aplicación en esa ruta.

Ejercicio C.3 · Tu propia página 404

Fichero `src/main/resources/static/error/404.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<title>Página no encontrada</title>
</head>
<body>
  <h1>Vaya, esa página no existe</h1>
  <p>Revisa la dirección o vuelve a la <a href="/">portada</a>.</p>
</body>
</html>
```

`http://localhost:8080/no-existe` muestra «Vaya, esa página no existe» con el enlace a la portada.

Ejercicio C.4 · Nombre y apellidos

Una ruta con dos variables, cada una con su `@PathVariable`. `/hola/Ana` responde con un 404, porque la ruta exige dos trozos.

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@Controller
public class SaludoController {

    @GetMapping("/hola/{nombre}/{apellidos}")
    public String holaCompleto(@PathVariable String nombre, @PathVariable String
apellidos, Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Hola, " + nombre + " " + apellidos);
        return "saludo/saludo";
    }

}
```

Ejercicio C.5 · Un prefijo para todo el controlador

`/hola` responde ahora con un 404: la ruta completa es `/saludos/hola`.

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/saludos")
public class SaludoController {

    @GetMapping("/hola")
    public String hola(Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Hola, mundo");
        return "saludo/saludo";
    }

    @GetMapping("/adios")
    public String adios(Model model) {
        model.addAttribute("titulo", "Saludos");
        model.addAttribute("mensaje", "Adiós, mundo");
        return "saludo/saludo";
    }

}
```

Ejercicio C.6 · Busca los errores

- **Falta** `@Controller` (y su `import`): Spring no registra la clase y `/app/` da un 404.
- `return "incio":` con `@Controller` puesto, la página da un error 500 «Error resolving template [incio]». Debe ser `return "inicio";`.
- `href="/css/estilos.css":` apunta fuera de `/app` y la página sale sin estilos. Debe ser `th:href="@{/css/estilos.css}"`, que genera `/app/css/estilos.css`.

Ejercicio C.7 · Apodo y tutor

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Perfil</title>
</head>
<body>
<p>Nombre: <span th:text="${nombre}">nombre</span></p>
<p>Apodo: <span th:text="${apodo} ?: 'sin apodo'">apodo</span></p>
<p>Tutor: <span th:text="${tutor?.nombre} ?: 'sin tutor asignado'">tutor</span></p>
</body>
</html>
```

Ejercicio C.8 · Nivel medio, mal escrito

Un solo `=` es una asignación, no una comparación, y `nivel` no tiene un método para escribirlo: «EL1010E: Property or field 'nivel' cannot be set on object of type 'Taller'». Se corrige con `==`:

```
<p th:if="${taller.nivel == 'medio'}">Nivel medio</p>
```

Ejercicio C.9 · Tres erratas, tres páginas

- **Fichero 1:** `${taller.plazaLibres}` en lugar de `${taller.plazasLibres}`. Error 500 «Property or field 'plazaLibres' cannot be found».
- **Fichero 2:** `cabecra` en lugar de `cabecera`, en `th:replace`. Error 500 «Error resolving fragment».
- **Fichero 3:** `${taller.nivel = 'medio'}` en lugar de `${taller.nivel == 'medio'}`. Error 500 «Property or field 'nivel' cannot be set».

Ejercicio C.10 · Índice y posición

```
<!DOCTYPE html>
```

```
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <ul>
    <li th:each="alumno, est : ${alumnos}" th:text="Índice ${est.index}, posición $
      {est.count}: ${alumno}|"></li>
  </ul>
</body>
</html>
```

Ejercicio C.11 · Dos parámetros en el mismo enlace

```
<a th:href="@{/orden(campo='precio', direccion='desc')}">Por precio, descendente</a>
```

Genera `/orden?campo=precio&direccion=desc`.

Ejercicio C.12 · Un parámetro con el tipo equivocado

Responde 400, no 200: «Failed to convert value of type 'java.lang.String' to required type 'int'; For input string: "dos"». El `defaultValue` solo actúa cuando el parámetro **falta**; aquí `pagina` está presente, pero su valor no se puede convertir a `int`. Para admitir cualquier texto sin fallar habría que declarar el parámetro como `String` y convertirlo a mano, comprobando el resultado.

Ejercicio C.13 · Contador sin GET

```
package com.ejemplo.primerproyecto.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class FavoritoController {

    private int veces = 0;

    @GetMapping("/favorito")
    public String favorito(Model model) {
        model.addAttribute("veces", veces);
        return "favorito";
    }

    @PostMapping("/favorito/sumar")
    public String sumar() {
        veces++;
        return "redirect:/favorito";
    }
}
```

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<body>
  <p th:text="|Marcado como favorito ${veces} veces.|"></p>
  <form th:action="@{/favorito/sumar}" method="post">
    <button type="submit">Marcar</button>
  </form>
</body>
</html>
```

Dos envíos del formulario, con una lectura de `/favorito` entre medias, dan «1 veces» y luego «2 veces»: cada `POST` cuenta una sola vez, y repetir el `GET /favorito` no suma nada.

Ejercicio C.14 · Un campo sin `set`

No da ningún error, ni siquiera un aviso en la consola: Spring simplemente no puede escribir un campo sin `set`, así que lo ignora en silencio. `precio` se queda en su valor por defecto, `0.0`, aunque el usuario haya escrito otro:

```
POST /producto/guardar (nombre=Cuaderno&precio=2.5) → 200
Nombre: Cuaderno, precio: 0.0
```

Es un error más traicionero que uno con excepción: la aplicación funciona, pero guarda datos incompletos sin que nadie lo note. Por eso conviene comprobar, al escribir una clase para un formulario, que todos sus campos tienen `get` y `set`.

Ejercicio C.15 · Editar sin el `id`

Se envía el formulario con `id=0`, así que `guardar` lo trata como un alta: crea un taller nuevo con los datos escritos, con un `id` distinto, y deja intacto el que se quería editar.

```
POST /talleres/guardar (id=0&nombre=Robótica editada&nivel=avanzado&plazas=8) → 302
```

Tabla resultante:

Robótica con Arduino	medio	16	
Fotografía con el móvil	básico	20	
Robótica editada	avanzado	8	← taller nuevo, no la edición esperada

Por eso el `id` real del taller que se edita tiene que ir siempre en el formulario, en el campo oculto: sin él, no hay forma de distinguir una edición de un alta.

Ejercicio C.16 · @Email **sin** @NotBlank

@Email sola no exige que el campo tenga contenido, solo que, **si lo tiene**, tenga forma de correo. Un valor vacío la pasa sin problema:

```
POST /contacto/comprobar (email=) → ¿Tiene errores? false
POST /contacto/comprobar (email=hola) → ¿Tiene errores? true
```

Para exigir que el campo no esté vacío hace falta añadir @NotBlank además de @Email, como en TallerFormulario.contacto.

Ejercicio C.17 · Todos los errores a la vez

Los recoge todos: resultado.getErrorCount() y resultado.getFieldErrors() devuelven los cuatro de una vez, no solo el primero.

```
POST /talleres/comprobar (id=0&nombre=a&nivel=&plazas=-2&contacto=hola)
→ Errores: 4, en: nombre nivel plazas contacto
```

Por eso el formulario puede mostrar los cuatro mensajes de error a la vez, en una sola vuelta, en vez de descubrirlos uno a uno en sucesivos envíos.

Ejercicio C.18 · BindingResult **mal** colocado

La aplicación arranca sin problema: el error no aparece hasta el primer envío con datos inválidos. Ahí, en vez de rebotar al formulario con los mensajes, responde con un error 500:

```
POST /talleres/guardar (id=0&nombre=a&nivel=&plazas=-2&contacto=hola) → 400
MethodArgumentNotValidException: Validation failed for argument [0] ...
```

BindingResult solo recoge los errores si va justo después del parámetro que valida; con algo en medio, Spring no los asocia y deja que la validación fallida se convierta en una excepción.

Respuestas de la autoevaluación

Autoevaluación · Del servidor al navegador

- 1.1 b)** Una página que el servidor genera en cada petición, con datos que pueden cambiar. Lo que la hace dinámica es que el servidor la fabrica en el momento. Las animaciones y el JavaScript se ejecutan en el navegador y también existen en páginas estáticas.
- 1.2 b)** En el navegador, cuando la página ya ha llegado. El servidor lo envía como texto dentro del HTML y es el navegador quien lo ejecuta.
- 1.3** La del servidor se escribió una sola vez, al generar la página, y llega al navegador como texto fijo; para cambiarla hay que pedir la página otra vez. La del navegador la actualiza cada segundo el JavaScript de la página, sin preguntar al servidor.
- 1.4 c)** La hora ya escrita dentro de la etiqueta, sin ningún `th:text`. Thymeleaf sustituye el contenido y quita sus atributos al generar la página; al navegador solo le llega el HTML final.
- 1.5** Porque el código del navegador se puede ver, cambiar o saltar (se puede enviar una petición sin usar la página). El servidor es el único lado en el que se puede confiar.
- 1.6** Responde al instante, sin esperar a otra petición, y quita carga al servidor y tráfico a la red. Por ejemplo, un reloj, un menú desplegable o un aviso mientras se escribe.
- 1.7** JSP y Thymeleaf, Java; PHP, PHP; Razor, C# (ASP.NET Core); Jinja, Python.
- 1.8 b)** Una plantilla que sigue siendo HTML válido y se puede abrir en el navegador sin servidor, con valores de ejemplo. Es la idea de Thymeleaf: sus atributos `th:` los ignora el navegador y el contenido de ejemplo hace de relleno.
- 1.9 c)** `00:00:00` en la hora del servidor, y el reloj del navegador funcionando. Nadie sustituye `th:text`, así que queda el valor de ejemplo; el `<script>` sí se ejecuta, porque lo hace el navegador.
- 1.10** Spring Web, Thymeleaf y Spring Boot DevTools.
- 1.11 b)** Porque las fija el `parent`, `spring-boot-starter-parent`, con versiones compatibles entre sí. El proyecto hereda de `spring-boot-starter-parent`, que lleva las versiones de todo el ecosistema de Spring Boot.
- 1.12** Las plantillas, en `src/main/resources/templates/`; la hoja de estilos, en `src/main/resources/static/` (por ejemplo, `static/css/estilos.css`), que se sirve tal cual.
- 1.13 b)** Uno en `com.ejemplo.primerproyecto.controller`. Spring busca en el paquete de la clase principal y en sus subpaquetes. El otro compila, pero su página da un 404.
- 1.14** Que el servidor está en marcha y responde, pero en la ruta `/` todavía no hay ninguna página. Si la aplicación no estuviera arrancada, el navegador no podría conectar.

1.15 La aplicación ya estaba arrancada (u otra usa el 8080). Se para la que estaba en marcha y se vuelve a arrancar, o se cambia el puerto con `server.port`.

1.16 b) Recargar el navegador. DevTools desactiva la caché de Thymeleaf en desarrollo, así que la plantilla se vuelve a leer en la siguiente petición. Si cambias un `.java`, en cambio, la aplicación se reinicia sola.

Autoevaluación · Controladores, rutas y vistas

2.1 c) El controlador. El controlador recibe la petición, prepara los datos en el modelo y devuelve el nombre de la vista.

2.2 La anotación `@Controller` y estar en el paquete base de la aplicación o en uno de sus subpaquetes.

2.3 b) `templates/alumnos/lista.html` El nombre de la vista es la ruta dentro de `templates/`, sin la extensión `.html`.

2.4 Con las dos rutas en la anotación: `@GetMapping({ "/", "/inicio" })`.

2.5 b) El método devuelve el nombre de una plantilla que no existe. El controlador sí se ejecuta, pero Thymeleaf no encuentra `templates/incio.html`. Hay una errata en el nombre o la plantilla está en otra carpeta.

2.6 Que a la clase le falte `@Controller`, que esté fuera del paquete base o que la ruta de `@GetMapping` no sea `/hola`. Ninguna de ellas es un error de compilación.

2.7 b) Declarándolo como parámetro del método: Spring lo proporciona. Basta con escribir `Model model` entre los parámetros.

2.8 c) El `<h1>` vacío. Un atributo que no existe no da error: la etiqueta sale vacía. Hay que revisar que los nombres coincidan.

2.9 Sustituye el contenido de la etiqueta por el valor del atributo `titulo` del modelo, así que la pestaña del navegador muestra ese texto.

2.10 b) `/img/logo.png` Lo que hay dentro de `static/` se sirve desde la raíz de la aplicación, sin controlador.

2.11 Porque `@{...}` añade el prefijo con que se publique la aplicación (por ejemplo `/app/hola`). Un `href` escrito a mano apuntaría fuera de la aplicación si no está en la raíz.

2.12 b) Con un `href` normal: es una dirección externa. Las direcciones externas no dependen de dónde se publique tu aplicación, así que basta con `href`.

2.13 "Pedro". Con `/hola/Pedro/Ruiz` la ruta no encaja, porque una variable ocupa un solo trozo sin `/`, y responde con un 404.

2.14 b) Un error 400: "veinte" no se puede convertir a `int`. La ruta encaja, pero Spring no puede convertir el texto al tipo del parámetro y responde «Bad Request».

2.15 Escribiendo el nombre de la variable en la anotación: `@PathVariable("nombre") String nombre`.

2.16 b) Antes de cada método manejador de su controlador. Se ejecuta en cada petición que atiende ese controlador y deja su resultado en el modelo con el nombre `titulo`.

2.17 Con un único método `@ModelAttribute("centro")` que devuelva "CEU FP Sevilla" y quitando las tres líneas repetidas.

Autoevaluación · Contenido dinámico

3.1 b) El texto `<b>Ojo</b>`, con las etiquetas visibles. `th:text` convierte `<` y `>` en texto visible. Con `th:utext` saldría «Ojo» en negrita.

3.2 Solo con HTML que has escrito tú y sabes que es seguro. Nunca con lo que escribe un usuario.

3.3 b) El script se ejecuta en el navegador de quien abre la página. Es un ataque XSS: el navegador no distingue el script del atacante del código de la página.

3.4 b) `getPlazasLibres()` Thymeleaf antepone `get` al nombre de la propiedad y pone la primera letra en mayúscula. Para un `boolean`, el método se llama `isXxx`.

3.5 `${taller.completo}`.

3.6 c) Error 500: «Property or field 'nombre' cannot be found on null». No se puede leer una propiedad de `null`. Se evita comprobando antes con `th:if` o leyendo con `?.`.

3.7 `'Plazas: ' + ${taller.plazasLibres}`, o con sustitución de literales, `|Plazas: ${taller.plazasLibres}|`.

3.8 b) `false` 12 no es mayor ni igual que 16.

3.9 «Con plazas». Es una condición con dos valores: el de antes de los dos puntos si es verdadera y el de después si es falsa.

3.10 El primero es una asignación y da error («EL1010E ... cannot be set»); el segundo compara y devuelve `true` o `false`.

3.11 c) `25.0` Es el `double` tal cual. Para dos decimales con coma hay que usar `#numbers.formatDecimal(taller.precio, 1, 2, 'COMMA')`.

3.12 `th:if` genera la etiqueta si la condición es verdadera; `th:unless`, si es falsa.

3.13 c) No: no está en el HTML que recibe el navegador. Thymeleaf no genera la etiqueta. Con «Ver código fuente» no aparece.

3.14 Recoge todos los valores que no coinciden con ningún otro `th:case`; hace de caso por defecto.

3.15 «sin apodo». El operador `?:` usa el valor de la derecha cuando el de la izquierda es `null`.

3.16 b) Para marcar un trozo de plantilla con nombre que otras plantillas pueden reutilizar. Se define una vez, en su propio fichero, y se usa con `th:replace` o `th:insert`.

3.17 `th:replace` sustituye la etiqueta por el fragmento; `th:insert` deja la etiqueta y mete el fragmento dentro de ella.

3.18 Que la ruta del fichero y el nombre del fragmento en `th:replace` coinciden con los de `th:fragment` (una errata, por ejemplo `cabecra`, produce este error).

Autoevaluación · Listas y parámetros

4.1 b) La repite una vez por cada elemento de `talleres`. Genera una copia de la etiqueta (con todo su contenido) por cada elemento de la lista.

4.2 `est.index` empieza en 0 (como la posición en la lista); `est.count` empieza en 1 (como un número de fila para mostrar).

4.3 b) El caso en que `talleres` sea `null`. `talleres.size()` falla si `talleres` es `null`; `#lists.isEmpty(...)` no.

4.4 `/talleres/3`.

4.5 b) `/talleres?nivel=medio` Sin llaves en la ruta, lo que va entre paréntesis se añade como parámetro tras el `?`.

4.6 `@{/paginas(pagina=${siguiente}, tamano=${tamano})}`.

4.7 `@PathVariable` va en un segmento de la ruta e identifica un recurso, obligatorio por defecto. `@RequestParam` va tras el `?`, para filtrar o ajustar el resultado, y puede ser opcional con un valor por defecto.

4.8 c) "todos" Al faltar el parámetro, se usa `defaultValue`.

4.9 Porque `texto` vale `""`, y `nombre.contains("")` es siempre verdadero: ningún taller queda descartado.

4.10 Sin `required = false` ni `defaultValue`, `@RequestParam` es obligatorio: la petición daría un error 400 al faltar el parámetro.

4.11 Para que una página fuera de rango (0, negativa o mayor que el total) no produzca un `subList` con índices inválidos, que lanzaría una excepción.

4.12 Para no mostrar un enlace a una página anterior a la primera; en la página 1 no tiene sentido.

4.13 b) No genera la tabla. `#lists.isEmpty(null)` devuelve `true`, así que `th:unless` no genera la etiqueta.

4.14 `<tr th:each="taller, est : ${talleres}" th:class="${est.even} ? 'par' : 'impar'">`.

4.15 Sirve para lo mismo, construir la URL. Es más frágil: hay que escribir a mano las barras y, con varios parámetros, los signos `?` y `&`, y es fácil equivocarse.

4.16 Genera un `<option>` por cada animal de la lista, igual que con cualquier otra etiqueta.

Autoevaluación · Formularios

- 5.1** Porque un GET se puede repetir sin que el usuario lo decida (al recargar, al volver atrás) y el navegador no avisa; el contador subiría sin que nadie lo pidiera de verdad.
- 5.2 b)** Que repetirlo no cambia el estado del servidor. Pedir la misma página varias veces debe dejar los datos igual que si se hubiera pedido una sola.
- 5.3** Fija el objeto sobre el que trabajan los campos del formulario, para que `th:field="*{...}"` pueda referirse a sus propiedades.
- 5.4** Selecciona la opción cuyo valor coincide con la propiedad `nivel` del objeto al mostrar el formulario, y al enviarlo asigna a esa propiedad el valor elegido.
- 5.5 b)** Un constructor sin parámetros y `get/set` para cada campo. Spring crea el objeto con el constructor vacío y usa los `set` para rellenarlo con lo que llega del formulario.
- 5.6** `List.of(...)` crea una lista fija, que no admite añadir ni quitar elementos; `new ArrayList<>(List.of(...))` copia esos elementos en una lista que sí se puede modificar.
- 5.7** Porque `Taller` no tiene métodos `set`, es inmutable: para cambiar sus datos hay que crear otro objeto y sustituirlo en la lista.
- 5.8 b)** Responde con un 302 y la cabecera `Location`; el navegador hace un GET a esa ruta. Es una redirección HTTP; el navegador hace una nueva petición, esta vez GET.
- 5.9** P (Post): se envía el formulario. R (Redirect): el servidor responde con un 302. G (Get): el navegador pide la página final con un GET.
- 5.10 b)** El navegador repite el POST y el taller se da de alta otra vez. La última petición que hizo el navegador fue el POST; F5 la repite y duplica el alta.
- 5.11** Un GET a la lista de talleres (`/talleres`), no el POST de guardar: repetirla con F5 no cambia nada.
- 5.12** No: avisan al recargar una página que llegó por POST, pero no si esa petición fue un GET (como en el ejemplo del contador) ni en otras situaciones donde se repite automáticamente.
- 5.13 b)** El valor del campo oculto `id: 0` para alta, el `id` real para edición. El mismo método `guardar` decide si añade un taller nuevo o sustituye uno existente según si `id` vale 0 o no.
- 5.14** Porque no es un dato que la persona que rellena el formulario deba escribir ni cambiar: lo pone el controlador al mostrar el formulario y viaja con el resto de campos al enviarlo.
- 5.15** `public String anadir(@ModelAttribute("producto") Producto producto) { ... }`, con `@PostMapping` en la ruta correspondiente.

Autoevaluación · Validación de formularios

- 6.1** Se guarda igual, sin ningún aviso: nada comprueba los datos antes de guardarlos.

6.2 b) El proyecto no compila. Las clases de `jakarta.validation.constraints` no están en el proyecto sin ese starter, así que ni siquiera compila.

6.3 Que su longitud esté entre 3 y 60 caracteres, ambos incluidos.

6.4 c) `@Min(1)` es la que se ha visto en la unidad para poner un mínimo a un número.

6.5 Le dice a Spring que valide ese objeto con Bean Validation antes de ejecutar el método, comprobando todas sus anotaciones de restricción.

6.6 b) Para recoger los errores de validación sin que se lance una excepción. Si no estuviera, un error de validación lanzaría una excepción en vez de dejar que el método decida qué responder.

6.7

`public String guardar(@Valid @ModelAttribute("tallerFormulario") TallerFormulario formulario, BindingResult result)` el `Model` va después del `BindingResult`, nunca entre `@Valid` y `BindingResult`.

6.8 c) Al enviar datos inválidos, la aplicación responde con un error 500 en vez de rebotar al formulario. Spring pierde la asociación entre el objeto y su `BindingResult`, y un envío inválido lanza `MethodArgumentNotValidException` (500), en vez de volver al formulario con los errores.

6.9 El `<span>` solo se genera cuando el campo `nombre` tiene algún error, y `th:errors` escribe dentro ese mensaje.

6.10 Nada, si el campo no tiene errores; si los tiene, añade la clase `campo-error` al `<input>` (para, por ejemplo, ponerle un borde rojo por CSS).

6.11 Porque el objeto con esos valores (aunque inválidos) sigue en el modelo, y `th:field` lo usa para rellenar el campo, igual que en cualquier formulario.

6.12 En `src/main/resources/`, con ese nombre exacto. `message` lleva la clave entre llaves, por ejemplo `message = "{taller.nombre.notblank}"`.

6.13 b) No: el navegador bloquea el envío antes de hacer la petición. La validación HTML5 impide que el formulario se envíe; no se genera ninguna petición.

6.14 No: `required` solo lo comprueba el navegador cuando envía el propio formulario. Una petición hecha por otro medio no pasa por esa comprobación, así que solo la validación del servidor (Bean Validation) la detiene.

6.15 La del cliente da una respuesta inmediata, sin esperar al servidor, y mejora la experiencia. La del servidor es la única que de verdad garantiza los datos, porque no se puede evitar enviando la petición de otra forma. Con solo la del cliente, los datos no están protegidos de verdad; con solo la del servidor, cada error exige un viaje de ida y vuelta.

Autoevaluación · Aplicación completa

7.1 Atiende `/` y responde con `redirect:/talleres`: no tiene plantilla propia, solo redirige a la lista.

7.2 Porque tiene que sobrevivir entre peticiones: es el estado en memoria de la aplicación, compartido por todas las peticiones mientras el controlador (un singleton) esté vivo.

7.3 talleres/lista.html, para generar una fila de la tabla por cada taller de la lista (ya filtrada por nivel).

7.4 La primera usa @PathVariable para identificar un taller concreto; la segunda usa @RequestParam para filtrar la lista, es opcional y tiene un valor por defecto (todos).

7.5 El <main th:if="{taller != null}"> no se genera, y en su lugar aparece el <main th:unless="{taller != null}"> con «Taller no encontrado»: la página no falla.

7.6 Taller (los datos guardados, con plazasLibres y completo calculados) y TallerFormulario (solo lo que se edita, con las anotaciones de validación). Separarlas evita mezclar campos que no se editan (como inscritos o fecha) con los del formulario.

7.7 El campo oculto id: 0 para alta (se crea un Taller nuevo), cualquier otro valor para edición (se sustituye el Taller de ese id en la lista).

7.8 b, a, c: primero se valida (antes de entrar en el cuerpo del método), luego se comprueba si hubo errores, y solo si no los hay se llega al redirect: .

7.9 Porque necesita mostrar los mensajes de error y los valores que escribió el usuario, que están en el objeto que llegó con el POST; un redirect: perdería esos datos, al ser una petición GET nueva.

7.10 fragmentos/comun :: cabecera(activa), pasando 'talleres' o 'nuevo' para marcar en el menú la página en la que se está.

7.11 Un error 404: Spring no lo registraría como controlador y ninguna de sus rutas respondería.

7.12 Se podrían guardar talleres con plazas negativas o cero, como en el ejemplo del principio de la sección 6, sin que nadie lo impida.

Bibliografía

Documentación oficial de las herramientas de la unidad. Casi toda en inglés, salvo la de MDN.

- Broadcom (VMware Tanzu). *Spring Boot Reference Documentation*. Documentación oficial de Spring Boot, en inglés.
<https://docs.spring.io/spring-boot/index.html>
- Broadcom (VMware Tanzu). *Web on Servlet Stack (Spring MVC)*. Referencia oficial de Spring Web MVC, en inglés.
<https://docs.spring.io/spring-framework/reference/web/webmvc.html>
- Broadcom (VMware Tanzu). *Spring Initializr*. El servicio que usa el asistente de STS para crear proyectos, en inglés.
<https://start.spring.io/>
- Broadcom (VMware Tanzu). *Spring Tools*. Descarga y documentación de Spring Tool Suite, en inglés.
<https://spring.io/tools>
- Thymeleaf. *Documentation*. Página de documentación oficial, con enlace a todos sus tutoriales, en inglés.
<https://www.thymeleaf.org/documentation.html>
- Fernández, D. y equipo de Thymeleaf. *Tutorial: Using Thymeleaf*. El tutorial de referencia del motor de plantillas, en inglés.
<https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>
- Fernández, D. y equipo de Thymeleaf. *Tutorial: Thymeleaf + Spring*. Cómo se integra Thymeleaf con Spring MVC, en inglés.
<https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>
- Eclipse Foundation. *Jakarta Bean Validation 3.0*. La especificación oficial de las anotaciones de validación, en inglés.
<https://jakarta.ee/specifications/bean-validation/3.0/>
- Red Hat / Hibernate. *Hibernate Validator Documentation*. La implementación de Bean Validation que usa Spring Boot, en inglés.
<https://hibernate.org/validator/documentation/>
- Mozilla y colaboradores. *HTTP*. Documentación de HTTP (métodos, códigos de estado, cabeceras) en MDN, en español.
<https://developer.mozilla.org/es/docs/Web/HTTP>
- The Apache Software Foundation. *Introduction to the POM*. Guía oficial de Maven sobre el `pom.xml`, en inglés.
<https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>

Cumplimiento de RA y CE

Esta unidad desarrolla **un único resultado de aprendizaje**, el **RA8**.

Dentro de un RA, **todos los criterios de evaluación (CE) tienen el mismo peso**.

RA8. Genera páginas web dinámicas analizando y utilizando tecnologías y frameworks del servidor web que añadan código al lenguaje de marcas

- a) Se han identificado las diferencias entre la ejecución de código en el servidor y en el cliente web.
- b) Se han reconocido las ventajas de unir ambas tecnologías en el proceso de desarrollo de programas.
- c) Se han identificado las tecnologías y frameworks relacionadas con la generación por parte del servidor de páginas web con guiones embebidos.
- d) Se han utilizado estas tecnologías y frameworks para generar páginas web que incluyan interacción con el usuario.
- e) Se han utilizado estas tecnologías y frameworks, para generar páginas web que incluyan verificación de formularios.
- f) Se han utilizado estas tecnologías y frameworks para generar páginas web que incluyan modificación dinámica de su contenido y su estructura.
- g) Se han aplicado estas tecnologías y frameworks en la programación de aplicaciones web.

Cobertura de CE por sección:

CE / Sección	1	2	3	4	5	6	7
RA8.a	✓						
RA8.b	✓					✓	
RA8.c	✓	✓					
RA8.d		✓		✓	✓		
RA8.e						✓	
RA8.f			✓	✓			
RA8.g							✓

Cobertura de criterios de evaluación: **7 de 7**.